

Computer Science NEA:

2.5D Rugby Game

Table Of Contents:

Problem:	5
Research:	6
Stakeholders:	6
Interview:	6
Researching Other Games:	8
Rugby Nations 24	8
Conclusion	10
Retro Bowl	11
Conclusion	12
Rugby 08	13
Conclusion	13
What I will include from my research:	15
What I won't include in my game:	15
How did the stakeholders react to my research?	16
Conclusion:	17
Program Objectives	17
Modelling:	20
Key Rugby Mechanics:	20
Class Diagram:	21
Test Plan:	24
Movement and Control Tests	24
Passing Tests:	25
Defensive and Tackling Tests:	25
Ruck Tests:	26
Lineout Tests:	27
Scoring, Conversion and Kickoff Tests	27
Match Flow and Timer Tests:	28
FSM and State Transition Tests:	28
Input Mapping and Rebinding Tests:	29
UI Tests:	29
Robustness Test:	30
Design:	31
System Overview:	31
Human-Computer Interactions (Wireframes):	31
Gameplay:	32

HUD:	33
Rebind Menu:	34
Passing:	35
FSM Modelling:	35
GameManagerFSM:	36
Player Controlled FSM:	36
Defensive AI FSM:	37
DefensiveAIFSM(tackler):	38
RuckFSM:	38
LineoutFSM:	39
Pause FSM:	40
FSM Overview:	40
System Hierarchy:	42
Sequence Diagrams:	44
Possession Change Sequence:	44
Pass System Sequence:	44
Tackle to Ruck Sequence:	45
Pseudocode for Core Systems and Algorithms:	45
Core Match Flow:	45
Match Initialisation:	46
Possession Change Flow:	46
Tackle to Ruck Flow:	49
Ruck Lifecycle:	50
Ruck Completion Flow:	51
Try to Conversion To Kickoff Flow:	51
Ball In Touch to Lineout Flow:	52
Lineout Lifecycle (2 methods):	52
Algorithms:	55
Passing Algorithm:	55
Purpose:	55
How it works:	55
Pseudocode:	56
Pass Flow:	59
Formation Planning Algorithm:	59
Purpose:	59
How it works:	60
Pseudocode:	61

Defensive Pairing Algorithm:	64
Purpose:	64
How it works:	64
Pseudocode:	65
Defensive AI State Update:	66
File Structure:	68
Scene Structure:	69
Key Flowcharts:	69
Key Gameloop Flowchart:	70
My Code:	72
Technical Solution Overview:	72
Design Philosophy:	73
Variables Table	73
Data Structure Table	78
Key Algorithms	80
State Machine architecture	99
Human Player States	102
AI defensive States	119
Match Flow	134
Ball and passing implementation	152
Input System	164
Ruck Subsystems	179
Lineout Subsystem	189
Conversion Subsystem	200
Core Gameplay Entities	210
UI Classes	215
Configs	229
Testing and Evaluation:	233
Test Plan:	233
Movement and Control Tests	233
Passing Tests:	234
Defensive and Tackling Tests:	235
Ruck Tests:	235
Lineout Tests:	236
Scoring, Conversion and Kickoff Tests	236
Match Flow and Timer Tests:	237
FSM and State Transition Tests:	237

Input Mapping and Rebinding Tests:	238
UI Tests:	239
Robustness Test (Edge Case Testing):	239
Evaluation:	240
Overall Success:	240
Evaluating Against Program Objectives:	240
Reference Tests:	244
Feedback From Stakeholders:	244
Reflection On Feedback:	244
Future Implementations:	245
Conclusion:	245
Bibliography:	245
Appendix A:	246

Analysis

Problem:

Over the past decade, there has been a decrease in the quality of rugby video games. For example, games such as Rugby 22 or Rugby Challenge 4 aim for full simulation but are criticised by players for being overly realistic and taking the joy out of rugby. These games deter casual gamers from playing them, stopping them from developing a passion for the game like I have developed. In contrast to this, older games like Jonah Lomu Rugby (created in 1997) were able to recreate the fast paced, exciting parts of rugby and are fondly remembered by fans for its simplicity and fun gameplay. The only problem with it, is that it was built on what are now outdated systems which are scarcely available anymore. Games like Retro Bowl have also shown recently that there is an audience for retro styled games, however that game is made for American Football not rugby. After conducting this market research., I have highlighted that there is a gap in the market for a 2.5D retro styled rugby game where the main identity is fast paced, fluid rugby. This will be achieved though implementing most not all of the key systems and components of rugby (rucks, lineouts, tackling, tries and conversion). This game will also have simple arcade style controls with rebindable controls using JSON files to save across games. The game will be built for PC in Unity using C# with potential support for controllers if time allows.

Research:

Stakeholders:

The main stakeholders for my game are two distinct groups. The first is the casual gamers who just want a fun, intuitive game where they can play for 5-10 minutes and the second is the rugby enthusiasts who want longer game times up to 20 minutes as well as more realistic gameplay. I will try to create a balance between these two groups and have got the help of 3 stakeholders to give me a balanced set of opinions. In the casual gamer group I have _____ who enjoys quick and engaging games where you can sit down and play without needing to fully understand the rules of rugby. On the opposite end of the spectrum I have _____ who is an avid rugby player (he trains 4-6 times a week). He will be giving feedback on the realism and accuracy of the game and whether the key parts are implemented correctly. To balance these two groups I have _____ who sits in between both groups. He plays rugby for fun and enjoys sitting down and playing a game every now and then. His feedback will provide a balanced point of view in between helping to identify where realism needs to be adjusted to improve accessibility without ruining the core experience. By remaining engaged with all my stakeholders throughout the game development process, I will be able to ensure the game doesn't deviate from its intended path due to scope creep and other factors.

Interview:

The following interview questions were intended to identify stakeholder preferences and ensure that the planned features aligned with what the target audience would actually want from the final project.

- **How long should matches last?**

Findings:

Interviewees suggested 3-10 minute halves, depending on free playing time

Impact on game:

I will implement configurable half lengths in the setting menu which gives greater flexibility and keeps the game accessible for casual play but still can be immersive if players have time.

- **Would you prefer a more arcade style gameplay or realistic**

Findings:

All interviewees preferred a more realistic gameplay experience (rules, scoring, mechanics) but wanted arcade-style controls (simpler inputs, not overloaded with button combos)

Impact on game:

I will aim for hybrid approach having some key aspects of rugby while removing some of the slower more boring parts (kicking and scrums)

- **Which rugby mechanic do you most enjoy?**

Findings:

Stakeholders gave varied feedback, some preferred lineouts while others, tackling and rucks. These represent the core elements of rugby.

Impact on game:

I will prioritise implementing rucks, lineouts, tackling and conversions, making a funner, fast paced experience.

- **Do you like having player statistic upgrades?**

Findings:

Interviewees like the idea of upgrades but did not see it as essential for the core game. They felt it could be an optional extra mode (Career Mode)

Impact on game:

I will keep player attributes (speed, stamina, handling, aggression) in the main game and in the future, I will create upgrade/progression features of a Career mode. This would be due to the time constraints of the NEA

- **Would you prefer rugby 7's or 15's?**

Findings:

There was a split with some preferring 7's to 15's and the other preferring 15's to 7's.

Impact on game:

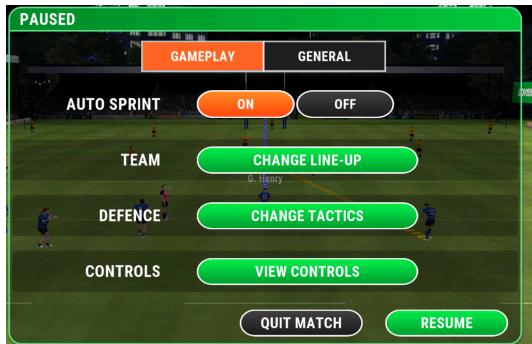
I will make a compromise by making 10's rugby with formations, forwards, backs and AI.

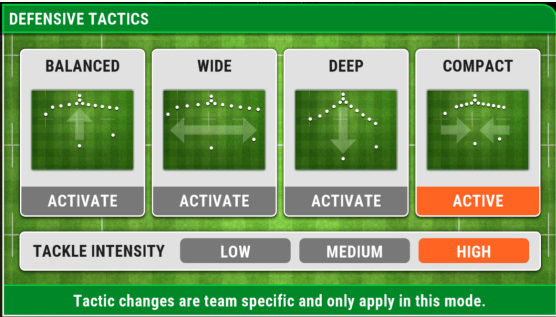
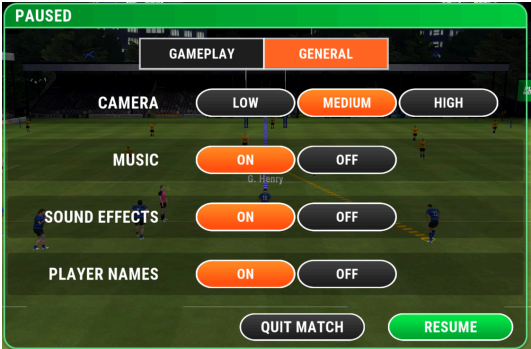
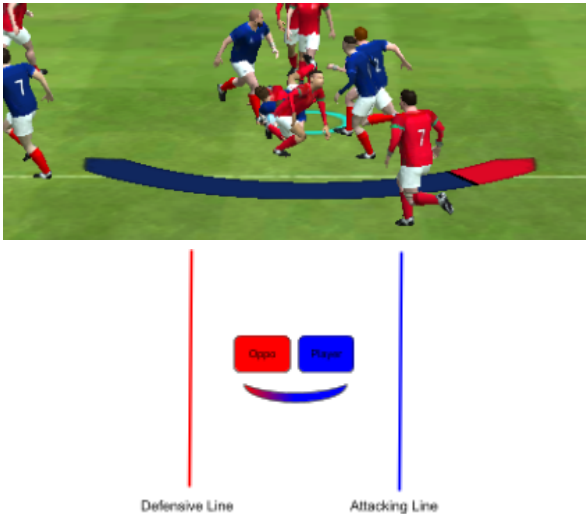
Overall, the interviews provided me with valuable information that has helped shape my design decisions. I will focus on creating a realistic 15 a side rugby game with simplified arcade style controls and some of the core mechanics (lineouts, rucks, tackling and conversions)). Having adjustable match lengths will make the game more flexible for both casual and dedicated players so, I will be including that as well. While player attributes will be included, advanced features like upgrades and Career mode will be developed after the NEA deadline as I continue to update

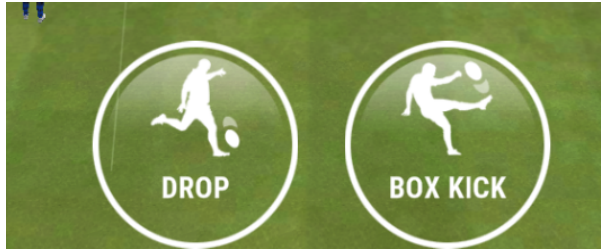
the game due to time constraints. This ensures the final project will be achievable while still aligning with the stakeholders' preferences.

Researching Other Games:

I analysed other games similar to my project in order to identify their strengths and weaknesses. This allowed me to decide which positive features I could incorporate into my game and which drawbacks I should avoid.

Rugby Nations 24	
Rugby Nations 24 uses colour-coded passing UI and I feel it provides clear visual representation on pass accuracy and range. I plan to implement something similar into my game as well as the highlighted circle to show which player is currently selected.	
In Rugby Nations 24, in the top left of the screen there is the scoreboard UI. I want to include this as it has the key information the player will need while keeping it simple like an arcade styled game.	
These are both tabs of the pause menu for Rugby Nations 24. I believe there is too much information for the player to have to deal with. Therefore, in my game I will simplify the pause menu so it only has a couple of the core features needed. This will keep it accessible for casual gamers. However, in the future, I may add some extra features such as defensive tactics (similar to picture below) to allow for hardcore rugby fans to enjoy an extra level of immersiveness and realism.	

	
This image shows a rust contest indicator from Rugby Nations 24. The different colours show the current possession between both teams. The bar shifts as more players join or as one side gains momentum. This gives players real time feedback on who's winning the ruck. I plan to implement something similar however I want it to be in a different position (as shown in image 2). This is not a fundamental part of the game and is more UI so I will add this after the project timeframe.	
Rugby Nations 24 uses a trigger plane along the goal line and touchline to detect the exact point where the ball crosses over the goal line/ tryline. I plan to implement this for my touchline and tryline. However, unlike Rugby Nations 24, I will use the tryline trigger plane only for the try scoring and use gameObjects for the	

This image shows the kick selection UI in Rugby Nations 24. The player can choose between different kick types by clicking on the button. However, since my game won't be made for mobile or won't have kicking, the buttons will have a symbol for the key that needs to be pressed and they will be for passing left or right instead.	
This image shows the kicking animation in Rugby Nations 24. It has three components to it. The first component is the parabolic curve going from the kicker to the landing spot. I plan on using a parabolic curve for my passing instead. The second is the landing sphere, this is to add some randomness and realism to the game as the further away you kick the ball, the lower the degree of accuracy. I will add this if the pass fails it will land in an area around the player to add some variability and to make it seem more realistic. The miss offset will increase with the pass distance so if a short pass fails it will land close to the target, if a long pass fails, it could land further from the target. I also plan to add a passing algorithm that calculates the probability of the pass, as it gets further away the probability decreases exponentially.	
In the image, there are simple buttons which can be pressed for throwing the ball to different positions from a lineout. I plan to implement this simple approach.	
Conclusion	
From Rugby Nations 24 I want to incorporate the kicking radius of accuracy into my passing where, if a pass fails it lands around the player in a certain area. I also want to incorporate the colour coded UI for the passing mechanics as this is quite a crucial part of the game and it	

would help visualise the success for the player. If I have time I would like to include the different defensive structures such as, adding people to the backfield, or narrow v wide defence. I feel this would add an extra layer of realism to the game for people who are more hardcore rugby fans, whilst keeping it simple for more casual gamers. For the kicking to goal, I will use a QTE where if they succeed, the ball transforms using the passing system to transform between the posts. If it fails it goes to one side.. I like the simple design of the lineout however as it isn't a mobile game, I would need to change it slightly so I can use the mouse to click on the assigned button for the specific throwing position. For the ruck mechanic, I want to remove the sliding scale as I feel it doesn't add to the realism of a ruck, as they aren't linear and you only know the outcome once a scrumhalf has got hold of the ball.

Retro Bowl

This image shows a control overlay in the bottom right corner of the screen during gameplay on PC. It provides a visual guide to the player's available actions. Whilst this design is effective for quickly reminding the player of control mappings, I feel it can be simplified to avoid visual clutter. In my game I plan to only display the button prompts and have the visual representation in the control tab of the settings.

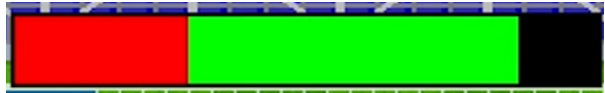
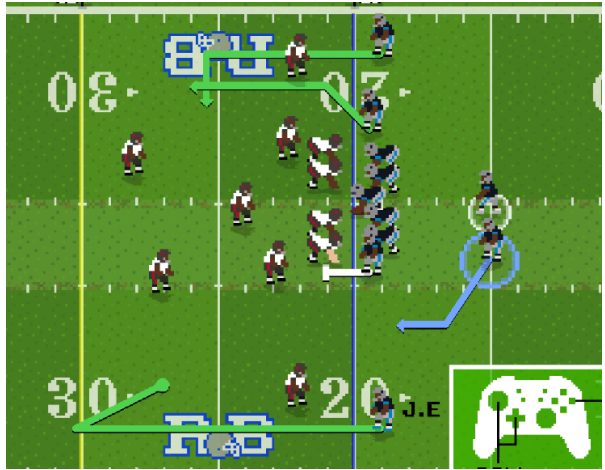


The design of the passing UI in Retro Bowl is very clean and simplistic. I plan to use this design for the design of the parabolic curve of the kicks if I add it.



Retro Bowl has Quick Time Event minigames for certain events such as kicks. I plan to use variations of these throughout my game such as for lineouts and kicks to goal.



	
Retro Bowl has specified paths for players to follow for specific set pieces. I would like to implement something similar from lineouts and scrums. However, depending on the stakeholders thoughts, this might be an optional extra that I make after the game is complete or just in the future after the NEA is due.	
Conclusion	
From Retro Bowl, I would like to incorporate the idea of event minigames and in the future I would like to change the design for different situations such as having a vertical needle-stop minigame for the lineout instead of horizontal. Also, as I have decided that I will not include kicking, I am not adding the UI showing where the kick will go	

Rugby o8

This image shows a power dial used during the goal kicking sequence. The dial includes a moving marker to a randomised “sweet spot” location which players must align with to achieve maximum kick power and accuracy. I plan to replicate this for my game by creating a needle stop style power meter with this dynamic sweet spot.



This image shows the player v player lineout UI. It gives button prompts for each player to click to compete for the ball in a certain position. I want to implement something similar into my game, however I need the buttons to be different as the players will be playing on the same device.



This image shows the passing UI from Rugby o8. I want to make it slightly simpler for a more arcade styled control system such as there being only one button for passing.



Conclusion

I will incorporate the power dial minigame for the kicking to goal event into a needle stop QTE. However, I would need to make the UI slightly more user friendly as it is slightly confusing. To combat this, I will not incorporate the design of the passing UI from Rugby o8 as I believe it will not fit the arcade style I am going for as I want the UI to be simple. For the

lineouts I am planning to just add one set of buttons for the attacking team as well as a QTE event to win the ball.

What I will include from my research:

Overall, after analysing Rugby Nations 24, Rugby 08 and Retro Bowl, I have identified several features that I plan to incorporate into my game. From Rugby 08, I will add a QTE for the conversion using a Needle Stop minigame to determine if it is successful or not. I will also implement a simplified version of its lineout competition mechanic which will be adapted for local multiplayer on a single device. From Retro Bowl, I will take inspiration from the retro visual style and use the simple event based minigames which can be adapted for different rugby scenarios. From Rugby Nations 24, I will incorporate the colour coded passing indicators to make the gameplay more intuitive. I will also simplify the pause menu for accessibility.

What I won't include in my game:

Through my research, I found several features which would be expected in a more professional game, but which I will not be implementing in my project. They include a kicking system with box kicks, drop kicks and kicks in open play, more advanced defensive structures with alternative defensive line formations, set pieces from lineouts and kickoffs and penalties. I considered these features during my planning however I came to the conclusion that, although they would add more realism and depth to my game, they would also add significant complexity.

This would cause the project to not remain achievable within the project timeframe. My main aim was to produce a working, functional rugby game with the key components of rugby I deemed essential. If I had added these components, I would not have been able to finish the project and it would have ended up poorly made.

However, If I had more time, resources and people helping like a professional development studio, I would have been able to add all these components. I still plan to add some of these components but it would be in my free time after A levels.

How did the stakeholders react to my research?

Stakeholders responded positively to my research and the proposed game features and their feedback helped refine the direction of several mechanics.

- **What do you think about me adding a power dial minigame for the goal kicking?**

Thoughts:

The stakeholders thought this was a good idea but that it was not a fundamental part of my game, so I am planning on simplifying this to a horizontal needle stop QTE which I am also using for the lineout, and if I have time, I will change it to a circular one.

- **Would you like simple lineout controls or more complicated ones?**

Thoughts:

All the stakeholders said they preferred simple, user-friendly lineout controls to maintain accessibility and smooth gameplay. Some suggested that future updates could introduce a couple optional set plays to add more realism into the game as well as a small tactical element. However, due to time constraints and the prioritisation of core features, I will not include this advanced option to the initial version.

- **What are your thoughts on colour coded passing indicators**

Thoughts:

This idea received unanimous approval. Stakeholders felt it would give players clear and intuitive visual cues when deciding where to pass.

- **What are your thoughts on having pop up notifications for ref decisions?**

Thoughts:

Stakeholders didn't like this as they said it would create too much clutter on screen, especially as I don't have penalties, so I will not be adding it.

- **What are your opinions on having a margin of error for the kicks, increasing the further the kick goes?**

Thoughts:

As I am not including kicking anymore, this question doesn't really apply, but I went back and asked them if they would like this sort of concept for passing and they unanimously agreed so I will create a passing algorithm that simulates this.

Conclusion:

Overall, the stakeholders feedback was extremely helpful and reinforced many of my design choices. Their input highlighted the importance of user friendly, intuitive controls while also suggesting potential areas for future development. I will prioritise implementing the features that enhance core gameplay and are achievable during the project timeframe.

Program Objectives

1. Gameplay
 - 1.1. Movement
 - 1.1.1. Player can move with WASD and set keybinds
 - 1.1.2. Sprinting consumes stamina which regenerates while resting
 - 1.1.3. Player switching selects the nearest player to the ball
 - 1.2. Passing
 - 1.2.1. When in passing range, icon appears above teammate
 - 1.2.2. Pass passing outcomes vary realistically with distance and stats
 - 1.2.2.1. Success curve (90% < 6m, 65% 6-12m, 35% 12-18m)
 - 1.2.2.2. Differing icon colours depending on success chance (R,Y,G)
 - 1.3. Kicking
 - 1.3.1. Player will perform a kick to goal (conversion) after scoring a try
 - 1.3.1.1. Player will use a Quick Time Event to see if it was a successful conversion, QTE will be a needle-stop-minigame
 - 1.4. Tackling and Defence
 - 1.4.1. Tackle initiated when defender in range and player presses tackle button
 - 1.4.2. Tackle initiated when the AI defender is in range
 - 1.4.3. Successful tackle causes a ruck
 - 1.5. Match Flow
 - 1.5.1. Full 10-a-side match flow implemented with all core mechanics
 - 1.5.2. Ruck
 - 1.5.2.1. When player is tackled, a ruck will form with a set number of players from each side
 - 1.5.2.1.1. A stack will be used to get the players involved in ruck
 - 1.5.2.1.2. During ruck sequence, player cannot move until ruck is won or lost
 - 1.5.2.1.2.1. 80-90% success rate of keeping ruck with equal players in ruck

- 1.5.2.2. Winning side recycles ball to scrumhalf who can be controlled by player
 - 1.5.2.3. Scrumhalf can only pass the ball away
 - 1.5.3. Lineouts
 - 1.5.3.1. Triggered when ball goes into touch
 - 1.5.3.2. Player can choose where they want to throw the ball
 - 1.5.3.2.1. QTE needle-stop game
 - 1.5.3.3. Player can choose where they want to throw
 - 1.5.3.4. If they mess up the needle stop minigame the defending team win the ball
 - 1.5.3.5. Play resumes with pass from scrumhalf
 - 1.5.4. Kick-offs
 - 1.5.4.1. Restarts with a tap and go from the team that didn't score
 - 1.5.5. Scoring and match flow
 - 1.5.5.1. Try = 5 points, Conversion = 2 points
 - 1.5.5.2. Match half lengths configured on main menu before the start of the game
 - 1.5.5.3. Match clock counts up with a break at half time
 - 1.5.5.4. Match clock resets at halftime
 - 1.5.5.5. Match clock speed changes with preset half lengths
 - 1.5.5.6. Match clock is paused when players go into settings/options screen
 - 1.5.5.7. Scoreboard updates automatically after each scoring event
- 2. Input and Controls
 - 2.1. Input Mapping
 - 2.1.1. Move, sprint, pass, tackle, switch, rebind and pause
 - 2.2. Rebinding
 - 2.2.1. All controls can be rebound in game
 - 2.2.1.1. Uses JSON save/load to save bindings
- 3. Finite State Machine
 - 3.1. Attacking FSM
 - 3.1.1. Non-ball carriers hold diagonal line behind ball carrier
 - 3.1.1.1. Players use stable assignments (stay in position if pass is made, only switch if someone becomes unavailable)
 - 3.1.1.2. Players can become unavailable from certain criteria such as:
 - 3.1.1.2.1. Get tackled
 - 3.1.1.2.2. In ruck
 - 3.1.1.2.3. Is scrumhalf
 - 3.1.1.2.4. In lineout
 - 3.1.1.3. Attacking Formation System

- 3.1.1.3.1. Arrow head created with the ball carrier at tip
 - 3.1.1.3.2. Only re-shuffles if players become unavailable
 - 3.2. Defensive FSM
 - 3.2.1. Opponents use FSM with states
 - 3.2.1.1. Defend Line → Chase → Tackle
 - 3.2.1.2. Aggression stat changes the range of tackle and chase radius
 - 3.2.1.3. Line integrity maintained (defenders stay in a straight line unless in Chase)
 - 3.2.2. Tackling
 - 3.2.2.1. AI tackle lunge triggered in range
 - 3.2.2.2. If a tackle is successful a ruck is triggered
 - 3.2.3. Rucks
 - 3.2.3.1. Nearest 2 players join ruck after tackle
 - 3.2.3.2. Nearest player comes to pass the ball on the attacking team once the ruck is resolved
- 4. Rules
 - 4.1. Offside
 - 4.1.1. Offside line enforced at last ruck/tackle
 - 4.1.2. No passes to offside players (position checks)
- 5. Physics and Ball
 - 5.1. Ball follows projectile motion curve for pass
 - 5.2. Kick follows projectile motion from pass
- 6. Attributes and Difficulties
 - 6.1. Player Stats
 - 6.1.1. Speed
 - 6.1.2. Stamina
 - 6.1.3. Handling
 - 6.1.3.1. Affects passing success
 - 6.1.4. Aggression
 - 6.1.4.1. Affects Chase and Tackle radii
 - 6.2. Difficulty Levels
 - 6.2.1. Affects AI aggression, tackle %, pass error, stamina drain
 - 6.2.2. Easy
 - 6.2.3. Medium
 - 6.2.4. Hard
- 7. UI/UX
 - 7.1. HUD
 - 7.1.1. Shows score, clock, possession and pass icons
 - 7.1.2. Start menu
 - 7.1.2.1. Match Length

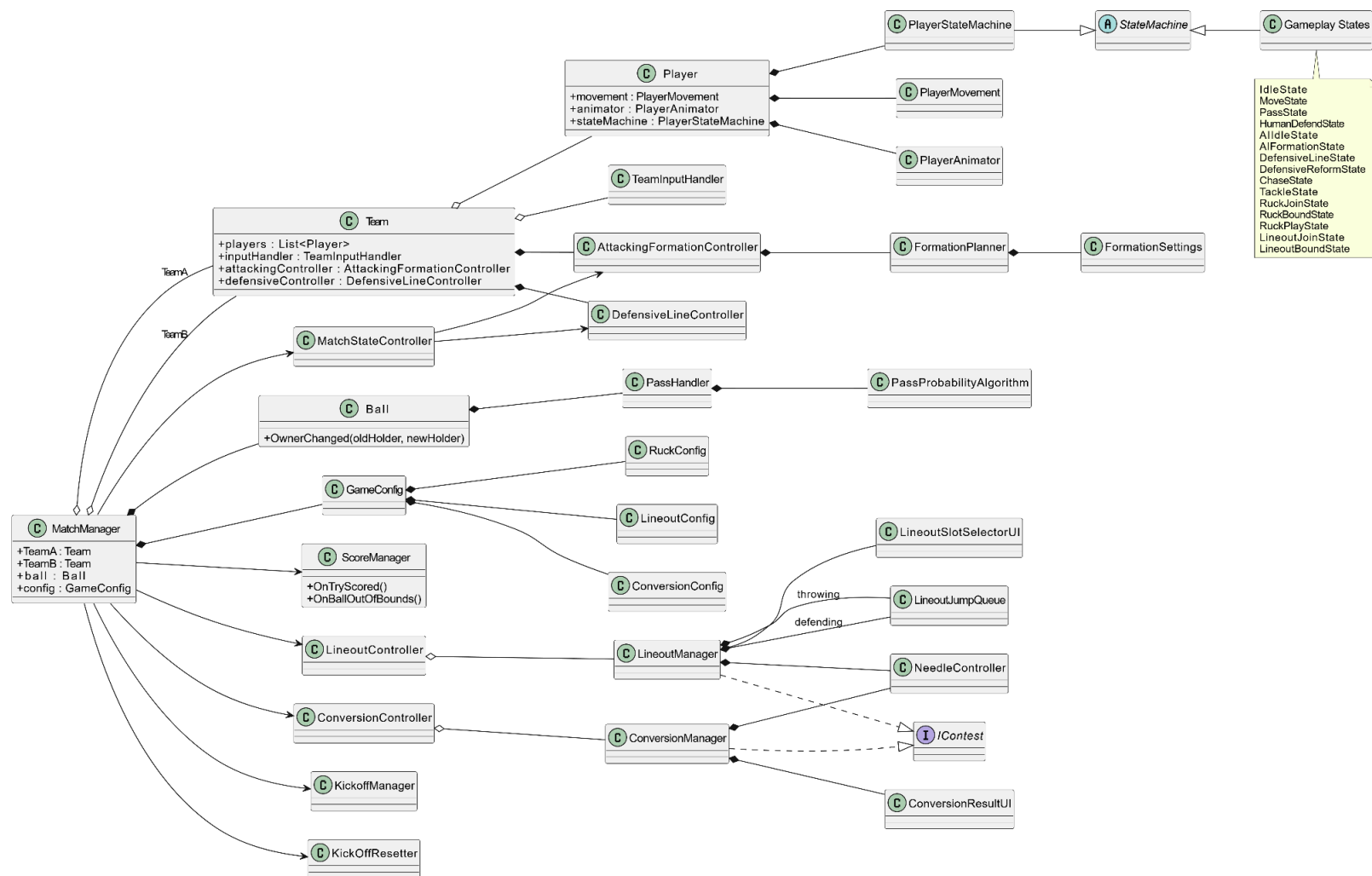
7.1.3. Pause menu

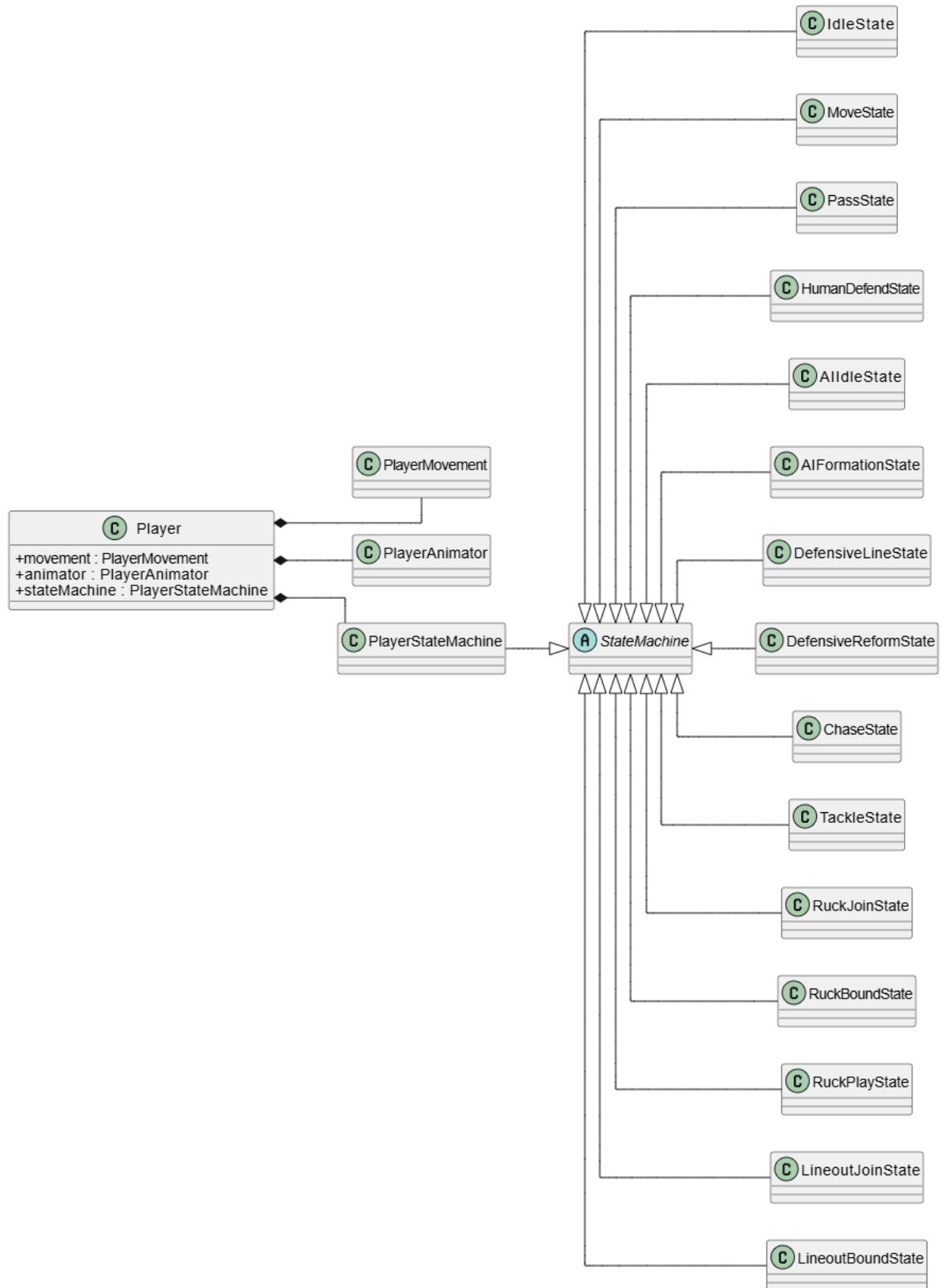
7.1.3.1. Allows rebinding

Modelling:

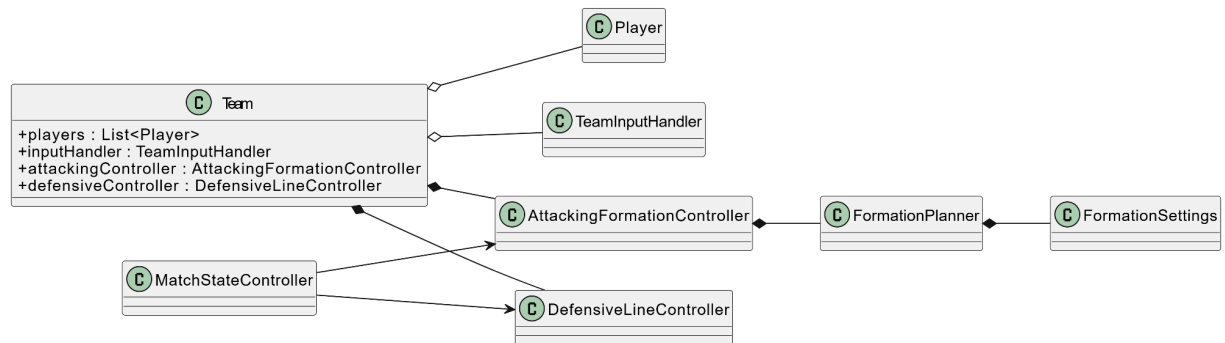
Key Rugby Mechanics:

What is it called?	What is it?	How will I make it?
Tackling	Ball carrier is brought to the ground and if the ball isn't offloaded quickly a ruck is formed	When an opposition player gets within range it enters the tackle stage and attempts to make a tackle.
Ruck	After a tackle, players from both teams bind over the ball and push against each other until the ball is played.	I will also use a stack to simulate the ruck as it is also a LIFO structure.
Lineout	Ball is thrown in from the touchline; jumpers are lifted to catch the ball and pass it to scrumhalf.	I will use a queue to add players to the lineout on both side, then I will use QTE events for the contest:
Conversions	After a try is scored, kicker takes a kick to goal in line with where the try is scored	I will use a QTE event to see if it is successful or not then I will use a pass to an object between the posts for a successful kick.
Scrum	N/A	Won't do (frustrating for user)
Knock On	N/A	Won't do (frustrating for user)
Interceptions	N/A	Won't do (frustrating for user)

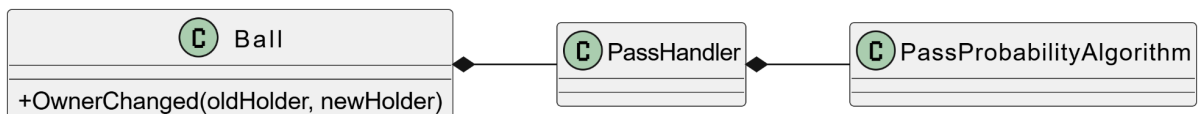




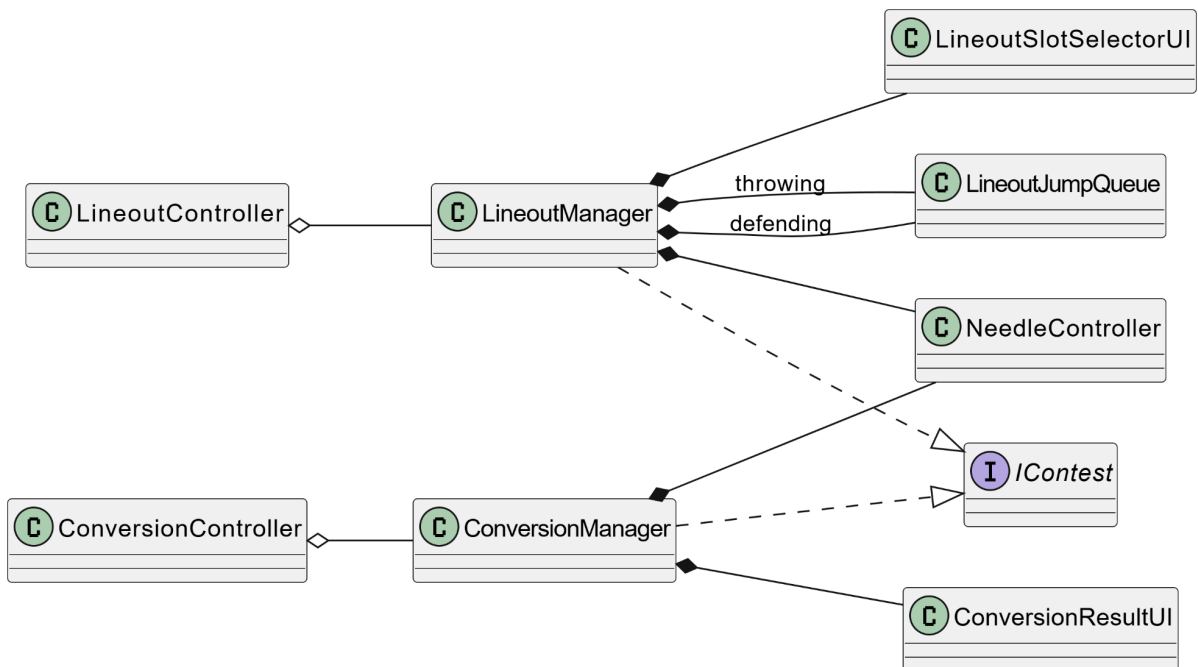
This diagram shows how each team groups players and how they use controllers for attacking and defensive formation and organisation:



This shows the passing system. The ball contains the pass handler which uses the probability algorithm to determine the outcome. The ball ownership is also changed using an event.



This diagram shows the lineout and conversion systems and how they use a shared interface



Made with (draw.io)

Test Plan:

Movement and Control Tests

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
To1	Player Movement	Check that the controlled player moves in the chosen direction	Press movement keys	Player moves correctly in the applied direction		
To2	Sprinting	Check that sprint increases movement speed	Hold sprint while moving	Player moves faster than normal movement speed		
To3	Stamina Drain	Check that sprinting reduces stamin	Sprint continuously for a couple seconds	Stamina value decreases while sprinting		
To4	Stamina Recovery	Checks that the stamina regenerates while not sprinting	Stop sprinting	Stop sprinting		
To5	Sprint Limit	Check if player cannot spring if stamina is drained	Hold sprint button when stamina is drained	Fully drain stamina and hold down sprint button		
To6	Player Switching	Check that the player switched to the closest defending player to ball carrier	Press switch player button while defending	Controlled player changes to nearest defender AI		
To7	Pause Input	Check pause menu opens	Press pause button during game	Game pauses and settings menu appears		
To8	Resume Input	Check match resumes from pause	Press resume button in pause menu	Match continues right from where it was paused		

Passing Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T09	Pass	Check a pass can be made and pass to the correct side	Press the pass button if a teammate is available on that side	Pass goes to the player on that side following the parabolic trajectory		
T10	Short Ranged Pass Accuracy	Check if short passes are successful most of the time	Repeatedly pass to nearby players	Most passes are successful and go to the player		
T11	Long Ranged Pass Accuracy	Check if long passes are less successful	Repeatedly pass to far away teammates	Less passes are successful and don't go to the player		
T12	No Teammate target	Check pass to ground works	Pass to a side where there is no teammate	The pass should go to the ground		
T13	Possession Update after a successful pass	Check new receiver becomes ball holder	Complete a successful pass	Receiver gains possession		
T14	Passing UI	Check pass indicators appear when relevant	Check if the pass indicators appear when the new holder gains possession	Pass indicator around the current player		
T15	Pass Colour		Compare the different passing options	Colour changed according to probability		

Defensive and Tackling Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T16	Defensive Line Shape	Check AI defenders hold a horizontal line	Start kickoff	Defenders stay in a line, and follow their opposite player, rather than just		

				bunching together		
T17	Chase Behaviour	Check defenders leave line when attacker comes within a certain distance	Run attacker towards defensive line	Nearest relevant defender moves into chase state		
T18	Tackle Trigger	Check tackle happens when defender is in range	Move defender into a valid tackle range and press tackle or allow the AI to tackle	Tackle triggers and ruck is formed		
T19	AI Aggression Effect	Check aggression affects chase range	Compare players with different aggression values	More aggressive defenders chase from further away		

Ruck Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T20	Ruck Creation	Check ruck starts after a successful tackle	Complete a successful tackle	Ruck State Begins		
T21	Ruck Joiners	Check nearest support players join the ruck	Trigger a ruck with support players nearby	Nearest eligible players move into ruck positions		
T22	Scrumhalf Assignment	Check a nearby attacking player becomes the scrumhalf when the ball is won	Allow ruck to form	Nearest eligible player moves into scrumhalf role (passing indicators show up and player is given ball)		
T23	Ruck Resolution	Check ruck resolves correctly	Pass the ball away from the ruck	Players leave on both sides using LIFO, one by one		
T24	Ruck Possession Update	Check possession changes correctly after ruck	Lose or win ruck	Possession updates to winning team		
T25	Invalid Ruck Participants	Check unavailable players do not join ruck	Trigger ruck when some players are unavailable	Only valid players join the ruck		

Lineout Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T26	Ball into Touch	Check lineout starts when ball goes into touch	Pass or run into touch	Lineout sequence begins		
T27	Lineout Setup	Check players move into their lineout positions	Start lineout	Eligible players move into their slots		
T28	Lineout Controls	Check the front middle and back buttons appear	Click one of the buttons	QTE pops up		
T29	Lineout QTE	Check the QTE event resolves the lineout correctly	Win the QTE	Attacking team win the ball		
T30	Return to Open Play	Check the game resumes after the lineout ends	Finish lineout	Open play resumes and possession is correct		

Scoring, Conversion and Kickoff Tests

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T31	Try Scoring	Check try adds 5 points	Run over tryline	Team score increases by 5		
T32	Scoreboard Updates	Check scoreboard updates after score	Score a try	Scoreboard reflects the new score		
T33	Conversion QTE starts	Check conversion minigame begins after scoring a try	Score a try	Conversion QTE begins		
T34	Successful Conversion	Check successful QTE adds 2 points	Complete a successful Conversion	Team score increases by 2		
T35	Missed Conversion	Check failed conversion doesn't add points	Complete a failed conversion	Team score stays same		
T36	Kickoff Restart	Check conceding team restarts after score sequence	Click kickoff button	Kickoff setup begins and conceding team gets the ball		

Match Flow and Timer Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T37	Match Timer	Check timer counts up to 40 during game	Start match and let game run	Match clock increases over time		
T38	Match half length	Check timer matches starting half length time use a stopwatch to measure time	Let game run	At 40 the time on stopwatch should match the minutes set at the start		
T39	Half Time Trigger	Check half time occurs at the correct time	Play until half time	Match enters half time state		
T40	Second Half Restart	Check play resumes after half time	Continue from half time	Second half begins correctly		
T41	Full Time Trigger	Check match ends at correct time	Play until timer hits 40 in second half	Match enters full time state		
T42	Clock Pauses in menu	Check timer stops in pause menus	Open setting mid match	Timer stops		

FSM and State Transition Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T43	Attack to Defence Transition	Check possession loss swaps systems	Lose the ball in open play	Team changes from attacking system to defensive system		
T44	Defence to Attack Transition	Check possession gainswaps systems	Gain the ball in open play	Team changes from defensive system to attacking system		
T45	Gameplay to Ruck FSM	Check successful tackler enters ruck state	Complete tackle	Gameplay FSM transitions to Ruck FSM		
T46	Ruck to Gameplay FSM	Check ruck completion returns to open play	Finish Ruck	Ruck FSM exits and gameplay resumes		

T47	Gameplay to Lineout FSM	Check ball in touch event triggers lineout state	Send ball into touch	Gameplay GSM transitions into Lineout FSM		
T48	Lineout to Gameplay FSM	Check lineout completion returns to open play	Complete lineout	Gameplay resumes		
T49	Try to Conversion FSM	Check scoring triggers conversion state	Score a try	Conversion FSM begins		
T50	T56	Conversion to Kickoff FSM	Check conversion result leads to kickoff	End Conversion		

Input Mapping and Rebinding Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/Fail	Evidence (Timestamp)
T51	Rebind Movement Key	Change movement binding and test	Rebind a move key then use it	New key performs movement correctly		
T52	Rebind Pass Key	Change pass binding and test	Rebind a pass key then use it	New key performs movement correctly		
T53	Rebind Tackle Key	Change tackle binding and test	Rebind a tackle key then use it	New key performs movement correctly		
T54	Rebind Switch Player Key	Change switch player binding and test	Rebind a switch player key then use it	New key performs movement correctly		
T55	Save Bindings	Check rebound controls last through a restart	Rebind controls, close and reopen game	New bindings remain saved		
T56	Invalid Double Use Handling	Check duplicate bindings behave sensibly	Try assigning same key values to multiple controls	Game prevents it or handles it consistently		

UI Tests:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T57	Scoreboard Visibility	Check scoreboard displays during match	Start Match	Scoreboard is visible and readable		
T58	Pause UI	Check settings menu opens and functions	Open settings during match	Settings menu appears correctly		
T59	Conversion UI	Check QTE UI appears for conversion	Score a try	Conversion UI appears		
T60	Lineout UI	Check lineout UI prompts appear	Trigger lineout	Correct lineout UI appear		
T61	Pass Indicator UI	Check pass prompts appear at appropriate times	Gain possession	Passing indicator appears clearly		

Robustness Test:

Test ID	Feature	Test Description	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T62	No Ball Holder	Check game remains stable if ball has no holder	Miss a pass	Game doesn't crash, player has to go and pick up the ball		
T63	No Valid Pass Target	Check pass still resolves correctly	Attempt pass with no teammate target	Pass goes to ground		
T64	Pause During an Event	Check pausing during a special event	Pause during a lineout or ruck or conversion	Game pauses and resumes correctly when unpaused		
T65	Rapid Input Spam	Check repeated button presses do not break stuff	Spam pass, switch and tackle buttons	Game ignores invalid repeats and remains stable		
T66	Duplicate Object Protection	Check singletons and managers do not duplicate incorrectly	Restart game	No duplicate managers remain		

Design:

System Overview:

This rugby game is built using Object Oriented design, combining several interacting subsystems to replicate the core mechanics of rugby. Key gameplay components such as the ball, player and passing system to name a few are represented through separate classes with clearly defined responsibilities.

Match flow is coordinated by manager classes such as MatchManager, while FSM's are used to manage player and AI behaviour. This allows for transitions between states to be handled in a modular and maintainable way.

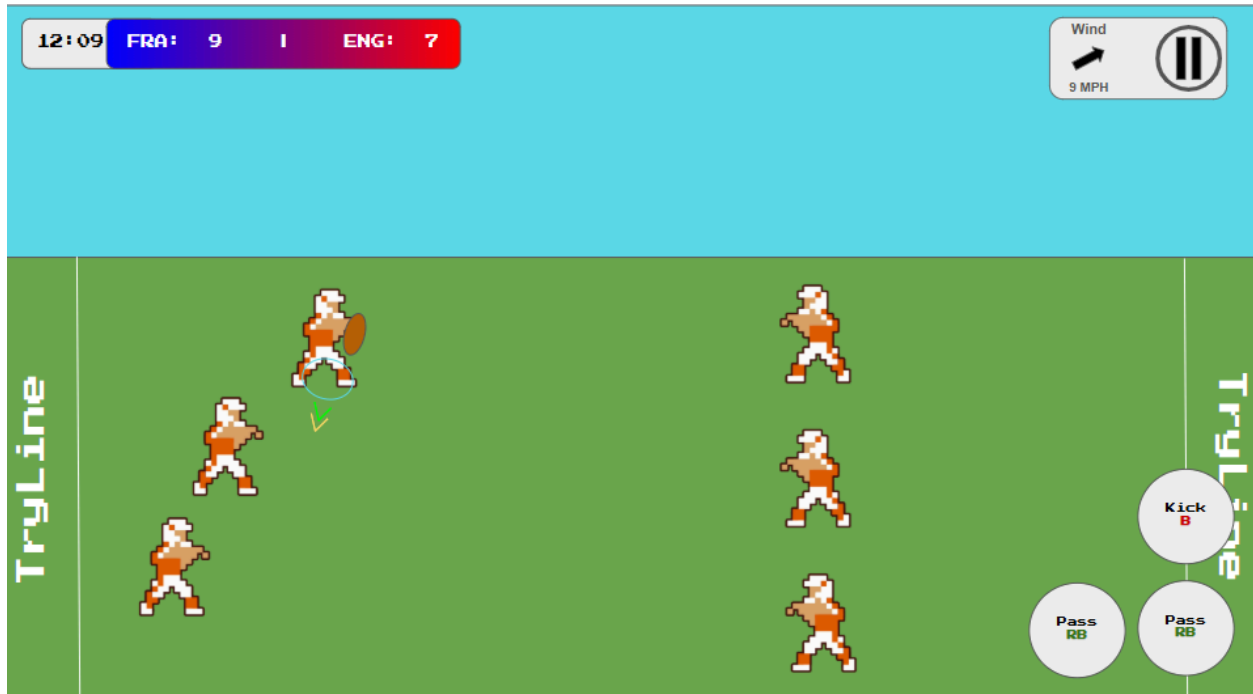
Individual mechanics like passing and ball handling are separated into dedicated handlers, controllers, or system classes.

Human-Computer Interactions (Wireframes):

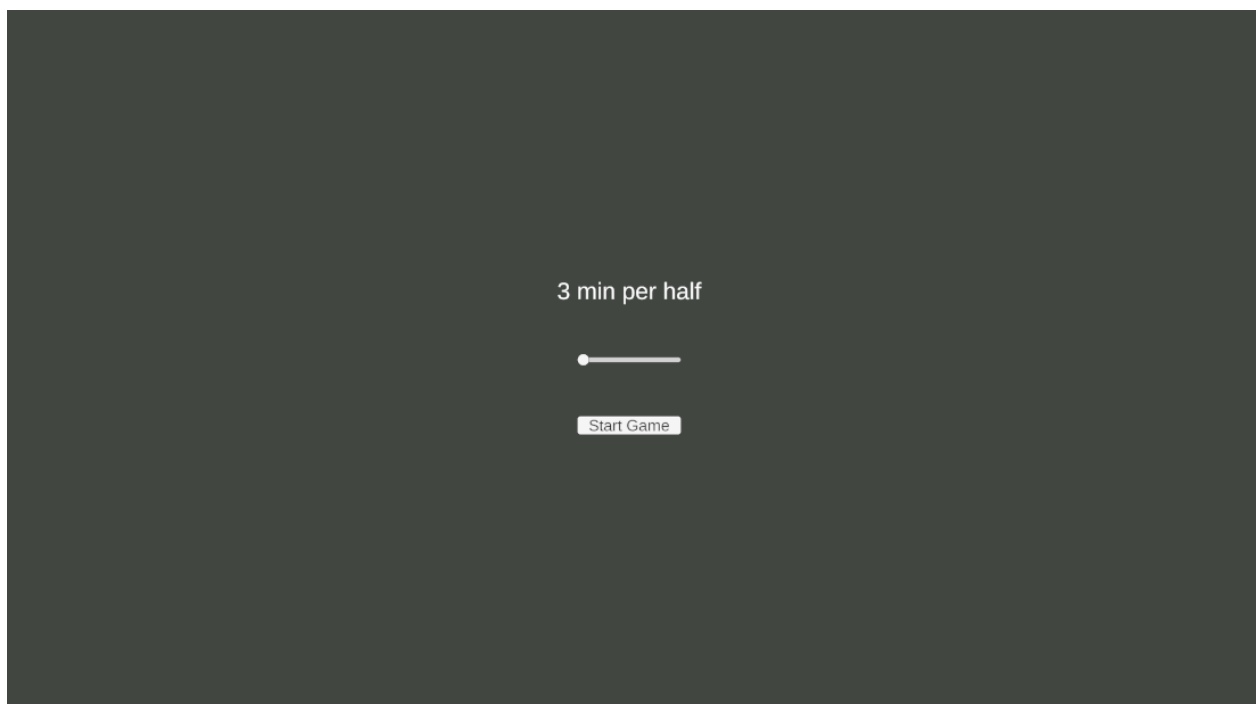
These diagrams outline the layout of menus, HUD and player input screens, showing how users interact with the game through simple and intuitive controls.

Gameplay:

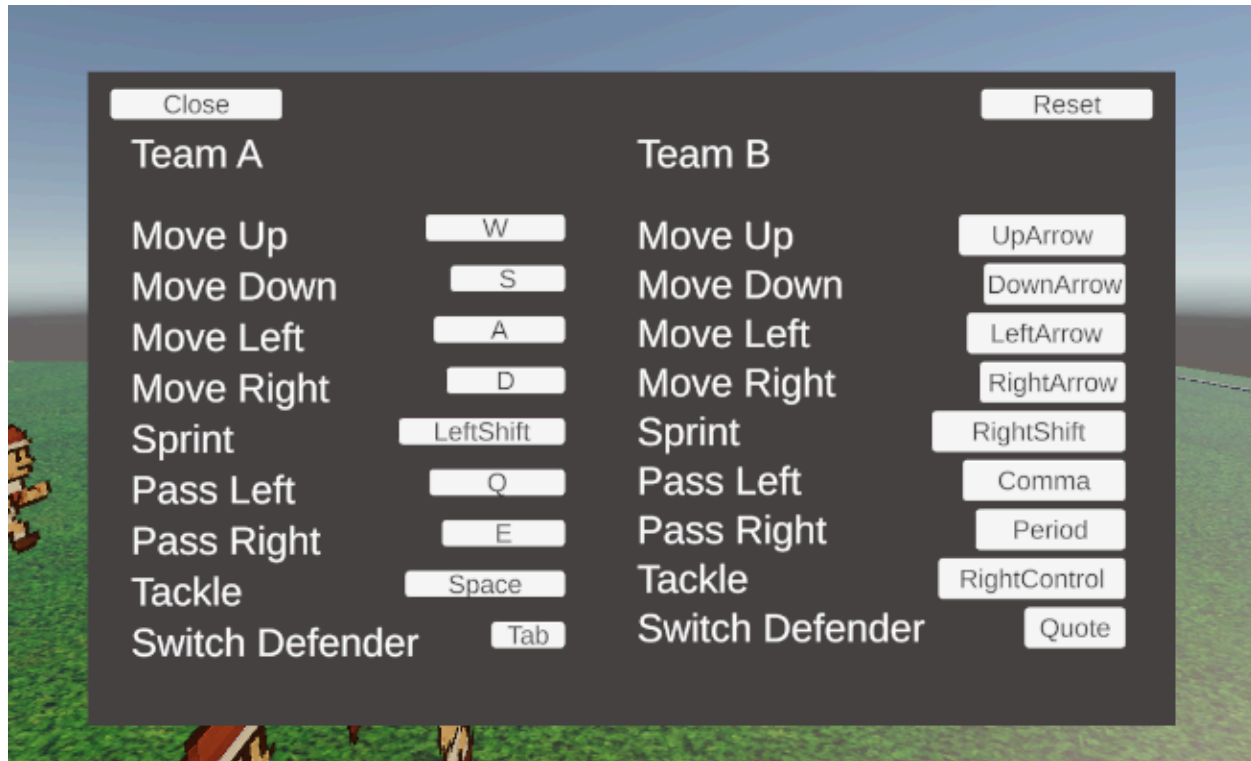
Initially I was planning on adding wind however, I came to the conclusion that I do not need it due to the fact I don't have any kicking.



HUD:



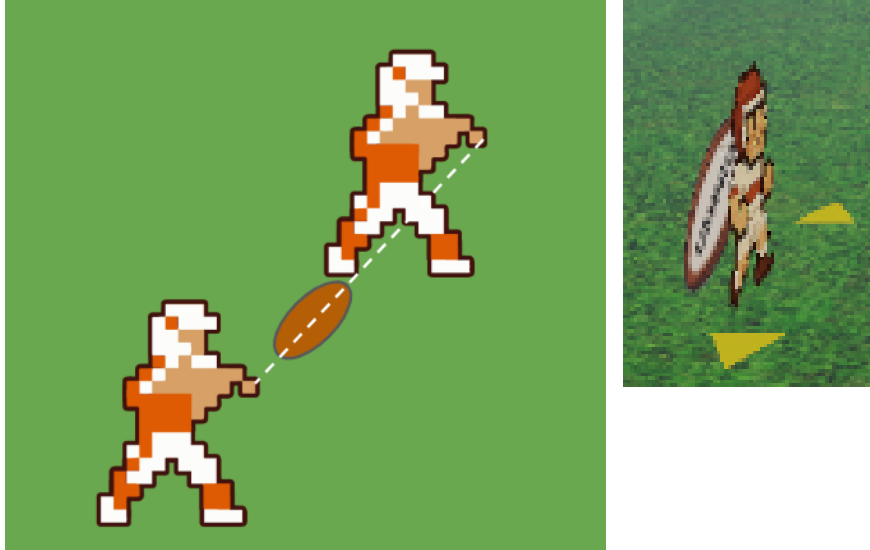
Rebind Menu:



The image below shows what happens when you click on a key to rebind it, the next button you press is the button that is added.



Passing:



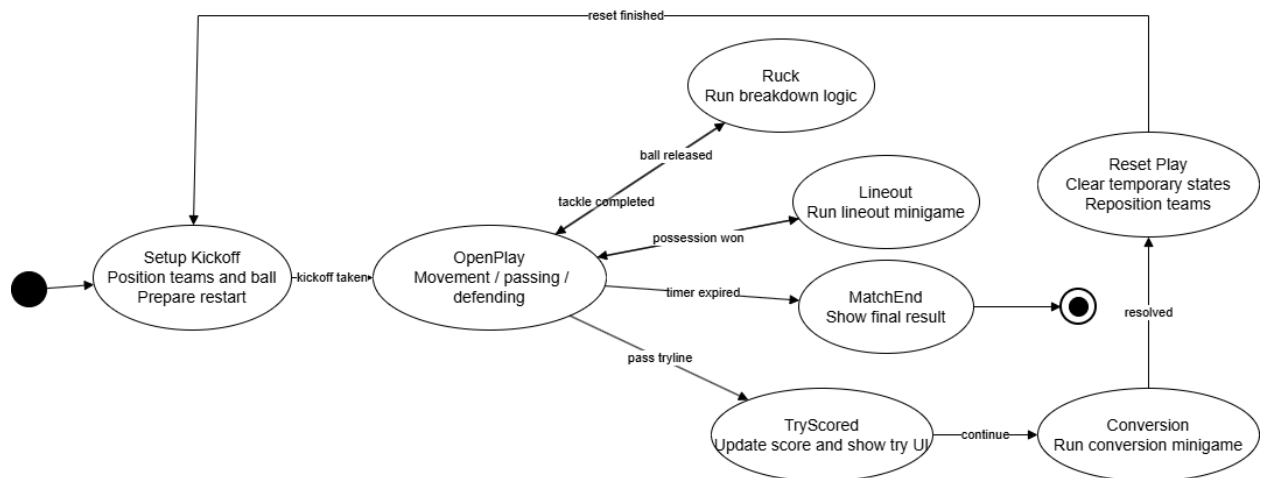
The right image shows the passing UI and the dynamically changing colour

FSM Modelling:

Each Finite State Machine diagram represents a major game phase. They show how the game transitions between states and how user inputs or in-game conditions trigger state changes.

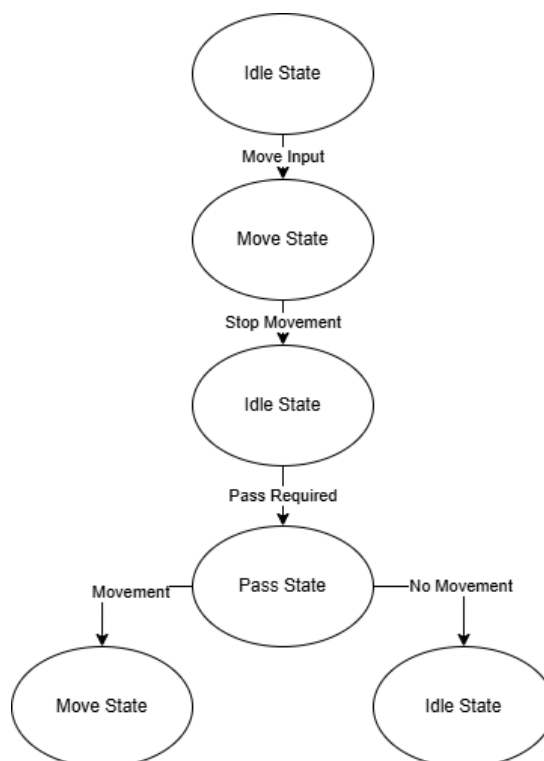
GameManagerFSM:

This FSM handles the overall gameplay of my game and outlines all the different parts and components of my game such as scrums, lineouts and tries.

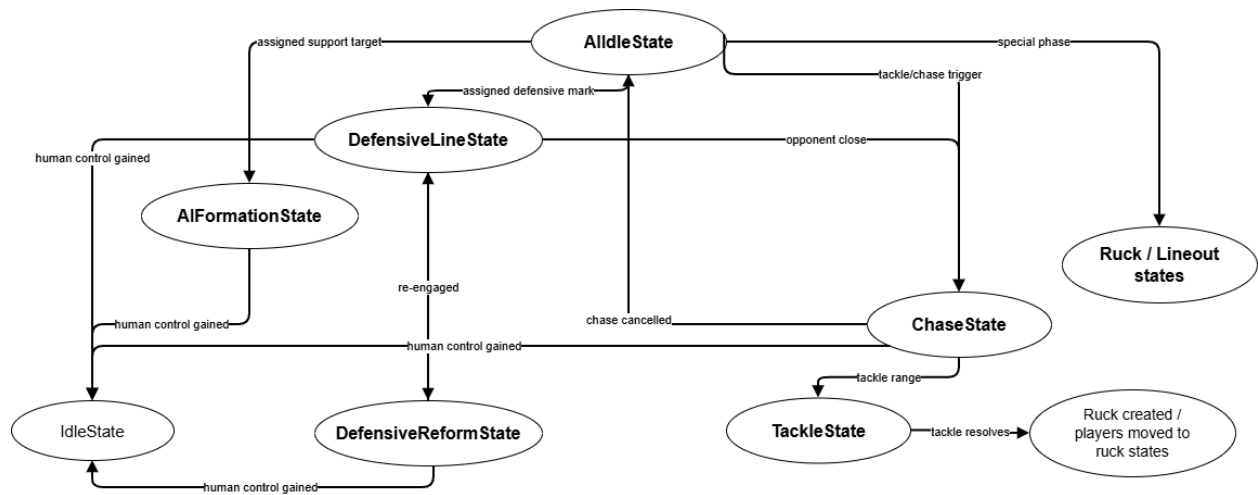


Player Controlled FSM:

This FSM is used when the player has the ball. It shows all the possible options the player can do

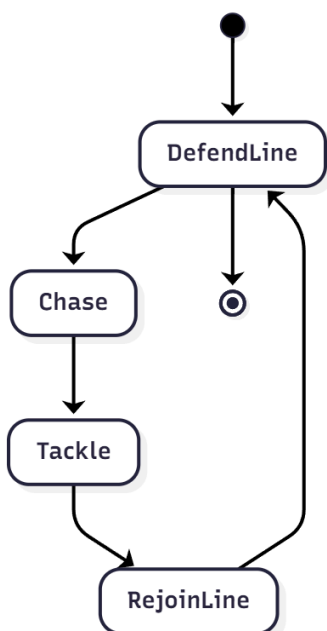


Defensive AI FSM:



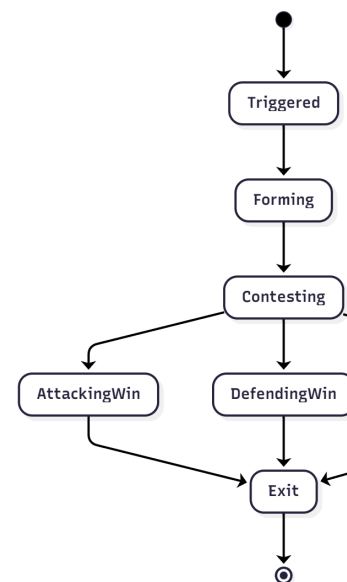
This FSM shows what all the possible options are for players on the attacking team with the ball or without the ball. I will use this for the non-controlled player team as well as using the without the ball parts for the supporting player on the users team.

DefensiveAIFSM(tackler):

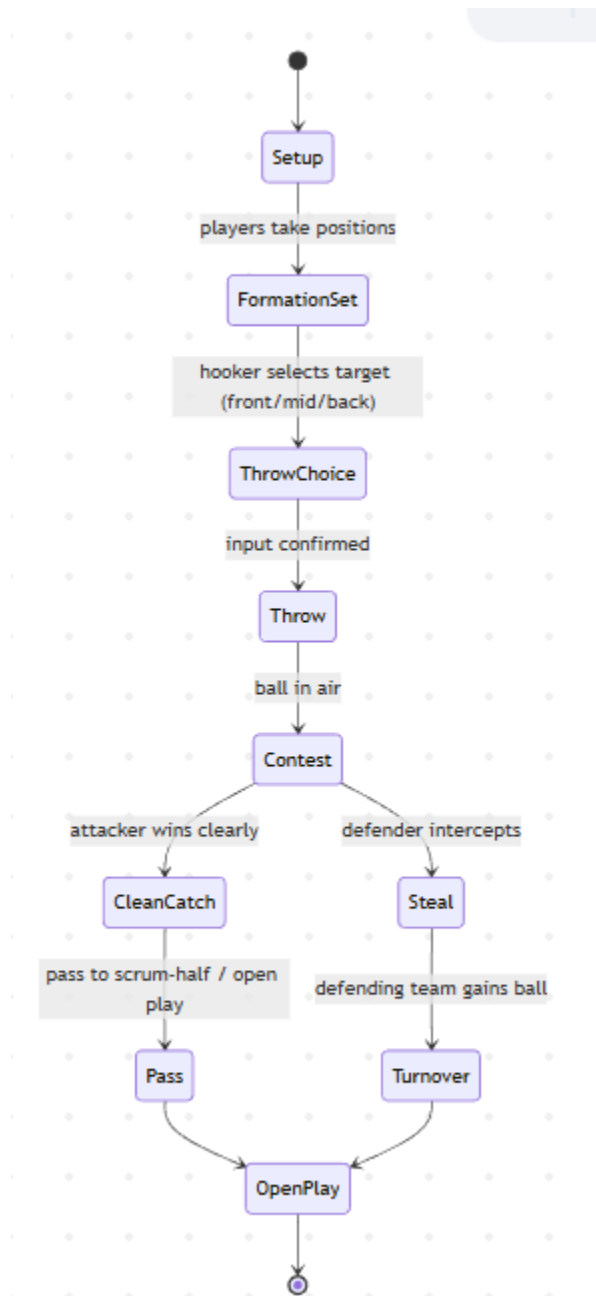


This FSM will be used for the defensive AI showing the possible everything the AI will be used for and will be able to do it.

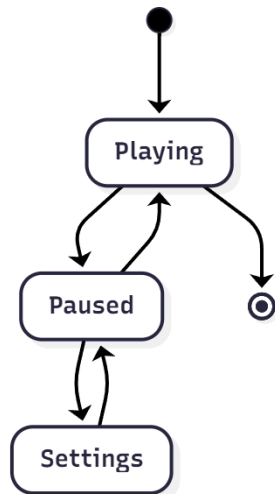
RuckFSM:



LineoutFSM:



Pause FSM:



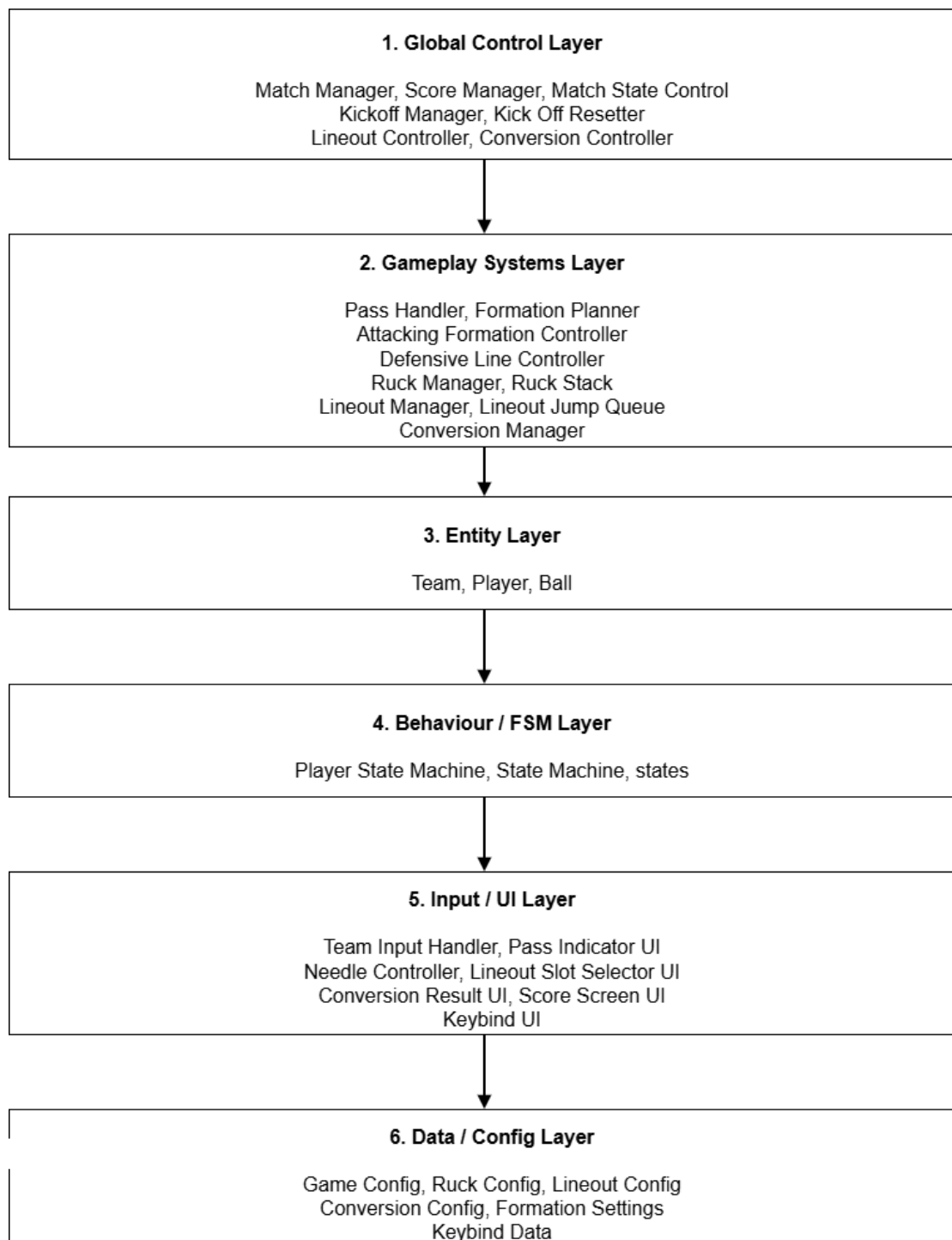
FSM Overview:

From FSM	Trigger / Event	To FSM	Description of Interaction
Gameplay Loop FSM	<i>Tackle Detected</i>	Ruck FSM	Starts a ruck sequence when a player is tackled.
Ruck FSM	<i>Ruck Finished</i>	Gameplay Loop FSM	Returns to open play when ball is passed from ruck
Gameplay Loop FSM	<i>Try Scored</i>	Conversion FSM	Triggers the goal-kicking mini-game after a try (QTE).

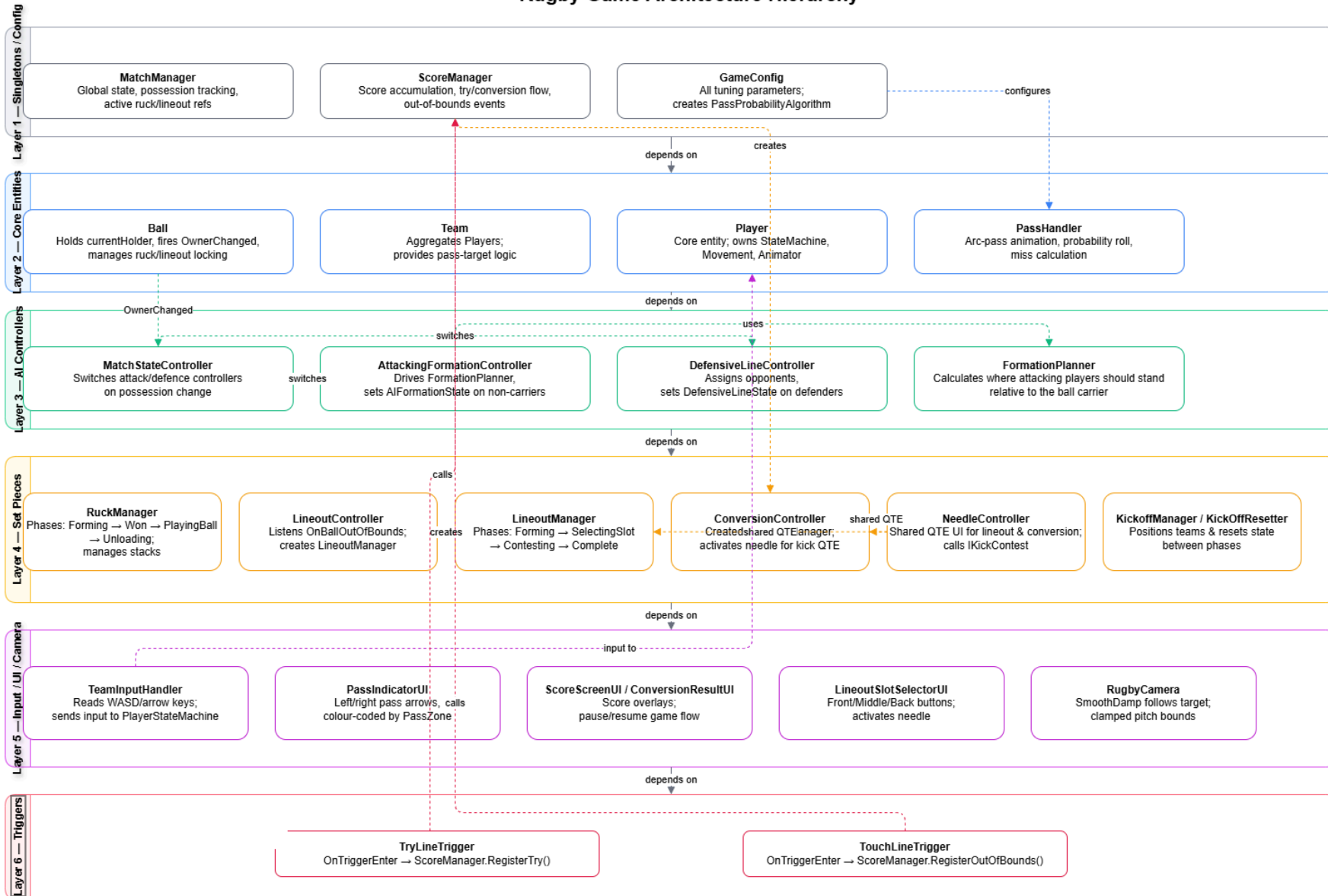
Conversion FSM	<i>KickResult(success/miss)</i>	Gameplay Loop FSM	Returns control to Kickoff state by the conceding team.
Gameplay Loop FSM	<i>Ball In Touch</i>	Lineout FSM	Activates lineout Quick Time Event (QTE).
Lineout FSM	<i>Lineout Completed</i>	Gameplay Loop FSM	Returns to open play after the ball is successfully thrown in and possession is established.
Gameplay Loop FSM	<i>Half Time Reached</i>	Half Time FSM	Pauses gameplay and transitions into the half time state
Half Time FSM	<i>Second Half Started</i>	Gameplay Loop FSM	Resumes gameplay with teams reset for the second half kickoff
Gameplay Loop FSM	<i>Full Time Reached</i>	Full Time FSM	Ends the match and transitions to the post match state
Any FSM	<i>Pause Input</i>	Pause FSM	Temporarily suspends all gameplay and freezes current state
Pause FSM	<i>Resume Input</i>	Previous FSM	Restores the exact FSM state that was active before pausing
Attacking FSM	<i>Possession Lost</i>	Defensive AI FSM	The player's team transitions from attack to defence.

System Hierarchy:

This rugby game is built using OOP design, it combines several subsystems to form the core components of rugby. The GameManager controls the match flow (kick off, open play, tackles, rucks, scrums lineouts, and scoring using a series of FSMs to manage transitions between phases (open play to ruck). Each mechanic such as tackling or kicking will be handled by its own service class.



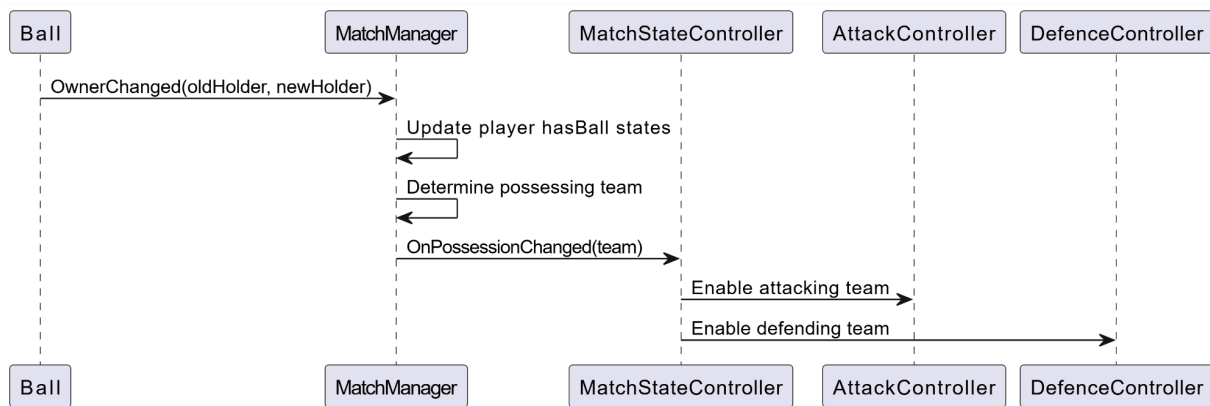
Rugby Game Architecture Hierarchy



Sequence Diagrams:

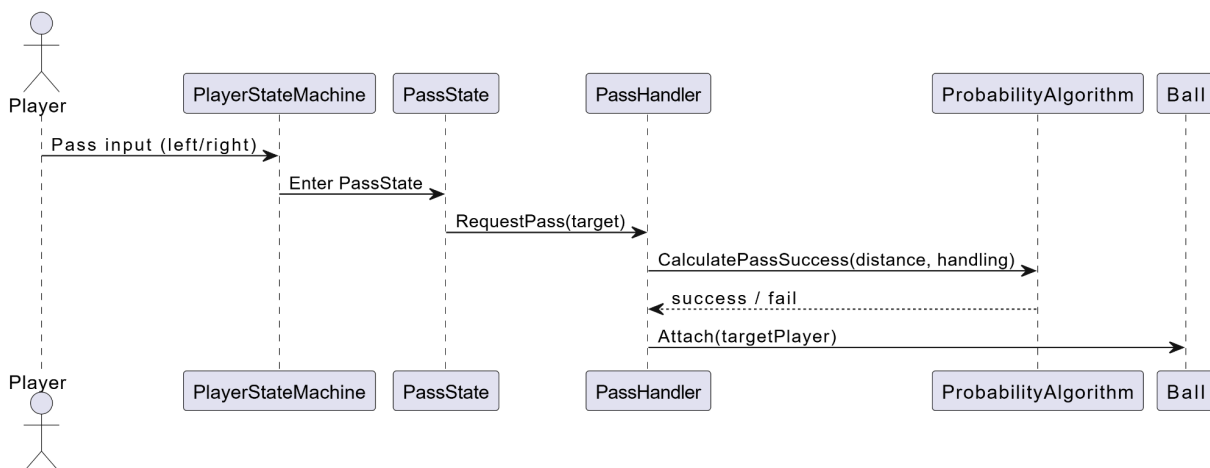
I used PlantUML to write the sequence diagrams into draw.io.

Possession Change Sequence:



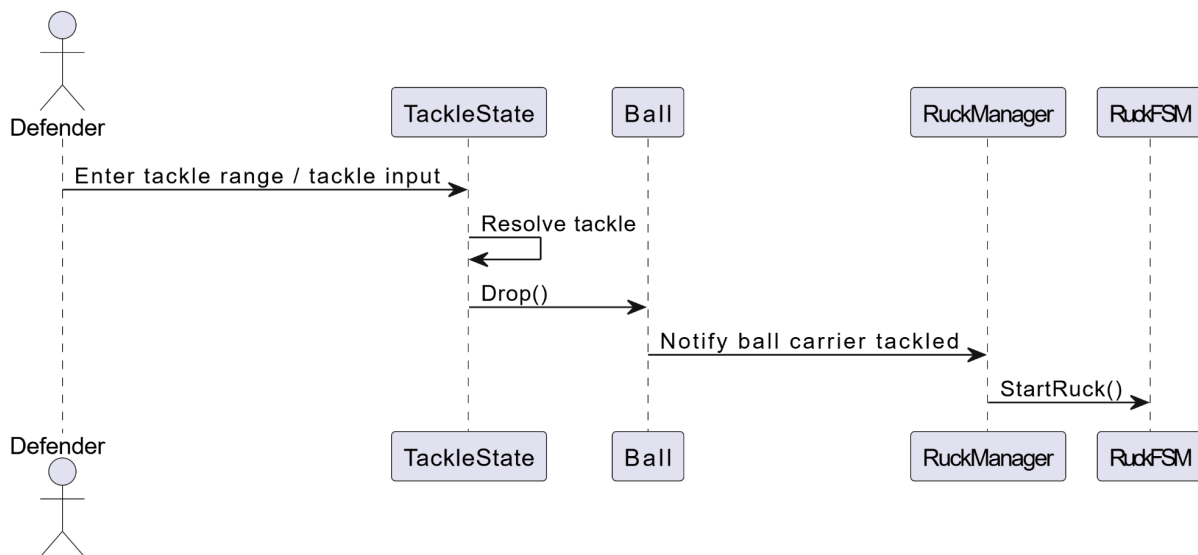
Pass System Sequence:

The sequence diagram shows how the passing works from a player input. The input is received by the player state machine and transitions to the pass state. A pass is then requested through the pass handler (which calculates the probability of a successful pass). The ball is then attached to the player if the pass is successful. If it fails it is passed to the ground.



Tackle to Ruck Sequence:

This sequence shows how the tackle event triggers a ruck. The defender attempts a tackle, the tackle state resolves it (success or fail). If the tackle fails, the defender returns to the line. If it is successful, the ball is dropped and the ruck manager is notified that the carrier has been tackled. The ruckmanager then activates the ruck FSM which transitions to the ruck contest and formation states.



Pseudocode for Core Systems and Algorithms:

The following pseudo code shows the most important logic used in the game. I have focused on the main gameplay flows, event driven transitions and the algorithms that control passing and team formations.

Core Match Flow:

These methods control the major transitions in the match/

Match Initialisation:

Match Manager Start:

```
None
METHOD Start
  IF ball is NULL OR TeamA is NULL OR TeamA.players is empty THEN
    RETURN
  ENDIF

  Attach ball to TeamA middle player
END METHOD
```

Match Manager Awake:

```
None
METHOD Awake
  IF Instance is NULL THEN
    Instance ← this
  ELSE
    destroy duplicate object
    RETURN
  ENDIF

  IF ball exists THEN
    subscribe to OnBallOwnerChanged to ball.OwnerChanged
  ENDIF
END METHOD
```

Possession Change Flow:

These methods handle the possession updates. When the ball changes owner, the previous holder is cleared and the new holder is assigned as well as the correct team controllers being activated.

Ball Ownership Change Event:

None

```
METHOD OnBallOwnerChanged(oldHolder, newHolder)
  IF oldHolder exists THEN
    oldPlayer ← Player on oldHolder
    IF oldPlayer exists THEN
      oldPlayer.SetHasBall(FALSE)
    ENDIF
  ENDIF

  IF newHolder exists THEN
    newPlayer ← Player on newHolder
    IF newPlayer exists THEN
      newPlayer.SetHasBall(TRUE)

      IF newPlayer.Team exists THEN
        SetPossession(newPlayer.Team)
      ELSE
        SetPossession(NULL)
      ENDIF
    ELSE
      SetPossession(NULL)
    ENDIF
  ELSE
    SetPossession(NULL)
  ENDIF
END METHOD
```

Set Possession:

None

```
METHOD SetPossession(newTeam)
  IF newTeam = PossessionTeam THEN
    RETURN
  ENDIF

  PossessionTeam ← newTeam

  fire OnPossessionChanged(PossessionTeam) event
END METHOD
```

Controller Update:

None

```
METHOD PossessionChanged(team)
  IF team = TeamA THEN
    enable TeamA attacking controller
    disable TeamA defending controller
    disable TeamB attacking controller
    enable TeamB defending controller
  ELSE IF team = TeamB THEN
    enable TeamB attacking controller
    disable TeamB defending controller
    disable TeamA attacking controller
    enable TeamA defending controller
  ELSE
    disable all controllers
  END IF
END METHOD
```

Tackle to Ruck Flow:

These two methods check whether the tackle was successful and creates and registers a new ruck if it was.

```
None
METHOD ResolveTackle(defender, carrier)
  IF defender is not in tackle range THEN
    RETURN fail
  ENDIF

  tackleSucceeded ← resolve tackle outcome

  IF tackleSucceeded = FALSE THEN
    reutrn defender to defensive line
    RETURN fail
  ENDIF

  Ball.Drop()

  create ruck
  RegisterRuck(ruck)

  start ruck FSM

  RETURN success
END METHOD
```

Register Active Ruck:

```
None
METHOD RegisterRuck(ruck)
  activeRuck ← ruck

  subscribe ClearActiveRuck to ruck.OnRuckComplete
  subscribe ReEvaluateControllers to ruck.OnRuckComplete
END METHOD
```

Ruck Lifecycle:

None

```
METHOD StartRuck(carrier, tackler, tacklePosition)
  ruckPlayers ← empty stack
  attackingSupport ← find nearest available attackers
  defendingSupport ← find nearest available defenders

  push tackler onto ruckPlayers
  push carrier onto ruckPlayers

  FOR EACH player IN attackingSupport
    push player onto ruckPlayers
  END FOR

  FOR EACH player IN defendingSupport
    push player onto ruckPlayers
  END FOR

  scrumHalf ← find nearest available attacking player not in ruck
  move all ruck players to ruck positions
  set ruck state to FORMING
END METHOD
```

None

```
METHOD UpdateRuck()
  IF not enough players have arrived THEN
    RETURN
  END IF

  set ruck state to CONTESTING
  winningTeam ← resolve ruck contest
  set ruck state to RESOLVED
  fire OnRuckComplete(winningTeam, scrumHalf)
END METHOD
```

Ruck Completion Flow:

This method resolves the ruck outcome. It attaches the ball to the scrumhalf it is there and goes to open play

```
None
METHOD OnRuckComplete(winningTeam, scrumHalf)
    ClearActiveRuck()

    IF scrumHalf exists THEN
        attach ball to scrumHalf
    ELSE
        attach ball to ground
    ENDIF

    SetPossession(winningTeam)

    ReEvaluateControllers()
END METHOD
```

Try to Conversion To Kickoff Flow:

This flow handles scoring, conversions and restarts.

```
None
METHOD OnTryScored(scoringTeam)
    add 5 points to scoringTeam
    update scoreboard UI

    start conversion FSM for scoringTeam
END METHOD
```

None

```
METHOD OnConversionComplete(scoringTeam, success)
    IF success = TRUE THEN
        add 2 points to scoringTeam
        update socreboard UI
    ENDIF

    start kickoff for Conceding team
END METHOD
```

None

```
METHOD StartKickoff(team)
    position players in kickoff formation
    give ball to restart player
    SetPossession(team)
    ReEvaluateControllers()
END METHOD
```

Ball In Touch to Lineout Flow:

This flow handles the transition from touch into a lineout then back into open play.

None

```
METHOD OnBallInTouch(lastTouchTeam, touchPosition)
    lineoutTeam ← opposite of lastTouchTeam

    position players for lineout at touchPosition
    start Lineout FSM with lineoutTeam
END METHOD
```

Lineout Lifecycle (2 methods):

This method shows how the players are selected and positioned for the lineout as well as how the possession is resolved and how the possession works afterwards.

None

```
METHOD StartLineout(touchPosition, throwingTeam)
```



```
    attackingQueue ← select lineout players for throwingTeam
    defendingQueue ← select lineout players for opposite team

    move players into lineout slots
    set lineout state to SETUP
END METHOD
```

None

```
METHOD ResolveLineout(throwChoice, contestResult)
    IF contestResult = clean catch THEN
        winningTeam ← throwingTeam
        receiver ← selected jumper
    ELSE
        winningTeam ← opposite team
        receiver ← defending catcher
    END IF

    scrumHalf ← find scrumHalf for winningTeam
    pass ball to scrumHalf or receiver
    fire OnLineoutComplete(winningTeam, scrumHalf)
END METHOD
```

None

```
METHOD OnLineoutComplete(winningTeam, scrumHalf)
    IF scrumHalf exists THEN
        attach ball to scrumHalf
    ELSE
        drop ball to ground
    ENDIF

    SetPossession(winningTeam)
    ReEvaluateControllers()

    resume open play
END METHOD
```


Algorithms:

(in Pseudocode)

Passing Algorithm:

Purpose:

This algorithm is used to make the passing more realistic in my game by ensuring that the passes are not perfectly accurate. Instead of every pass being 100% successful, the game calculates the chance of success based on certain factors like distance, player handling stat and game difficulty. This simulates real life more as every pass isn't accurate and sometimes they completely miss the person who is meant to receive it.

How it works:

When a pass is requested, the intended target is found, the target could be a teammate or the ground on the selected side (left or right) if there is no player to pass to. Once the target position is found, the algorithm calculates the probability that the pass will succeed using these inputs: distance, player handling stat, difficulty modifier.

For calculating the distance penalty, I will use:

$\text{distanceFactor} = 1 - ((\text{distance} / \text{maxPassDistance}) ^ \text{fallOffPower})$

This means for short distances $1 - (5/25)^{1.8} = 0.945$

But for longer distances $1 - (20/25)^{1.8} = 0.331$

In my code I currently have a fallOffPower of 1.8 which creates a curved falloff. (2 = quadratic, 1 = linear)

The handling stat is normalised between 1 and 0 using:

$\text{handlingFactor} = \text{handlingStat} / 100 * \text{handlingWeight}$

The difficulty penalty is calculated with:

$\text{difficultyFactor} = 1 / (0.5 + \text{difficultyModifier})$

0.5 difficulty means = 1

1 difficulty means = 0.667

2 difficulty means = 0.4

The final probability is calculated with:

$\text{rawProbability} = (\text{distanceFactor} * \text{distanceWeight}) + (\text{handlingFactor} * \text{handlingWeight}) + (\text{difficultyFactor} * \text{difficultyWeight})$

I then use clamp to make sure the pass cannot be impossible or guaranteed with:

$\text{minProbability} = 0.05$

$\text{maxProbability} = 0.98$

The clamp formula is:

Clamp (x, min, max)

So $\text{finalProbability} = \text{Clamp}(\text{rawProbability}, \text{minProbability}, \text{maxProbability})$

Then I generate a random number between 0 and 1 and if finalProbability is greater than this number the pass succeeds, if not it fails.

If the pass fails it then calculates a missRadius which is linked to the probability and distance,

$\text{baseRadius} = \text{distance} * 0.15$

$\text{inverseFactor} = \text{Lerp}(5, 1.2, \text{probability})$

Lerp does $5 + (1.2 - 5) * \text{probability}$

So for a low probability it will be a large miss, for a high probability it will be a small miss.

Then the final $\text{missRadius} = \text{baseRadius} * \text{inverseFactor}$

Pseudocode:

None

```
METHOD CalculatePassProbability(distance, handlingStat, difficultyModifier)

    normalisedDistance ← distance / maxPassDistance
    normalisedDistance ← CLAMP(normalisedDistance, 0, 1)

    distanceFactor ← 1 - (normalisedDistance ^ distanceFalloffPower)
    distanceFactor ← CLAMP(distanceFactor, 0, 1)

    normalisedHandling ← handlingStat / 100
    normalisedHandling ← CLAMP(normalisedHandling, 0, 1)

    handlingFactor ← normalisedHandling ^ 0.85

    difficultyFactor ← 1 / (0.5 + difficultyModifier)

    rawProbability ← (distanceFactor × distanceWeight) +
    (handlingFactor × handlingWeight) + (difficultyFactor × difficultyWeight)

    finalProbability ← CLAMP(rawProbability, minProbability, maxProbability)

    RETURN finalProbability

END METHOD
```

None

```
METHOD RollPassSuccess(probability)

    randomValue ← RANDOM(0, 1)

    IF randomValue < probability THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF

END METHOD
```

None

```
METHOD CalculateMissRadius(distance, probability)
```

```
    baseRadius ← distance × 0.15
```

```
    inverseFactor ← LERP(5, 1.2, probability)
```

```
    missRadius ← baseRadius × inverseFactor
```

```
    RETURN missRadius
```

```
END METHOD
```

None

```
METHOD CalculatePassOutcome(distance, handlingStat, difficultyModifier)
```

```
    probability ← CalculatePassProbability(distance, handlingStat,  
difficultyModifier)
```

```
    success ← RollPassSuccess(probability)
```

```
    IF success = TRUE THEN
```

```
        actualTarget ← intendedTarget
```

```
    ELSE
```

```
        missRadius ← CalculateMissRadius(distance, probability)
```

```
        actualTarget ← GenerateRandomOffsetTarget(intendedTarget, missRadius)
```

```
    END IF
```

```
    RETURN actualTarget, probability, success
```

```
END METHOD
```

Pass Flow:

None

```
METHOD RequestPass(direction, passer)
    intendedTarget ← find best teammate on chosen side

    IF intendedTarget does not exist THEN
        intendedTarget ← ground target on chosen side
    END IF

    distance ← measure distance from passer to intendedTarget
    probability ← CalculatePassProbability(distance, passer.handling,
difficulty)
    success ← RollPassSuccess(probability)

    IF success = TRUE THEN
        actualTarget ← intendedTarget
    ELSE
        missRadius ← CalculateMissRadius(distance, probability)
        actualTarget ← GenerateRandomOffsetTarget(intendedTarget, missRadius)
    END IF

    launch ball to actualTarget
END METHOD
```

Formation Planning Algorithm:

Purpose:

The purpose of the formation planning algorithm is to position the attacking support players around the ball carrier such that the team is able to maintain width, depth, and shape in open play. Without the algorithm, the attacking support players would just swarm around the ball carrier, making the attacking shape unrealistic. It would also make the game less enjoyable. This is achieved by the formation planning algorithm, which determines the position of the attacking support player relative to the ball carrier and the attacking shape.

How it works:

The algorithm starts by locating the current ball carrier and the available players. A player is available if they are not the carrier and not engaged in any other event, e.g., a ruck or lineout.

After locating all the players, the algorithm uses the forward reference vector to determine the attacking direction for the team. It then determines the lateral direction (90 degrees to the attacking direction). This is used for placing the players at the left and right sides of the carrier.

The algorithm then places each support player in a stable lane. These lanes have 2 main properties: side (left or right of carrier) and the lane index (how far they are from the centre of formation).

The stable definition I am using means the algorithm tries and keeps the player on the correct side as much as possible. This prevents the player constantly moving from left to right and vice versa, which would look chaotic and unrealistic.

After assigning players to lanes, the algorithm calculates the target world position for each player. This is done by starting from the carrier's position then moving them backwards and sideways to form an arrow shape with the carrier at the head of the arrow.

$$\text{targetPosition} = \text{carrierPosition} - \text{attackDirection} * \text{depthOffset} + \text{lateralDirection} * \text{widthOffset}$$

I have added a **recursive** part to this algorithm where the target positions are made one at a time. The base case is when the index reaches the end of the available player list. Otherwise, the algorithm calculates the target position for the current player, stores it, and then calls itself again for the next player.

The results of these calculations are then stored in a dictionary that maps each player to their assigned position. The attacking controller can then use these assignments to place each player in their formation following state and move them to the correct position.

The dictionary works as a hash table, storing the player and the assigned position as a key-value pair. This makes the program efficient as hash tables have an $O(1)$ time complexity.

Pseudocode:

None

```
METHOD CalculateFormation(carrier, availablePlayers, attackDirection)

    assignments ← empty dictionary

    IF carrier is NULL OR availablePlayers is empty THEN
        RETURN assignments
    END IF

    normalisedAttack ← NORMALISE(attackDirection)
    lateralDirection ← NORMALISE(CROSS(UP, normalisedAttack))

    IF available player set has changed THEN
        RebuildStableAssignments(availablePlayers)
    END IF

    AssignFormationRecursive(availablePlayers, 0, carrier, normalisedAttack,
    lateralDirection, assignments)

    RETURN assignments

END METHOD
```

None

```
METHOD AssignFormationRecursive(players, index, carrier, normalisedAttack,
lateralDirection, assignments)

    IF index ≥ players.count THEN
        RETURN
    END IF

    player ← players[index]

    side ← side assignment for player
    laneIndex ← lane index for player
    depthIndex ← depth row for player

    IF side = LEFT THEN
        sideSign ← -1
    ELSE
        sideSign ← 1
    END IF

    depthOffset ← depthSpacing × depthIndex
    widthOffset ← widthSpacing × sideSign × laneIndex

    targetPosition ← carrier.position - normalisedAttack × depthOffset
        + lateralDirection × widthOffset

    assignments[player] ← targetPosition

    AssignFormationRecursive(players, index + 1, carrier, normalisedAttack,
lateralDirection, assignments)

END METHOD
```

This method makes sure that there is an equal number of players on each side of the ball.

None

```
METHOD RebuildStableAssignments(players)
```

```
    clear stableAssignments
```

```
    leftLane ← 1
```

```
    rightLane ← 1
```

```
    FOR EACH player IN players
```

```
        IF player had previous side assignment THEN
```

```
            IF previous side = LEFT THEN
```

```
                assign player to LEFT with lane leftLane
```

```
                leftLane ← leftLane + 1
```

```
            ELSE
```

```
                assign player to RIGHT with lane rightLane
```

```
                rightLane ← rightLane + 1
```

```
            END IF
```

```
        ELSE
```

```
            IF leftLane <= rightLane THEN
```

```
                assign player to LEFT with lane leftLane
```

```
                leftLane ← leftLane + 1
```

```
            ELSE
```

```
                assign player to RIGHT with lane rightLane
```

```
                rightLane ← rightLane + 1
```

```
            END IF
```

```
        END IF
```

```
    END FOR
```

```
END METHOD
```

Defensive Pairing Algorithm:

Purpose:

The purpose of this algorithm is to organise defenders so that each one is assigned to an attacker, allowing the defence to maintain a structured defensive line across the pitch. Without this algorithm, the players would all honeypot towards the attacker making it very unrealistic and easy to score.

How it works:

First, the algorithm builds two lists, one for the available attackers and the other for available defenders. The players are then sorted from left to right across the field. The algorithm then pairs the players by their index (leftmost defender assigned to leftmost attacker). The results are stored in a dictionary where each defender maps to a specific attacker.

Once assignments are made, each defender uses the position of their assigned attacker to calculate where they should stand in the line. There is an offset so the defenders are a certain distance away from the attackers at the start so they are all onside.

$$\text{targetPosition} = \text{attackerPosition} - \text{attackDirection} * \text{defensiveDepth}$$

The algorithm provides the foundation for the defensive system as if the attacker gets within a certain distance, the defender moves into chase state then tackle state. This is done by the state machine which handles the higher priority actions.

Pseudocode:

None

METHOD RebuildAssignments(defenders, attackers)

 assignments $\leftarrow$ empty dictionary

 SORT defenders by x position

 SORT attackers by x position

 pairCount $\leftarrow$ MINIMUM(defenders.count, attackers.count)

 FOR i $\leftarrow$ 0 TO pairCount - 1

 assignments[defenders[i]] $\leftarrow$ attackers[i]

 END FOR

 RETURN assignments

END METHOD

None

METHOD CalculateDefensiveTarget(defender, assignedAttacker, attackDirection)

 IF assignedAttacker is NULL THEN

 RETURN defender.currentPosition

 END IF

 normalisedAttack $\leftarrow$ NORMALISE(attackDirection)

 targetPosition $\leftarrow$ assignedAttacker.position

 - normalisedAttack $\times$ defensiveDepth

 RETURN targetPosition

END METHOD

Defensive AI State Update:

None

```
METHOD UpdateDefender(defender, assignedAttacker, carrier)

    IF defender is in ruck or lineout THEN
        RETURN
    END IF

    IF assignedAttacker is NULL THEN
        set state to AIIdleState
        RETURN
    END IF

    distanceToCarrier ← calculate distance(defender, carrier)
    distanceToAssigned ← calculate distance(defender, assignedAttacker)

    IF distanceToCarrier < tackleRange THEN
        set state to TackleState

    ELSE IF distanceToAssigned < chaseRange THEN
        set state to ChaseState

    ELSE
        set state to DefensiveLineState
        targetPosition ← CalculateDefensiveTarget(defender, assignedAttacker,
        attackDirection)
        update defender target to targetPosition
    END IF
END METHOD
```

None

```
METHOD UpdateDefensiveLine(defenders, attackers, attackDirection)

    IF defenders is empty OR attackers is empty THEN
        RETURN
    END IF

    assignments ← RebuildAssignments(defenders, attackers)

    FOR EACH defender IN defenders

        IF defender has no assignment THEN
            CONTINUE
        END IF

        assignedAttacker ← assignments[defender]

        IF defender is human controlled THEN
            CONTINUE
        END IF

        IF defender is already in chase, tackle, ruck, or lineout state THEN
            CONTINUE
        END IF

        targetPosition ← CalculateDefensiveTarget(defender, assignedAttacker,
attackDirection)

        IF defender is not already in DefensiveLineState THEN
            set defender state to DefensiveLineState
        END IF

        update defender target to targetPosition

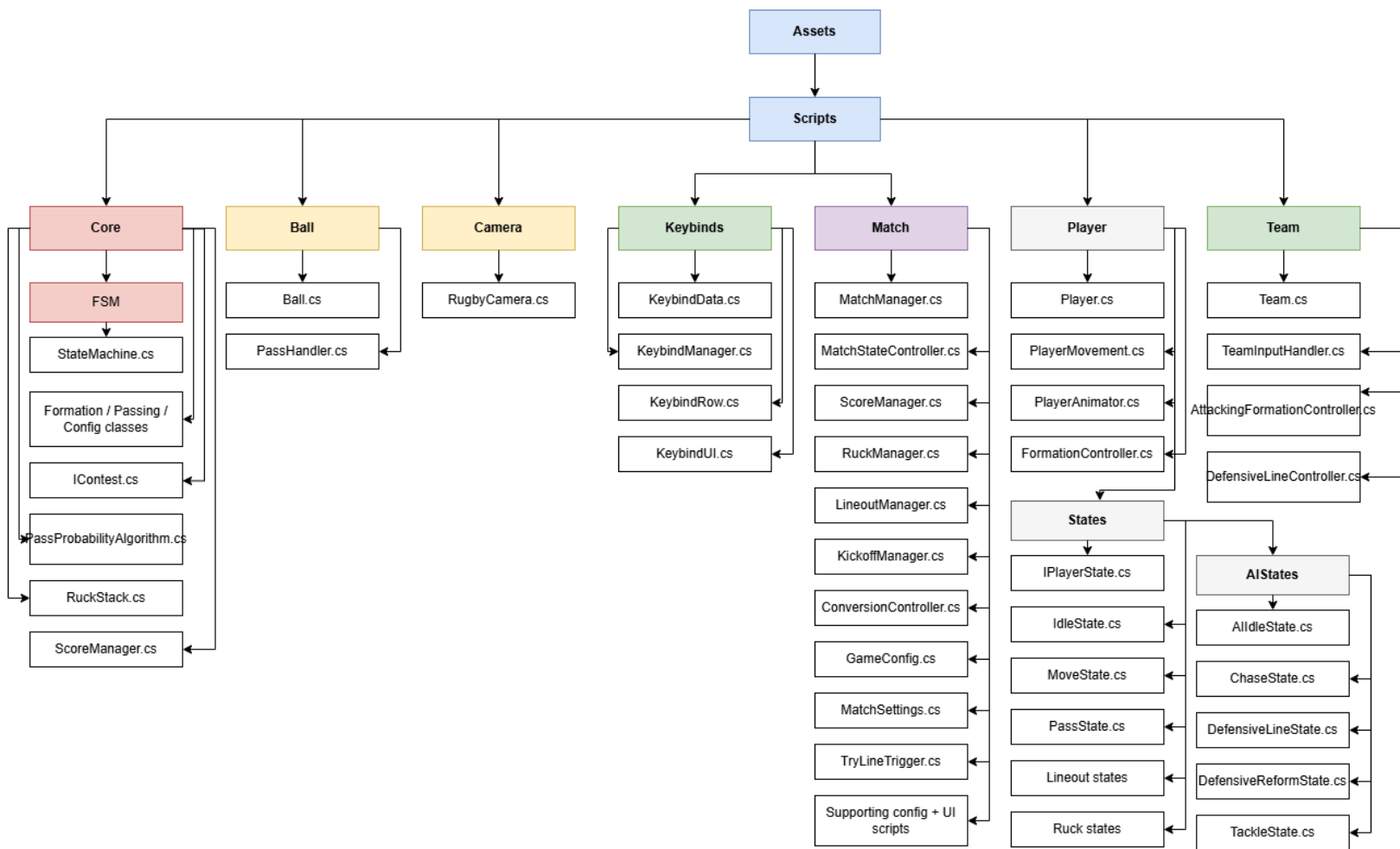
    END FOR

END METHOD
```

File Structure:

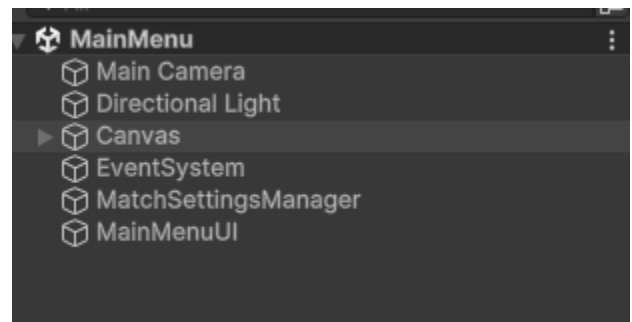
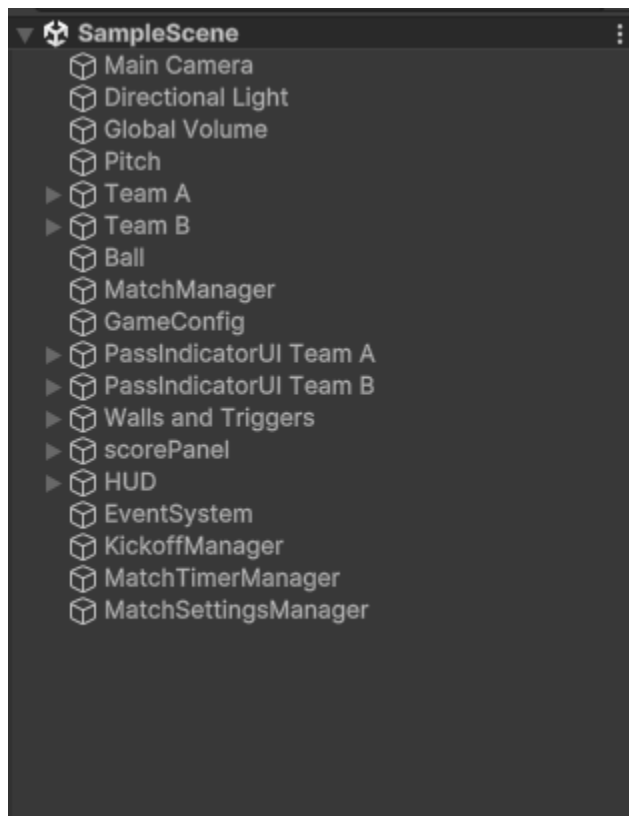
This project uses modular file structure organised by systems and responsibilities. Core functions like FSM, algorithms and shared interfaces are grouped together in the core folder while gameplay systems are separated into dedicated modules such as Ball, Match, Player etc.

Each subsystem divides further such as the Player folder containing movement, animation and state based architecture and the Match folder manages game flow systems such as rucks, lineouts and scoring.



Scene Structure:

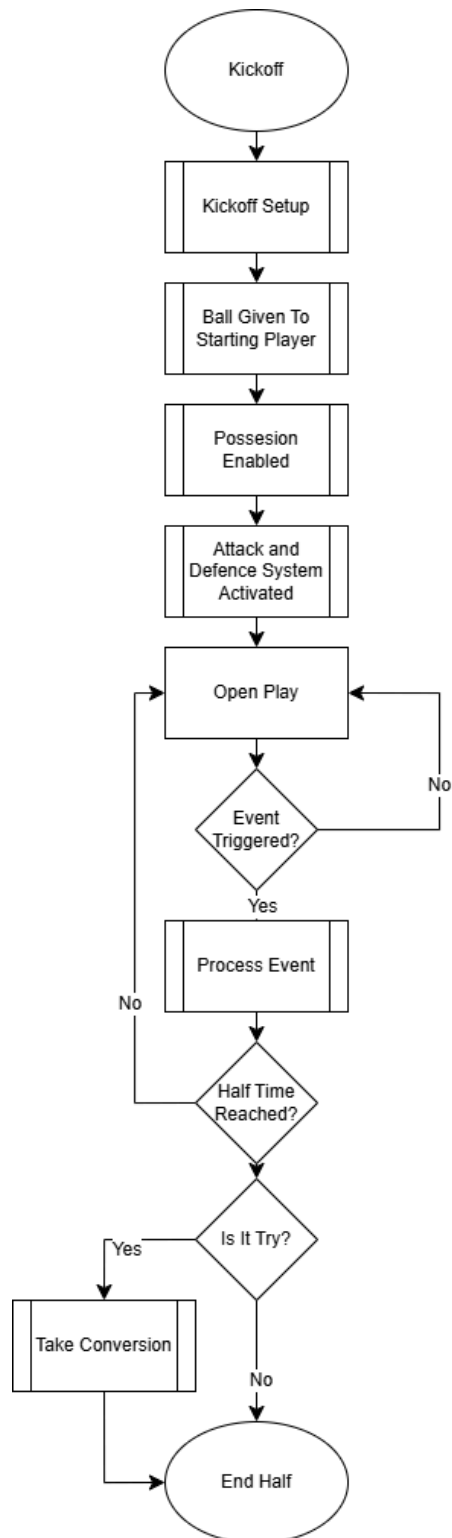
The scene structure mirrors the modular file structure, with gameplay systems, managers and UI elements separated. This ensures consistency between the code architecture and the in game object hierarchy. The left picture is the main scene where the game is and the right scene is the main menu which just has the match length slider.



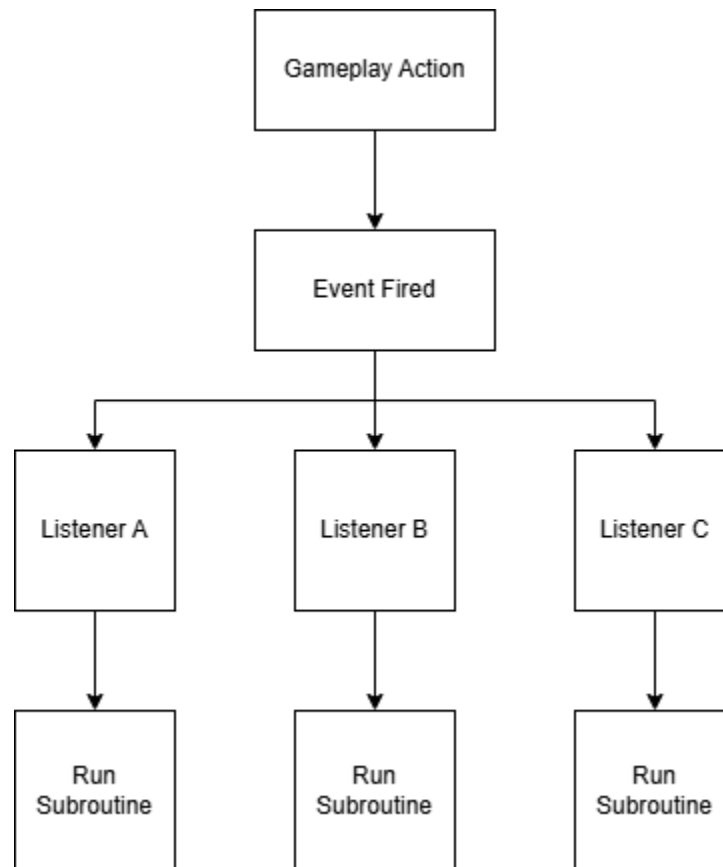
Key Flowcharts:

Key Gameloop Flowchart:

This is my gameloop flowchart which shows the overall flow of my game. Underneath I have added an event loop to show the different events that can happen and how they work.



This is a visualisation of how a unities event system works linking to the diagram above (process event part).



each event triggers a subroutine

My Code:

Technical Solution Overview:

The technical solution is a 2.5D version of a Rugby Union game built using C# and Unity. The overall system is composed of several different subsystems that interact with one another. The subsystems include match control, player control, ball control, AI, finite state machines, user control, and user interface/game management events.

The center of the overall system is the MatchManager. This is the primary coordinator for the overall match. This class is responsible for setting up the match, monitoring possession, registering rucks and lineouts, and updating when necessary as the state of play changes. This class works closely with the MatchStateController, which is responsible for enabling and disabling the attack and defense systems depending on possession.

In addition, the game uses an event driven architecture for important gameplay changes. For example, when the ball changes owner, the Ball object fires an OwnerChanged event. The MatchManager is listening for this event and updates which player has possession, this then fires the OnPossessionChanged event. Other subscribed systems such as MatchControllerState react by updating the controller states. This means the objects don't have to be constantly checking each other's states and only has to respond when a relevant event occurs.

Several systems use Finite State Machines (FSM's). These control the overall matchflow as well as each individual player. For example, gameplay transitions between states like kickoff, open play and ruck, while the players switch between idle, movement, pass, chase etc. Each state contains specific behaviour and transitions.

The ball system is another important aspect of the game. In this aspect, the ball is considered an object and handles various activities such as attaching, dropping, movement, and changing ownership. Passing is done through a PassHandler object using a probability algorithm for the result of the pass. In addition, the ball uses projectile-style movement for the simulation of the actual rugby pass.

There are various managers used for important rugby game activities. These include the RuckManager for managing the formation of a ruck when a tackle occurs and the LineoutManager for managing player positions and jumping order for lineouts. In all these activities, the required data structures are used appropriately, for example, stacks for the players involved in a ruck and a queue for lineouts.

Formation controllers are used to control the positioning and behaviour of the current non player controlled characters. The `AttackingFormationController` handles the player formation for the attacking team and the `DefensiveAIController` handles it for the defending team.

User input is managed through a custom key bind system which is stored in a JSON file. The `KeybindManager` loads the control mappings from the JSON file when the game is run and converts the key names into Unity `KeyCode` values. This allows the player to rebind controls and save them back to the JSON file.

The UI includes systems for displaying the scoreboard, pass indicators and QTE events.

Design Philosophy:

To ensure my code was clearly structured, and remained maintainable as the project grew with complexity, I followed a consistent internal architecture. This included separating responsibilities across different systems (input, state machine, and execution) and structuring methods using a validation → execution → state update pattern. I used AI to suggest an internal architecture structure which I then adapted and implemented across my project. For example, validation logic was separated into dedicated methods, core functionality was handled by system classes and debug methods were isolated at the end of each class. This ensured that each method had a single responsibility and that systems were loosely coupled making the code easier to read, test and debug.

Variables Table

Variable / Attribute	Type	Used In	Purpose
greenMaxDistance	float	PassProbabilityAlgorithm	Max distance for green pass zone.
yellowMaxDistance	float	PassProbabilityAlgorithm	Max distance for yellow pass zone.

distanceWeight	float	PassProbabilityAlgorithm	Controls how strongly pass distance affects final probability.
handlingWeight	float	PassProbabilityAlgorithm	Controls how much the passer's handling stat contributes to pass success.
difficultyWeight	float	PassProbabilityAlgorithm	Controls the effect of game difficulty on pass success chance.
distanceFalloffPower	float	PassProbabilityAlgorithm	Controls how quickly pass success drops as distance increases.
maxPassDistance	float	PassProbabilityAlgorithm	Maximum pass range considered by the algorithm.
minProbability	float	PassProbabilityAlgorithm	Lower clamp to stop probability becoming unrealistically low.
maxProbability	float	PassProbabilityAlgorithm	Upper clamp to stop probability becoming unrealistically high.
handling	float	PlayerStats	Player attribute used to affect pass accuracy.
speed	float	PlayerStats	Player attribute used to affect movement speed.
stamina	float	PlayerStats	Player attribute used to affect endurance.

aggression	float	PlayerStats	Player attribute affecting chase and tackle behaviour.
TeamAScore	int	ScoreManager	Stores the score for Team A.
TeamBScore	int	ScoreManager	Stores the score for Team B.
PointsPerTry	const int	ScoreManager	Stores the number of points awarded for a try.
PointsPerConversion	const int	ScoreManager	Stores the number of points awarded for a conversion.
tryFiredThisPhase	bool	ScoreManager	Prevents a try from being registered multiple times in the same phase.
pendingScoringTeam	Team	ScoreManager	Temporarily stores the team that has scored before conversion is resolved.
kickoffManager	KickoffManager	KickOffResetter	Reference used to reposition teams and restart play after scoring.
ruckPosition	Vector3	RuckStack	Stores the location of the ruck in game space.
stackDirection	Vector3	RuckStack	Stores the direction in which stacked ruck players are arranged.

lateralOffset	float	RuckStack	Separates stacked players sideways so they remain visible and look “organised”.
stack	Stack	RuckStack	Stores ruck players in LIFO order.
isUnloading	bool	RuckStack	Controls whether players are currently being released from the stack.
unloadInterval	float	RuckStack	Time gap between releasing players from the ruck stack.
unloadTimer	float	RuckStack	Tracks elapsed time before the next player is released.
Phase	LineoutPhase	LineoutManager	Tracks the current stage of the lineout lifecycle.
LineoutPosition	Vector3	LineoutManager	Stores the position where the lineout is being played.
throwingTeam	Team	LineoutManager	Stores the team putting the ball into the lineout.
defendingTeam	Team	LineoutManager	Stores the defending team in the lineout.
ball	Ball	LineoutManager	Reference to the current match ball used by the subsystem.

config	LineoutConfig / other config class	subsystem classes	Holds editable gameplay parameters so tuning can be done without rewriting code.
throwingQueue	LineoutJumpQueue	LineoutManager	Queue containing jump order for the throwing side.
defendingQueue	LineoutJumpQueue	LineoutManager	Queue containing jump order for the defending side.
throwingArrived	int	LineoutManager	Counts how many throwing-team players have reached their lineout positions.
defendingArrived	int	LineoutManager	Counts how many defending-team players have reached their lineout positions.
throwingExpected	int	LineoutManager	Stores how many players are expected for the throwing side.
defendingExpected	int	LineoutManager	Stores how many players are expected for the defending side.

Data Structure Table

Structure / Technique	Where Used	Why It Was Chosen	What It Shows
List	Teams, player collections, switching, iteration	Easy traversal of team members, supports looping and indexed access.	List operations and structured collection handling.
Stack	RuckStack	A ruck is modelled as LIFO, so the most recently joined player is released first.	Use of an advanced data structure to match how the real problem works.
Queue structure	LineoutManager via LineoutJumpQueue	Players join and are processed in ordered sequence for lineout behaviour.	Queue operations and ordered event handling.
Hash table / dictionary	Defensive pairing and assignment logic	Efficient mapping of one player to another for stable defensive marking with a lookup time of $O(1)$	A fast and efficient way to match players without searching through lists.
JSON file storage	Keybind system	Allows keybinds to be saved across games.	Files organised for direct access and separation of data from logic.
Event driven architecture	ScoreManager, possession flow, UI updates	Systems can react to gameplay changes without needing tightly	Loose coupling and interface based communication (meaning they don't need direct links).

		coupled dependencies.	
Finite State Machines	Human player, defensive AI, ruck, lineout, pause, game flow	To allow each part of the game to run independently so it's easier to understand and change.	Complex user-defined OOP and modular control logic.
Config classes	Pass algorithm and subsystems	Values can be tuned without changing algorithm code.	Keeps configuration separate from logic, making the system easier to change.
Object composition	Player, Ball, Team, controllers, managers	Game entities are made of multiple linked classes that each handle a specific responsibility.	Shows OOP design, composition, aggregation to create reusable components.
Interface abstraction	IContest and subsystem reuse	Allows different contest systems to share the same behaviour.	Abstraction, polymorphism and supporting different parts of the system being changed without needing to change everything.
Custom probability algorithm	PassProbabilityAlgorithm	Calculates pass success from distance, handling, and difficulty.	Complex user-defined algorithm.
Formation planning algorithm	AttackingFormationController	Positions attacking players relative to the carrier.	Complex user-defined algorithm and positional calculations.

Defensive pairing algorithm	DefensiveAIController	Assigns defenders to opponents efficiently.	Complex user-defined algorithm and hash table usage.
Defensive coding checks	Validation methods, null guards, early returns	Prevents runtime errors and invalid state transitions.	Robustness and safe programming practice.

Key Algorithms

Algorithm	Purpose	Implemented In
Pass Probability Algorithm	Determine whether the pass was a success or not and where the ball lands	PassProbabilityAlgorithm and used in PassHandler
Formation Planning Algorithm	Positions attacking support players relative to carrier	AttackingFormationController
Defensive Pairing Algorithm	Assigns defenders to attacking threats	DefensiveAIController

Class: Pass Probability Algorithm	
These global variables control the different characteristics of the algorithm (weighting etc). Keeping these as configurable parameters allows the algorithm to be	<pre> using Unity.Collections; using UnityEngine; public class PassProbabilityAlgorithm public float greenMaxDistance = 5f; public float yellowMaxDistance = 12f; public float distanceWeight = 0.6f; public float handlingWeight = 0.25f; //Player skills are actually used public float difficultyWeight = 0.15f; public float distanceFalloffPower = 1.8f; // How quickly </pre>

tuned without modifying core logic.	<pre> probability drops with distance (2 is closer to quadratic and 1 is linear) public float maxPassDistance = 25f; public float minProbability = 0.05f; public float maxProbability = 0.98f; </pre>
This method calculates the pass probability using distance, handling stat and game difficulty modifier.	<pre> public (float probability, PassZone zone) CalculatePassProbability(float distance, float handlingStat, float difficultyModifier = 1f) { float normalisedHandling = Mathf.Clamp01(handlingStat / 100f); //(0-1) float distanceFactor = CalculateDistanceFactor(distance); //(0-1) float handlingFactor = CalculateHandlingFactor(normalisedHandling); //(0-1) float difficultyFactor = CalculateDifficultyFactor(difficultyModifier); // 0.5 -> 1, 1-> 0.67, 2 -> 0.4 (makes it easier with the formula applied instead of 0.75,0.5,0.25 float rawProbability = distanceFactor * distanceWeight + handlingFactor * handlingWeight + difficultyFactor * difficultyWeight; float finalProbability = Mathf.Clamp(rawProbability, minProbability, maxProbability); PassZone zone = GetZoneForDistance(distance); return (finalProbability, zone); } </pre>
This method calculates the radius around the target the ball can land in if the pass fails.	<pre> //Calculates the expected miss radius if the pass fails public float CalculateMissRadius(float probability, float distance) { float baseRadius = distance * 0.15f; //Distance it will miss from base float inverseFactor = Mathf.Lerp(5f, 1.2f, probability); // p =0.9 -> x1.2 , p=0.1 -> x5 return baseRadius * inverseFactor; } </pre>
This method calculates how	<pre> //Uses falloff curve so longer passes are exponentially harder private float CalculateDistanceFactor(float distance) </pre>

the pass difficulty increases with the distance. Uses exponential falloff curve as explained in design.	<pre> { float normalisedDistance = Mathf.Clamp01(distance / maxPassDistance); float factor = 1f - Mathf.Pow(normalisedDistance, distanceFalloffPower); // creates the exponential curve return Mathf.Clamp01(factor); } </pre>
This method converts the players' handling stat into a probability factor.	<pre> private float CalculateHandlingFactor(float normalisedHandling) { float factor = Mathf.Pow(normalisedHandling, 0.85f); // 0.5 handling -> 0.55 factor, 0.9 handling -> 0.91 factor return Mathf.Clamp01(factor); } </pre>
This method adjusts probability depending on the difficulty modifier	<pre> private float CalculateDifficultyFactor(float difficultyModifier) { // When modifier = 0.5 -> 1/(0.5+0.5) = 1.0 // When modifier = 1.0 -> 1/(0.5+1) = 0.667 // When modifier = 2.0 -> 1/(0.5+2) = 0.4 float factor = 1f / (0.5f + difficultyModifier); return Mathf.Clamp01(factor); } </pre>
This enum categorises passes into 3 accuracy zones which are visualised with UI to indicate pass difficulty.	<pre> public enum PassZone { Green, Yellow, Red } </pre>

This section demonstrates the use of multiple advanced programming techniques such as hash tables, recursion, vector based calculations and OOP design.

Class: Formation planner	
Initialises formation planner and	<pre> using System.Collections.Generic; using System.Linq; using UnityEngine; </pre>

stores persistent data structures. The dictionary (hash table) is used to map players to their assigned lanes	<pre> public class FormationPlanner FormationSettings settings; //survives carrier changes (consistent throughout change) Dictionary<Player, PlayerSlotAssignment> stableAssignments = new Dictionary<Player, PlayerSlotAssignment>(); //Tracks who was available last tick to detect change List<Player> lastAvailablePlayers = new List<Player>(); public FormationPlanner(FormationSettings settings) { this.settings = settings; } </pre>
Main part of the algorithm. It returns a dictionary mapping each player to the world position	<pre> public Dictionary<Player, Vector3> CalculateFormation(Transform carrier, List<Player> availablePlayers, Vector3 attackDirection) { if (!ValidateInputs(carrier, availablePlayers)) { return new Dictionary<Player, Vector3>(); } Vector3 normalisedAttackDirection = NormaliseAttackDirection(attackDirection); List<Player> nonCarrierPlayers = ExcludeCarrier(availablePlayers, carrier); //Exclude carrier before detection so passing never triggers reshuffle if (HasAvailablePlayersChanged(nonCarrierPlayers)) { RebuildStableAssignments(nonCarrierPlayers); CacheAvailablePlayers(nonCarrierPlayers); } return BuildPositionsFromStableAssignments(carrier.position, normalisedAttackDirection, nonCarrierPlayers); } </pre>
Filters the list of players and removes the	<pre> private List<Player> ExcludeCarrier(List<Player> players, Transform carrier) { List<Player> result = new List<Player>(); </pre>

ball carrier	<pre>foreach (Player player in players) { if (player.transform != carrier) { result.Add(player); } } return result; }</pre>
Assigns players to their lanes while preserving previous assignments.	<pre>private void RebuildStableAssignments(List<Player> availablePlayers) { Dictionary<Player, PlayerSlotAssignment> newAssignments = new Dictionary<Player, PlayerSlotAssignment>(); int leftLane = 0; int rightLane = 0; availablePlayers.Sort(CompareByGroup); foreach (Player player in availablePlayers) { if (HasExistingAssignment(player)) { // Keep their existing side - only update lane index PlayerSlotAssignment existing = stableAssignments[player]; bool keepLeft = existing.isLeft; int lane; if (keepLeft) { lane = leftLane; leftLane++; } else { lane = rightLane; rightLane++; } newAssignments[player] = new PlayerSlotAssignment(lane, keepLeft); } } }</pre>

	<pre> else { // New player - assign to whichever side has fewer people bool assignLeft = leftLane <= rightLane; int lane; if (assignLeft) { lane = leftLane; leftLane++; } else { lane = rightLane; rightLane++; } newAssignments[player] = new PlayerSlotAssignment(lane, assignLeft); } stableAssignments = newAssignments; } </pre>
This compares the forwards from backs and if the player is a forwards, they get the score 0 meaning they are assigned first.	<pre> private int CompareByGroup(Player a, Player b) { int aScore; int bScore; if (a.playerGroup == PlayerGroup.Forward) aScore = 0; else aScore = 1; if (b.playerGroup == PlayerGroup.Forward) bScore = 0; else bScore = 1; return aScore.CompareTo(bScore); } </pre>
Converts lane assignments into world positions relative to the ball carrier using directional	<pre> // Converts lane assignments into positions in world (anchored around carrier) private Dictionary<Player, Vector3> BuildPositionsFromStableAssignments(Vector3 carrierPosition, Vector3 attackDirection, List<Player> availablePlayers) { Dictionary<Player, Vector3> positions = new Dictionary<Player, Vector3>(); } </pre>

vectors	<pre> Vector3 lateral = CalculateLateralDirection(attackDirection); BuildPositionsRecursive(availablePlayers, 0, carrierPosition, attackDirection, lateral, positions); return positions; } </pre>
Recursively processes each player to calculate their target position.	<pre> private void BuildPositionsRecursive(List<Player> availablePlayers, int index, Vector3 carrierPosition, Vector3 attackDirection, Vector3 lateral, Dictionary<Player, Vector3> positions) { if (availablePlayers == null index >= availablePlayers.Count) { return; } Player player = availablePlayers[index]; if (player != null && stableAssignments.ContainsKey(player)) { PlayerSlotAssignment assignment = stableAssignments[player]; Vector3 slotPosition = CalculateSlotPosition(carrierPosition, attackDirection, lateral, assignment); positions[player] = slotPosition; } BuildPositionsRecursive(availablePlayers, index + 1, carrierPosition, attackDirection, lateral, positions); } </pre>
Calculates the final position of a player by combining depth and lateral offsets relative to the carrier forming the attacking shape.	<pre> //Calculates position in world of single slot private Vector3 CalculateSlotPosition(Vector3 carrierPosition, Vector3 attackDirection, Vector3 lateral, PlayerSlotAssignment assignment) { float depth = CalculateDepth(assignment.laneIndex); float width = CalculateWidth(assignment.laneIndex); float side; // Position behind carrier Vector3 position = carrierPosition - (attackDirection * depth); </pre>

	<pre>// Offset left or right if (assignment.isLeft) { side = -1f; } else { side = 1f; } Vector3 sideOffset = lateral * width * side; position += sideOffset; return position; }</pre>
Compares current and previous player sets using hash sets to detect changes (regardless of order).	<pre>private bool HasAvailablePlayersChanged(List<Player> currentPlayers) { if (currentPlayers.Count != lastAvailablePlayers.Count) { return true; } HashSet<Player> currentSet = new HashSet<Player>(currentPlayers); // order doesn't matter so used to check if changes were made (players becoming unavailable) HashSet<Player> lastSet = new HashSet<Player>(lastAvailablePlayers); return !currentSet.SetEquals(lastSet); }</pre>
These are getters and setters used in the program	<pre>private void CacheAvailablePlayers(List<Player> players) { lastAvailablePlayers = new List<Player>(players); } private bool HasExistingAssignment(Player player) { return stableAssignments.ContainsKey(player); } private float CalculateDepth(int laneIndex) { return settings.firstLaneDepth + (laneIndex * settings.depthIncrement); } private float CalculateWidth(int laneIndex)</pre>

	<pre> { float baseWidth = (settings.forwardBaseWidth + settings.backBaseWidth) / 2f; return baseWidth + (laneIndex * settings.widthIncrement); } private Vector3 CalculateLateralDirection(Vector3 attackDirection) { return Vector3.Cross(attackDirection, Vector3.up).normalized; } private Vector3 NormaliseAttackDirection(Vector3 attackDirection) { Vector3 normalised = attackDirection; normalised.y = 0f; if (normalised.sqrMagnitude < 0.01f) normalised = Vector3.forward; return normalised.normalized; } </pre>
Ensures that the required inputs are valid before running the algorithm. This is an object representing player line assignment which has laneIndex and side.	<pre> private bool ValidateInputs(Transform carrier, List<Player> players) { if (carrier == null) { DebugLogNullCarrier(); return false; } if (players == null players.Count == 0) { return false; } return true; } private class PlayerSlotAssignment { public int laneIndex; public bool isLeft; public PlayerSlotAssignment(int lane, bool left) { laneIndex = lane; isLeft = left; } } </pre>
This outputs a	<pre> private void DebugLogNullCarrier() </pre>

warning if the carrier is missing	<pre> { Debug.LogWarning("FormationPlanner: carrier is null, cannot calculate formation"); } </pre>
public class FormationFollower : MonoBehaviour	
This class is the movement helper that makes the AI players move towards their assigned positions. Once the formation system calculates a target position this class stores it and checks whether the player can still follow. It moves the player towards then stops the player if it is within the arrivalThreshold.	<pre> using UnityEngine; public class FormationFollower : MonoBehaviour private Player player; private PlayerMovement movement; private FormationSettings settings; private Vector3? targetPosition; private bool hasTarget; public void Initialise(Player player, FormationSettings settings) { this.player = player; movement = player.movement; this.settings = settings; ClearTarget(); } public void SetTargetPosition(Vector3 position) { targetPosition = position; hasTarget = true; } public void ClearTarget() { targetPosition = null; hasTarget = false; } public bool HasTarget() { return hasTarget && targetPosition.HasValue; } </pre>
	<pre> public void FollowFormation() { if (!ValidateCanFollow()) { return; } } </pre>

	<pre> Vector3 directionToTarget = CalculateDirectionToTarget(); float distanceToTarget = CalculateDistanceToTarget(); if (HasArrivedAtTarget(distanceToTarget)) { StopMovement(); return; } MoveTowardTarget(directionToTarget); } </pre>
	<pre> private Vector3 CalculateDirectionToTarget() { Vector3 direction = targetPosition.Value - player.transform.position; direction.y = 0; return direction.normalized; } private float CalculateDistanceToTarget() { Vector3 horizontalDifference = targetPosition.Value - player.transform.position; horizontalDifference.y = 0; return horizontalDifference.magnitude; } private bool HasArrivedAtTarget(float distance) { return distance < settings.arrivalThreshold; } private void MoveTowardTarget(Vector3 direction) { if (movement == null) return; Vector2 moveInput = ConvertDirectionToInput(direction); //convert 3D direction to 2D input bool sprint = false; movement.Move(moveInput, sprint); } private Vector2 ConvertDirectionToInput(Vector3 direction) { //PlayerMovement uses input.y = up/down (x) and input.x = left/right (z) // direction.x right // direction.z forward return new Vector2(-direction.z, direction.x); } private void StopMovement() { if (movement != null) </pre>

	<pre> { movement.Move(Vector2.zero, false); } } </pre>
	<pre> private bool ValidateCanFollow() { if(!hasTarget !targetPosition.HasValue) return false; if (player == null) { DebugLogMissingPlayer(); return false; } if (movement == null) { DebugLogMissingMovement(); return false; } return true; } private void DebugLogMissingPlayer() { Debug.LogWarning("FormationFollower: Missing player reference"); } private void DebugLogMissingMovement() { Debug.LogWarning("FormationFollower: Player missing PlayerMovement component"); } public void DebugDrawTarget() { if (!hasTarget !targetPosition.HasValue) return; Gizmos.color = Color.cyan; Gizmos.DrawWireSphere(targetPosition.Value, 0.5f); Gizmos.DrawLine(player.transform.position, targetPosition.Value); } </pre>

Defensive Pairing Algorithm Defensive Line Controller	
Initialises the defensive system and	<pre> using UnityEngine; using System.Collections.Generic; </pre>

stores key data structures. Dictionary (hash table) used to map defenders to assigned opponents. The lastDefenders and lastOpponents are cached from the previous frame so the algorithm can check for changes without needing to recompute the assignment every tick.	<pre> public class DefensiveLineController : MonoBehaviour public Team team; public Team opposingTeam; public bool drawGizmos = true; Dictionary<Player, Player> opponentAssignments; List<Player> lastDefenders = new List<Player>(); List<Player> lastOpponents = new List<Player>(); bool assignmentsDirty = true; // something has changed recalculate private void Awake() { InitialiseAssignments(); } private void OnEnable() { MarkAssignmentsDirty(); } private void FixedUpdate() { UpdateDefensiveLine(); } private void InitialiseAssignments() { opponentAssignments = new Dictionary<Player, Player>(); } private void MarkAssignmentsDirty() { assignmentsDirty = true; } </pre>
This method runs every tick and it is defensively programmed with early exit guard clauses (there is a lineout or ruck) and stops the method so it doesn't affect these systems.	<pre> private void UpdateDefensiveLine() { if (MatchManager.Instance != null && MatchManager.Instance.IsConversionActive()) return; if (MatchManager.Instance != null && MatchManager.Instance.IsLineoutActive()) return; if (!ValidateTeams()) return; List<Player> defenders = GetAvailableDefenders(); List<Player> opponents = GetAvailableOpponents(); if (!HasEnoughPlayersForLine(defenders, opponents)) { ClearAllDefensiveStates(); return; } } </pre>

I have then separated the code into <code>RebuildStableAssignments</code> and <code>ApplyDefensiveStates</code> so the pairing and movement logic is handled by other subroutines. This means the movement is updated every frame but the pairing only runs when there is a change.	<pre> if (ShouldRebuildAssignments(defenders, opponents)) { RebuildAssignments(defenders, opponents); } ApplyDefensiveStates(defenders); } </pre>
In <code>ShouldRebuildAssignments</code>, they are only rebuilt if the <code>assignmentsDirty</code> flag is set. This means this algorithm doesn't run every tick.	<pre> private bool HasEnoughPlayersForLine(List<Player> defenders, List<Player> opponents) { return defenders.Count > 0 && opponents.Count > 0; } private bool ShouldRebuildAssignments(List<Player> defenders, List<Player> opponents) { return assignmentsDirty HaveDefendersChanged(defenders) HaveOpponentsChanged(opponents); } private void RebuildAssignments(List<Player> defenders, List<Player> opponents) { AssignOpponentsToDefenders(defenders, opponents); CachePlayerLists(defenders, opponents); assignmentsDirty = false; } private bool HaveDefendersChanged(List<Player> defenders) { return !PlayerListMatches(defenders, lastDefenders); } private bool HaveOpponentsChanged(List<Player> opponents) { return !PlayerListMatches(opponents, lastOpponents); } private bool PlayerListMatches(List<Player> a, List<Player> b) </pre>

	<pre> { if (a.Count != b.Count) return false; for (int i = 0; i < a.Count; i++) { if (a[i] != b[i]) return false; } return true; } private void CachePlayerLists(List<Player> defenders, List<Player> opponents) { lastDefenders = new List<Player>(defenders); lastOpponents = new List<Player>(opponents); } </pre>
Both the lists are sorted by X position (left to right). This means the leftmost defender is paired with the leftmost attacker every time. I have also used <code>Mathf.Min</code> to handle when the team sizes are uneven so any unpaired defenders are removed	<pre> private void AssignOpponentsToDefenders(List<Player> defenders, List<Player> opponents) { opponentAssignments.Clear(); SortByXPosition(defenders); SortByXPosition(opponents); int pairCount = Mathf.Min(defenders.Count, opponents.Count); AssignPairedDefenders(defenders, opponents, pairCount); ClearUnpairedDefenders(defenders, pairCount); } </pre>
I used the built in sort function with custom comparisons where the players are sorted by their X coord. Without this it would mean a	<pre> private void SortByXPosition(List<Player> players) { players.Sort(CompareByX); } private int CompareByX(Player a, Player b) { return a.transform.position.x.CompareTo(b.transform.position.x); } private void AssignPairedDefenders(List<Player> defenders, List<Player> opponents, int pairCount) </pre>

player on left could be paired with a player on the right causing the line to collapse.	<pre> { for (int i = 0; i < pairCount; i++) { opponentAssignments[defenders[i]] = opponents[i]; } } private void ClearUnpairedDefenders(List<Player> defenders, int pairCount) { for (int i = pairCount; i < defenders.Count; i++) { ClearDefenderState(defenders[i]); } } private void ApplyDefensiveStates(List<Player> defenders) { List<Player> allDefenders = new List<Player>(defenders); foreach (Player defender in defenders) { TryApplyDefensiveState(defender, allDefenders); } } </pre>
More defensive programming shown with the guard clauses are the start.	<pre> private void TryApplyDefensiveState(Player defender, List<Player> allDefenders) { if (!opponentAssignments.ContainsKey(defender)) return; if (!ShouldControlPlayer(defender)) return; if (IsPlayerInLineout(defender)) return; Player assignedOpponent = opponentAssignments[defender]; EnsurePlayerInDefensiveLineState(defender, assignedOpponent, allDefenders); } </pre>
The method follows the FSM state priorities set. This higher priority states are Chase, Tackle, in a ruck or in the defensive line	<pre> private void EnsurePlayerInDefensiveLineState(Player defender, Player opponent, List<Player> allDefenders) { if (IsInHigherPriorityState(defender)) return; if (IsInDefensiveLineState(defender)) { UpdateExistingDefensiveLineState(defender, opponent, allDefenders); } else { EnterDefensiveLineState(defender, opponent, allDefenders); } } </pre>

	<pre> } } </pre>
I use null conditional operators (?.) throughout my code to silently skip the call instead of calling a <code>NullReferenceException</code>. An example of where I use this is (<pre> lineState?.UpdateAssignment(opponent, allDefenders); </pre>)so it skips the call silently if the state has already been exited.	<pre> private bool IsInHigherPriorityState(Player defender) { return IsInChaseOrTackle(defender) IsInRuckState(defender); } private bool IsInChaseOrTackle(Player defender) { var current = defender.stateMachine.SM.CurrentState; return current is ChaseState current is TackleState; } private bool IsInRuckState(Player defender) { var current = defender.stateMachine.SM.CurrentState; return current is RuckBoundState current is RuckJoinState current is RuckPlayState; } private bool IsInDefensiveLineState(Player defender) { return defender.stateMachine.SM.CurrentState is DefensiveLineState; } private void UpdateExistingDefensiveLineState(Player defender, Player opponent, List<Player> allDefenders) { var lineState = defender.stateMachine.SM.CurrentState as DefensiveLineState; lineState?.UpdateAssignment(opponent, allDefenders); } private void EnterDefensiveLineState(Player defender, Player opponent, List<Player> allDefenders) { defender.stateMachine.SM.SetState(new DefensiveLineState(defender.stateMachine, opponent, allDefenders)); DebugLogAssignment(defender, opponent); } </pre>
	<pre> private void ClearDefenderState(Player defender) { if (!CanClearDefenderState(defender)) return; if (IsPlayerInLineout(defender)) return; if (IsInDefensiveLineState(defender)) { defender.stateMachine.SM.SetState(new AIIIdleState(defender.stateMachine)); } } </pre>

	<pre>} private bool CanClearDefenderState(Player defender) { return defender != null && !defender.isControlled; } private void ClearAllDefensiveStates() { if (team?.players == null) return; foreach (Player defender in team.players) { ClearDefenderState(defender); } opponentAssignments.Clear(); MarkAssignmentsDirty(); }</pre>
	<pre>private List<Player> GetAvailableDefenders() { List<Player> available = new List<Player>(); if (team?.players == null) return available; foreach (Player p in team.players) { if (IsAvailableForDefensiveLine(p)) available.Add(p); } return available; } private List<Player> GetAvailableOpponents() { List<Player> available = new List<Player>(); if (opposingTeam?.players == null) return available; foreach (Player p in opposingTeam.players) { if (p != null) available.Add(p); } return available; }</pre>
	<pre>private bool IsAvailableForDefensiveLine(Player player) { if (player == null) return false; if (player.isControlled) return false; if (IsPlayerInLineout(player)) return false; return true; }</pre>

	<pre>private bool ShouldControlPlayer(Player player) { if (player == null) return false; if (player.isControlled) return false; if (IsPlayerInLineout(player)) return false; return true; } private bool IsPlayerInLineout(Player player) { var current = player.stateMachine.SM.CurrentState; return current is LineoutJoinState current is LineoutBoundState; } private bool ValidateTeams() { if (team == null) { DebugLogMissingTeam("team"); return false; } if (opposingTeam == null) { DebugLogMissingTeam("opposingTeam"); return false; } return true; }</pre>
	<pre>private void DebugLogAssignment(Player defender, Player opponent) { //Debug.Log(\$"{defender.name} assigned to mark {opponent.name}."); } private void DebugLogMissingTeam(string fieldName) { Debug.LogWarning(\$"DefensiveLineController: {fieldName} is not assigned."); }</pre>
Gizmos are used to visually debug my defensive AI system. I used this to make sure the code worked as intended in real	<pre>private void OnDrawGizmos() { if (!drawGizmos) return; if (team?.players == null) return; foreach (Player defender in team.players) { if (defender == null) continue;</pre>

time so I could see how everything reacted.	<pre> DefensiveLineState lineState = defender.stateMachine?.SM?.CurrentState as DefensiveLineState; if (lineState == null) continue; // Cyan cube at the slot target Vector3 target = lineState.GetDebugTarget(); Gizmos.color = Color.cyan; Gizmos.DrawWireCube(target, Vector3.one * 0.4f); // Line from defender to slot target Gizmos.color = new Color(0f, 1f, 1f, 0.5f); Gizmos.DrawLine(defender.transform.position, target); // Red line to assigned opponent + sphere on them Player opponent = lineState.GetDebugOpponent(); if (opponent != null) { Gizmos.color = Color.red; Gizmos.DrawLine(defender.transform.position, opponent.transform.position); Gizmos.DrawWireSphere(opponent.transform.position, 0.25f); } } } </pre>
---	---

State Machine architecture

Public class StateMachine

This method is the core of my FSM architecture as it controls all the transitions between states.

The current state is stored as an interface type

```

using UnityEngine;

public class StateMachine

public IPlayerState CurrentState { get; private set; }
public void SetState(IPlayerState newState)
{
    if (newState == CurrentState) return;

    // ?. (null conditional operator) only do this if thing on
left is NOT null
    CurrentState?.Exit();
    CurrentState = newState;
    // Enter new state
    CurrentState.Enter();
}

```

(IPlayerState). This allows different states to be used interchangeably due to polymorphism. The state machine uses composition by holding references to the active state and the behaviour is carried out in the actual state not in this class. The null conditional operator means that the method is only called if the current state is exited.	<pre> } public void Tick() { CurrentState?.Tick(); } public void FixedTick() { CurrentState?.FixedTick(); } </pre>
This interface creates a defined structure that all my states must follow, meaning they can be used by the state machine in any order. This creates loose coupling meaning new	<pre> using UnityEngine; public interface IPlayerState { void Enter(); void Tick(); void FixedTick(); void Exit(); } </pre>

states can be added without needing to change the state machine.	
public class PlayerStateMachine : MonoBehaviour	
This class is the player specific version of the generic FSM. It is an example of aggregation as it holds and manages a state machine which is part of the player, but the player can act independently even though it is using the same structure.	<pre> using UnityEngine; public class PlayerStateMachine : MonoBehaviour Player player; StateMachine stateMachine; public StateMachine SM { get { return stateMachine; } } public Vector2 moveInput { get; private set; } public bool sprintInput { get; private set; } public bool passRequested { get; private set; } public bool passLeftSide { get; private set; } public bool tackleRequested { get; private set; } public void Initialise(Player player) { this.player = player; stateMachine = new StateMachine(); } </pre>
	<pre> private void Start() { stateMachine.SetState(new IdleState(this)); } private void Update() { stateMachine.Tick(); } private void FixedUpdate() { stateMachine.FixedTick(); } </pre>
	<pre> //Called by TeamInputHandler public void SetInput(Vector2 move, bool sprint) </pre>

	<pre> { moveInput = move; sprintInput = sprint; } //Request a pass public void RequestPass(bool leftSide) { passRequested = true; passLeftSide = leftSide; } </pre>
	<pre> public void ClearPassRequest() { passRequested = false; } public void RequestTackle() { tackleRequested = true; } public void ClearTackleRequest() { tackleRequested = false; } public Player GetPlayer() { return player; } public Team GetTeam() { return player.team; } </pre>
Human Player States	
public class IdleState: IPlayerState	
Each state handles only one behaviour meaning the code is easier to debug and maintain. Each state also only handles one	<pre> using Unity.VisualScripting; using UnityEngine; public class IdleState: IPlayerState readonly PlayerStateMachine fsm; public IdleState(PlayerStateMachine fsm) { this.fsm = fsm; } </pre>

thing and the transitions are all condition based.	<pre> } public void Enter() { UpdateAnimatorToIdle(); } public void Tick() { if (MatchManager.Instance != null && MatchManager.Instance.IsConversionActive()) return; if (ShouldTransitionToMove()) { TransitionToMoveState(); return; } if (ShouldTransitionToPass()) { TransitionToPassState(); return; } } public void FixedTick() { } public void Exit() { } </pre>
	<pre> private void UpdateAnimatorToIdle() { Player player = fsm.GetPlayer(); // ?. means if animController is null it doesn't crash player.animController?.UpdateMovement(false); } private bool ShouldTransitionToMove() { return fsm.moveInput.sqrMagnitude > 0.01f; } private bool ShouldTransitionToPass() { return fsm.passRequested; } private void TransitionToMoveState() { fsm.GetPlayer().stateMachine.SM.SetState(new MoveState(fsm)); } // Switches to pass state </pre>

	<pre> private void TransitionToPassState() { fsm.GetPlayer().stateMachine.SM.SetState(new PassState(fsm)); } </pre>
public class MoveState : IPlayerState	
	<pre> using UnityEngine; public class MoveState : IPlayerState { readonly PlayerStateMachine fsm; public MoveState(PlayerStateMachine fsm) { this.fsm = fsm; } public void Enter() { UpdateAnimatorToMoving(); } public void Tick() { if (ShouldTransitionToIdle()) { TransitionToIdleState(); return; } } public void FixedTick() { if (MatchManager.Instance != null && MatchManager.Instance.IsConversionActive()) return; ExecuteMovement(); } public void Exit() { UpdateAnimatorToIdle(); } } </pre>
	<pre> private void UpdateAnimatorToMoving() { Player player = fsm.GetPlayer(); player.animController?.UpdateMovement(true); } private void UpdateAnimatorToIdle() </pre>

	<pre> { Player player = fsm.GetPlayer(); player.animController?.UpdateMovement(false); } private void ExecuteMovement() { Player player = fsm.GetPlayer(); player.movement?.Move(fsm.moveInput, fsm.sprintInput); } private bool ShouldTransitionToIdle() { return fsm.moveInput.sqrMagnitude <= 0.01f; } private void TransitionToIdleState() { Player player = fsm.GetPlayer(); player.stateMachine.SM.SetState(new IdleState(fsm)); } </pre>
public class PassState : IPlayerState	
The pass is turned into a dedicated phase in this class where the pass is validated then the target is selected. This then triggers the pass animation. A timer is also used to sync up the pass animation.	<pre> using UnityEngine; public class PassState : IPlayerState readonly PlayerStateMachine fsm; GameConfig config; Player targetPlayer; Vector3 groundTargetPos; bool hasPassed; float timer; public PassState(PlayerStateMachine fsm) { this.fsm = fsm; } public void Enter() { if (!ValidateCanPass()) { ExitToIdleState(); return; } } </pre>

	<pre> InitialisePassState(); DeterminePassTarget(); StartPassAnimation(); } public void Tick() { AdvanceTimer(); if (ShouldExecutePass()) { ExecutePass(); } if (ShouldExitState()) { TransitionToNextState(); } } public void FixedTick() { } public void Exit() { } private void InitialisePassState() { config = FindConfig(); timer = 0; hasPassed = false; } </pre>
	<pre> private void DeterminePassTarget() { Player passer = fsm.GetPlayer(); Team team = fsm.GetTeam(); targetPlayer = FindPlayerTarget(passer, team); if (HasPlayerTarget()) { groundTargetPos = targetPlayer.transform.position; } else { groundTargetPos = CalculateGroundTarget(passer, team); } } </pre>
	<pre> private Player FindPlayerTarget(Player passer, Team team) { if (team == null) return null; return team.FindBestPassTarget(passer, fsm.passLeftSide); } </pre>

	<pre>private Vector3 CalculateGroundTarget(Player passer, Team team) { if (team == null) return passer.transform.position; return team.CalculateGroundPassTarget(passer, fsm.passLeftSide); } private void StartPassAnimation() { fsm.ClearPassRequest(); TriggerPlayerPassAnimation(); } private void TriggerPlayerPassAnimation() { Player player = fsm.GetPlayer(); player.animController?.TriggerPass(); } private void AdvanceTimer() { timer += Time.deltaTime; } private bool ShouldExecutePass() { return !hasPassed && timer >= config.passAnimationDelay; }</pre>
	<pre>private void ExecutePass() { Ball ball = GetBall(); if (!ValidateBallExists(ball)) return; Player passer = fsm.GetPlayer(); float handlingStat = GetPasserHandlingStat(passer); float difficultyModifier = 1f; if (HasPlayerTarget()) { RequestPlayerPass(ball, handlingStat, difficultyModifier); } else { RequestGroundPass(ball, handlingStat, difficultyModifier); } hasPassed = true; }</pre>

	<pre> private void RequestPlayerPass(Ball ball, float handlingStat, float difficultyModifier) { ball.passHandler.RequestPassToPlayer(targetPlayer.transform, handlingStat, difficultyModifier); } private void RequestGroundPass(Ball ball, float handlingStat, float difficultyModifier) { ball.passHandler.RequestPassToGround(groundTargetPos, handlingStat, difficultyModifier); } private float GetPasserHandlingStat(Player passer) { if (passer != null) return passer.handlingStat; if (config != null) return config.defaultHandlingStat; return 70f; } </pre>
	<pre> private bool ShouldExitState() { return timer >= config.passStateExitTime; } private void TransitionToNextState() { IPlayerState nextState = DetermineNextState(); fsm.GetPlayer().stateMachine.SM.SetState(nextState); } private IPlayerState DetermineNextState() { if (IsPlayerMoving()) { return new MoveState(fsm); } else { return new IdleState(fsm); } } </pre>
	<pre> private bool IsPlayerMoving() { return fsm.moveInput.sqrMagnitude > 0.01f; } private Ball GetBall() { return MatchManager.Instance?.ball; } private GameConfig FindConfig() </pre>

	<pre>{ if (MatchManager.Instance != null) { return MatchManager.Instance.config; } return Object.FindAnyObjectByType<GameConfig>(); } private bool HasPlayerTarget() { return targetPlayer != null; } private void ExitToIdleState() { fsm.ClearPassRequest(); fsm.GetPlayer().stateMachine.SM.SetState(new IdleState(fsm)); }</pre>
	<pre>private bool ValidateCanPass() { Player player = fsm.GetPlayer(); Ball ball = GetBall(); if (!ValidateBallExists(ball)) { return false; } if (!ValidatePlayerHasBall(player, ball)) { DebugPlayerDoesNotHaveBall(player); return false; } return true; } private bool ValidateBallExists(Ball ball) { if (ball == null) { DebugBallBecameNull(); ExitToIdleState(); return false; } return true; } private bool ValidatePlayerHasBall(Player player, Ball ball) { return ball.currentHolder == player.transform; }</pre>

	<pre>private void DebugPlayerDoesNotHaveBall(Player player) { Debug.LogWarning(\$"{player.name} tried to pass but doesn't have the ball"); } private void DebugBallBecameNull() { Debug.LogWarning("Ball became null during pass state"); }</pre>
public class HumanDefendState : IPlayerState	
	<pre>using UnityEngine; public class HumanDefendState : IPlayerState readonly PlayerStateMachine fsm; readonly Team opposingTeam; const float TackleRange = 1.2f; public HumanDefendState(PlayerStateMachine fsm, Team opposingTeam) { this.fsm = fsm; this.opposingTeam = opposingTeam; } public void Enter() { SetMovingAnimation(false); } public void Tick() { if (ShouldExitToAI()) { TransitionToAIIdle(); return; } if (HasTackleBeenRequested()) { HandleTackleRequest(); } } public void FixedTick() { ProcessMovement(); }</pre>

	<pre>public void Exit() { StopRigidbody(); SetMovingAnimation(false); fsm.ClearTackleRequest(); }</pre>
	<pre>private void ProcessMovement() { Vector2 input = fsm.moveInput; bool sprint = fsm.sprintInput; fsm.GetPlayer().movement.Move(input, sprint); UpdateMovementAnimation(input); } private void UpdateMovementAnimation(Vector2 input) { bool isMoving = input.sqrMagnitude > 0.01f; fsm.GetPlayer().animController?.UpdateMovement(isMoving); }</pre>
	<pre>private void HandleTackleRequest() { fsm.ClearTackleRequest(); Player nearestOpponent = FindNearestOpponentInRange(); if (nearestOpponent == null) { DebugLogTackleMissed(); return; } TransitionToTackle(nearestOpponent); }</pre>
	<pre>private Player FindNearestOpponentInRange() { if (opposingTeam?.players == null) return null; Player nearest = null; float nearestDistance = TackleRange; foreach (Player opponent in opposingTeam.players) { if (opponent == null) continue; float distance = CalculateDistanceTo(opponent);</pre>

	<pre> if (distance < nearestDistance) { nearestDistance = distance; nearest = opponent; } return nearest; } </pre>
	<pre> private float CalculateDistanceTo(Player target) { Vector3 diff = target.transform.position - fsm.GetPlayer().rb.position; diff.y = 0f; return diff.magnitude; } private bool HasTackleBeenRequested() { return fsm.tackleRequested; } private bool ShouldExitToAI() { return !fsm.GetPlayer().isControlled; } private void TransitionToTackle(Player opponent) { fsm.SM.SetState(new TackleState(fsm, opponent)); } private void TransitionToAIIdle() { fsm.SM.SetState(new AIIdleState(fsm)); } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb != null) player.rb.linearVelocity = Vector3.zero; } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } </pre>
	<pre> private void DebugLogTackleMissed() { </pre>

	<pre> Debug.Log(\$"{fsm.GetPlayer().name} attempted tackle but no opponent in range."); } </pre>
public class RuckJoinState: IPlayerState	
This state controls a player joining the ruck. It moves the player to a target position until they arrive then switches them to RuckBoundState if they are a support player. If they are not it triggers the scrumhalf logic.	<pre> using UnityEngine; public class RuckJoinState: IPlayerState readonly PlayerStateMachine fsm; RuckManager ruckManager; Vector3 targetPosition; bool isScrumhalf; const float MoveSpeed = 5f; const float ArrivalThreshold = 0.3f; public RuckJoinState(PlayerStateMachine fsm, RuckManager ruckManager, Vector3 targetPosition, bool isScrumhalf) { this.fsm = fsm; this.ruckManager = ruckManager; this.targetPosition = targetPosition; this.isScrumhalf = isScrumhalf; } public void Enter() { SetMovingAnimation(true); } public void Tick() { if (HasArrived()) { NotifyArrival(); } } public void FixedTick() { if (!HasArrived()) { MoveTowardsTarget(); } } public void Exit() { </pre>

	<pre> SetMovingAnimation(false) ; } </pre>
	<pre> private void MoveTowardsTarget() { Player player = fsm.GetPlayer(); Vector3 direction = CalculateDirection(); Vector3 newPosition = player.rb.position + direction * MoveSpeed * Time.fixedDeltaTime; player.rb.MovePosition(newPosition); UpdateSpriteDirection(direction); } private Vector3 CalculateDirection() { Vector3 direction = targetPosition - fsm.GetPlayer().rb.position; direction.y = 0f; return direction.normalized; } private float CalculateDistance() { Vector3 diff = targetPosition - fsm.GetPlayer().rb.position; diff.y = 0f; return diff.magnitude; } </pre>
	<pre> private bool HasArrived() { return CalculateDistance() <= ArrivalThreshold; } private void NotifyArrival() { if (isScrumhalf) { ruckManager.NotifyScrumhalfArrived(); } else { NotifySupportArrived(); EnterRuckBoundState(); } } private void NotifySupportArrived() { Team team = fsm.GetTeam(); ruckManager.NotifySupportArrived(fsm.GetPlayer(), team); } </pre>

	<pre> } private void EnterRuckBoundState() { fsm.SM.SetState(new RuckBoundState(fsm)); } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } private void UpdateSpriteDirection(Vector3 direction) { Player player = fsm.GetPlayer(); if (player.sr == null) return; if (direction.z < -0.01f) player.sr.flipX = false; if (direction.z > 0.01f) player.sr.flipX = true; } </pre>
public class RuckBoundState: IPlayerState	
This State is used to bind players to a ruck. It repeatedly stops the movement of players, keeping them stationary.	<pre> using UnityEngine; public class RuckBoundState: IPlayerState readonly PlayerStateMachine fsm; public RuckBoundState(PlayerStateMachine fsm) { this.fsm = fsm; } public void Enter() { StopRigidbody(); SetMovingAnimation(false); } public void Tick() { } public void FixedTick() { StopRigidbody(); } public void Exit() { SetMovingAnimation(false); } </pre>

	<pre> private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb == null) return; player.rb.linearVelocity = Vector3.zero; player.rb.angularVelocity = Vector3.zero; } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } </pre>
public class RuckPlayState: IPlayerState	
This state is where the scrumhalf can pass from the ruck. It locks the movement of the scrumhalf and waits for a pass to be requested. This switches it to match state and then the ruck manager is told that the ball has been played.	<pre> using UnityEngine; public class RuckPlayState: IPlayerState readonly PlayerStateMachine fsm; RuckManager ruckManager; public RuckPlayState(PlayerStateMachine fsm, RuckManager ruckManager) { this.fsm = fsm; this.ruckManager = ruckManager; } public void Enter() { StopRigidbody(); SetMovingAnimation(false); ZeroMoveInput(); } public void Tick() { if (HasPassBeenRequested()) { ExecutePass(); } } public void FixedTick() { StopRigidbody(); } public void Exit() { </pre>

	<pre>fsm.ClearPassRequest(); }</pre>
	<pre>private bool HasPassBeenRequested() { return fsm.passRequested; } private void ExecutePass() { fsm.ClearPassRequest(); TransitionToPassState(); ruckManager.NotifyPassMade(); } private void TransitionToPassState() { fsm.SM.SetState(new PassState(fsm)); } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb == null) return; player.rb.linearVelocity = Vector3.zero; } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } private void ZeroMoveInput() { fsm.SetInput(Vector2.zero, false); }</pre>
public class LineoutJoinState: IPlayerState	
I created this state to move players from their current position to the lineout, but they weren't setting up in the right formation.	<pre>using UnityEngine; public class LineoutJoinState: IPlayerState PlayerStateMachine fsm; LineoutManager lineoutManager; Vector3 targetPosition; bool isAttacking; float moveSpeed; float arrivalThreshold;</pre>

So in Lineout Manager I made a <code>SendTeamToSlots</code> method which teleports all the players to their slots. To try and fix the issue. This means this state is no longer used.	<pre> public LineoutJoinState(PlayerStateMachine fsm, LineoutManager lineoutManager, Vector3 targetPosition, bool isAttacking, float moveSpeed, float arrivalThreshold) { this.fsm = fsm; this.lineoutManager = lineoutManager; this.targetPosition = targetPosition; this.isAttacking = isAttacking; this.moveSpeed = moveSpeed; this.arrivalThreshold = arrivalThreshold; } public void Enter() { Debug.Log(\$"{fsm.GetPlayer().name} entered LineoutJoinState, target={targetPosition}"); SetMovingAnimation(true); } public void Tick() { if (HasArrived()) { Debug.Log(\$"{fsm.GetPlayer().name} HasArrived triggered"); NotifyArrival(); } } public void FixedTick() { if (!HasArrived()) { MoveTowardsTarget(); } } public void Exit() { SetMovingAnimation(false); StopRigidbody(); } </pre>
	<pre> private void MoveTowardsTarget() { Player player = fsm.GetPlayer(); Vector3 direction = CalculateDirection(); Vector3 newPosition = player.rb.position + direction * moveSpeed * Time.fixedDeltaTime; player.rb.MovePosition(newPosition); UpdateSpriteDirection(direction); } </pre>

	<pre>} private Vector3 CalculateDirection() { Vector3 direction = targetPosition - fsm.GetPlayer().rb.position; direction.y = 0f; return direction.normalized; } private float CalculateDistance() { Vector3 diff = targetPosition - fsm.GetPlayer().rb.position; diff.y = 0f; return diff.magnitude; }</pre>
	<pre>private bool HasArrived() { return CalculateDistance() <= arrivalThreshold; } private void NotifyArrival() { Player player = fsm.GetPlayer(); Debug.Log(\$"{fsm.GetPlayer().name} arrived at lineout slot"); fsm.SM.SetState(new LineoutBoundState(fsm)); lineoutManager.NotifyPlayerArrived(fsm.GetPlayer(), isAttacking); } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb != null) player.rb.linearVelocity = Vector3.zero; } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } private void UpdateSpriteDirection(Vector3 direction) { Player player = fsm.GetPlayer(); if (player.sr == null) return; if (direction.z < -0.01f) player.sr.flipX = false; if (direction.z > 0.01f) player.sr.flipX = true; }</pre>

public class LineoutBoundState: IPlayerState	
This state fixes players to the lineout by stopping their movement until the lineout is resolved.	<pre> using UnityEngine; public class LineoutBoundState: IPlayerState { PlayerStateMachine fsm; public LineoutBoundState(PlayerStateMachine fsm) { this.fsm = fsm; } public void Enter() { StopRigidbody(); SetMovingAnimation(false); } public void Tick() { } public void FixedTick() { StopRigidbody(); } public void Exit() { SetMovingAnimation(false); } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb == null) return; player.rb.linearVelocity = Vector3.zero; player.rb.angularVelocity = Vector3.zero; } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } } </pre>
AI defensive States	
public class DefensiveLineState : IPlayerState	

This state controls the defensive line behaviour. The FixedTick is used for the movement and keeping defenders in line whereas the Tick is used to check to see if the player should actually stay in the defensive line or go to ChaseState if the player is too close.	<pre> using UnityEngine; using System.Collections.Generic; public class DefensiveLineState : IPlayerState readonly PlayerStateMachine fsm; Player assignedOpponent; List<Player> defensiveLine; const float moveSpeed = 4f; const float chaseEntryDistanceMin = 1.5f; const float chaseEntryDistanceMax = 4f; const float arrivalThreshold = 0.1f; public DefensiveLineState(PlayerStateMachine fsm, Player assignedOpponent, List<Player> defensiveLine) { this.fsm = fsm; this.assignedOpponent = assignedOpponent; this.defensiveLine = defensiveLine; } public void Enter() { SetMovingAnimation(true); } public void Tick() { if (ShouldExitToHumanControl()) { TransitionToIdle(); return; } if (ShouldEnterChase()) { TransitionToChase(); return; } if (HasOpponentBrokenThrough()) { TransitionToReform(); } } public void FixedTick() { if (!ValidateOpponent()) return; Vector3 target = CalculateLineTarget(); </pre>
--	---

	<pre> MoveTowardsTarget(target); } public void Exit() { StopRigidbody(); SetMovingAnimation(false); } </pre>
	<pre> public void UpdateAssignment(Player newOpponent, List<Player> newLine) { assignedOpponent = newOpponent; defensiveLine = newLine; } </pre>
	<pre> private bool HasOpponentBrokenThrough() { if (!ValidateOpponent()) return false; Player player = fsm.GetPlayer(); Vector3 attackDir = GetAttackDirection(); Vector3 toOpponent = assignedOpponent.transform.position - player.rb.position; toOpponent.y = 0f; // Opponent is behind = they've beaten the line float dot = Vector3.Dot(toOpponent.normalized, attackDir); return dot > 0.5f && toOpponent.magnitude > 2f; } </pre>
	<pre> private Vector3 GetAttackDirection() { if (assignedOpponent?.team == null) return Vector3.forward; Vector3 dir = assignedOpponent.team.attackDirection; dir.y = 0f; if (dir.sqrMagnitude < 0.01f) { return Vector3.forward; } return dir.normalized; } private void TransitionToReform() { fsm.SM.SetState(new DefensiveReformState(fsm, assignedOpponent)); } </pre>

The target's position is found with the three axis values. The X position is then maintained for the man to man marking that I use. The Y stays the same and the Z is calculated as the average Z of all the players in the defensive line. If the defensive line list is empty or null then the Z position is just the player's Z position. This is another example of defensive programming	<pre>private Vector3 CalculateLineTarget() { float targetX = GetOpponentX(); float targetY = GetPlayerY(); float targetZ = CalculateAverageLineZ(); return new Vector3(targetX, targetY, targetZ); } private float GetOpponentX() { return assignedOpponent.transform.position.x; } private float GetPlayerY() { return fsm.GetPlayer().rb.position.y; } private float CalculateAverageLineZ() { if (!IsValidDefensiveLine()) return GetPlayerZ(); return SumLineZ() / CountValidLineMembers(); }</pre>
	<pre>private bool IsValidDefensiveLine() { return defensiveLine != null && defensiveLine.Count > 0 && CountValidLineMembers() > 0; } private float SumLineZ() { float total = 0f; foreach (Player p in defensiveLine) { if (p != null) total += p.transform.position.z; } return total; }</pre>

	<pre>private int CountValidLineMembers() { int count = 0; foreach (Player p in defensiveLine) { if (p != null) count++; } return count; }</pre>
	<pre>private float GetPlayerZ() { return fsm.GetPlayer().rb.position.z; }</pre>
	<pre>private void MoveTowardsTarget(Vector3 target) { float distance = CalculateDistance(target); if (HasArrivedAtTarget(distance)) { StopRigidbody(); return; } Vector3 direction = CalculateDirection(target); ApplyMovement(direction); UpdateSpriteDirection(direction); }</pre>
	<pre>private float CalculateDistance(Vector3 target) { Vector3 diff = target - fsm.GetPlayer().rb.position; diff.y = 0f; return diff.magnitude; } private bool HasArrivedAtTarget(float distance) { return distance < arrivalThreshold; } private Vector3 CalculateDirection(Vector3 target) { Vector3 direction = target - fsm.GetPlayer().rb.position; direction.y = 0f; return direction.normalized; } private void ApplyMovement(Vector3 direction) { Player player = fsm.GetPlayer();</pre>

	<pre> Vector3 newPosition = player.rb.position + direction * moveSpeed * Time.fixedDeltaTime; player.rb.MovePosition(newPosition); } </pre>
	<pre> private bool ShouldEnterChase() { if (!ValidateOpponent()) return false; if (MatchManager.Instance != null && MatchManager.Instance.IsRuckActive()) return false; //if (IsOpponentAlreadyBeingChased()) return false; return DistanceToOpponent() <= GetChaseEntryDistance(); } </pre>
Mathf.Lerp is linear interpolation and I use this by normalising the aggression stat then using it between the min and max values I want to use (in this example chase distance.) I also use it for the chase speed, and tackle radius.	<pre> private float GetChaseEntryDistance() { float t = fsm.GetPlayer().stats.aggression / 100f; return Mathf.Lerp(chaseEntryDistanceMin, chaseEntryDistanceMax, t); } private bool ShouldExitToHumanControl() { return fsm.GetPlayer().isControlled; } private float DistanceToOpponent() { Vector3 diff = assignedOpponent.transform.position - fsm.GetPlayer().rb.position; diff.y = 0f; return diff.magnitude; } private void TransitionToChase() { fsm.SM.SetState(new ChaseState(fsm, assignedOpponent)); } private void TransitionToIdle() { fsm.SM.SetState(new IdleState(fsm)); } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb != null) player.rb.linearVelocity = Vector3.zero; } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } </pre>

	<pre> private void UpdateSpriteDirection(Vector3 direction) { Player player = fsm.GetPlayer(); if (player.sr == null) return; if (direction.z < -0.01f) player.sr.flipX = false; if (direction.z > 0.01f) player.sr.flipX = true; } </pre>
	<pre> private bool ValidateOpponent() { return assignedOpponent != null; } // Gizmo helpers - called by // DefensiveLineController.OnDrawGizmos public Vector3 GetDebugTarget() { if (!ValidateOpponent()) return Vector3.zero; return CalculateLineTarget(); } public Player GetDebugOpponent() { return assignedOpponent; } </pre>
public class ChaseState: IPlayerState	
This state manages an AI defender chasing the ball carrier. The FixedTick is used for moving towards the player and Tick is used for deciding if it should stay in this state or transition (control changed, player within tackle range)	<pre> using UnityEngine; public class ChaseState: IPlayerState { readonly PlayerStateMachine fsm; readonly Player targetOpponent; const float chaseSpeedMin = 5f; const float chaseSpeedMax = 8f; const float tackleRadiusMin = 1.2f; const float tackleRadiusMax = 2.8f; public ChaseState(PlayerStateMachine fsm, Player targetOpponent) { this.fsm = fsm; this.targetOpponent = targetOpponent; } public void Enter() { SetMovingAnimation(true); DebugLogEnter(); } } </pre>

	<pre>public void Tick() { if (ShouldExitToHumanControl()) { TransitionToIdle(); return; } if (ShouldExitBecauseTargetLostBall()) { TransitionToDefensiveLine(); return; } if (ShouldEnterTackle()) { TransitionToTackle(); } } public void FixedTick() { if (targetOpponent == null) return; MoveTowardsOpponent(); } public void Exit() { StopRigidbody(); SetMovingAnimation(false); }</pre>
	<pre>public bool IsTargeting(Player opponent) { return targetOpponent == opponent; } private void MoveTowardsOpponent() { Player player = fsm.GetPlayer(); Vector3 direction = CalculateDirection(); Vector3 newPosition = player.rb.position + direction * GetChaseSpeed(player) * Time.fixedDeltaTime; player.rb.MovePosition(newPosition); UpdateSpriteDirection(direction); } private Vector3 CalculateDirection() { </pre>

	<pre> Vector3 direction = targetOpponent.transform.position - fsm.GetPlayer().rb.position; direction.y = 0f; return direction.normalized; } </pre>
	<pre> private bool ShouldEnterTackle() { if (targetOpponent == null) return false; return DistanceToOpponent() <= GetTackleRadius(fsm.GetPlayer()); } private float GetChaseSpeed(Player player) { float t = player.stats.aggression / 100f; return Mathf.Lerp(chaseSpeedMin, chaseSpeedMax, t); } private float GetTackleRadius(Player player) { float t = player.stats.aggression / 100f; return Mathf.Lerp(tackleRadiusMin, tackleRadiusMax, t); } private bool ShouldExitToHumanControl() { return fsm.GetPlayer().isControlled; } private bool ShouldExitBecauseTargetLostBall() { if (targetOpponent == null) return true; return BallPickedUpByOtherPlayer(); } private bool BallPickedUpByOtherPlayer() { Ball ball = GetBall(); if (ball == null) return false; if (ball.currentHolder == null) return false; // loose ball - stay in chase return ball.currentHolder != targetOpponent.transform; // someone else has it } private float DistanceToOpponent() { Vector3 diff = targetOpponent.transform.position - fsm.GetPlayer().rb.position; diff.y = 0f; return diff.magnitude; } </pre>

	<pre> private void TransitionToTackle() { DebugLogTackleEntry(); fsm.SM.SetState(new TackleState(fsm, targetOpponent)); } private void TransitionToIdle() { fsm.SM.SetState(new IdleState(fsm)); } private void TransitionToDefensiveLine() { fsm.SM.SetState(new AIIdleState(fsm)); } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb != null) player.rb.linearVelocity = Vector3.zero; } private void SetMovingAnimation(bool isMoving) { fsm.GetPlayer().animController?.UpdateMovement(isMoving); } private void UpdateSpriteDirection(Vector3 direction) { Player player = fsm.GetPlayer(); if (player.sr == null) return; if (direction.z < -0.01f) player.sr.flipX = false; if (direction.z > 0.01f) player.sr.flipX = true; } private Ball GetBall() { return MatchManager.Instance?.ball; } </pre>
	<pre> private void DebugLogEnter() { Debug.Log(\$"{fsm.GetPlayer().name} entering ChaseState, chasing {targetOpponent?.name}."); } private void DebugLogTackleEntry() { Debug.Log(\$"{fsm.GetPlayer().name} entering TackleState, tackling {targetOpponent?.name}. Distance: {DistanceToOpponent()}"); } </pre>

	<pre> } </pre>
<pre> public class TackleState : IPlayerState </pre>	
This state is used to go from open play to the ruck system.	<pre> using UnityEngine; public class TackleState : IPlayerState readonly PlayerStateMachine fsm; readonly Player targetOpponent; RuckManager ruckManager; public TackleState(PlayerStateMachine fsm, Player targetOpponent) { this.fsm = fsm; this.targetOpponent = targetOpponent; } public void Enter() { StartRuck(); } public void Tick() { } public void FixedTick() { } public void Exit() { StopRigidbody(); } </pre>
It builds a new RuckManager and subscribes to ruck completion events. This shows loose coupling through event driven completion.	<pre> private void StartRuck() { if (!ValidateRuckCanStart()) { return; } ruckManager = BuildRuckManager(); SubscribeToRuckComplete(); ruckManager.StartRuck(); MatchManager.Instance.RegisterRuck(ruckManager); } private RuckManager BuildRuckManager() { Ball ball = GetBall(); </pre>

	<pre> GameConfig config = GetConfig(); Team attackingTeam = FindAttackingTeam(); Team defendingTeam = fsm.GetTeam(); return new RuckManager(targetOpponent, fsm.GetPlayer(), attackingTeam, defendingTeam, ball, config.ruckConfig); } private void SubscribeToRuckComplete() { ruckManager.OnRuckComplete += OnRuckComplete; } private void OnRuckComplete() { ruckManager.OnRuckComplete -= OnRuckComplete; fsm.SM.SetState(new AIIdleState(fsm)); } </pre>
	<pre> private Team FindAttackingTeam() { return targetOpponent?.team; } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb != null) player.rb.linearVelocity = Vector3.zero; } private Ball GetBall() { return MatchManager.Instance?.ball; } private GameConfig GetConfig() { return MatchManager.Instance?.config; } </pre>
This is used to make sure a ruck cannot be made if another ruck is already active or if any of the other reasons said in the code are happening. This is another	<pre> private bool ValidateRuckCanStart() { if (MatchManager.Instance.IsRuckActive()) { DebugLogRuckActive(); return false; } if (GetBall() == null) { DebugLogMissingBall(); return false; } } </pre>

example of defensive programming.	<pre> if (targetOpponent == null) { DebugLogMissingOpponent(); return false; } if (GetConfig()?.ruckConfig == null) { DebugLogMissingConfig(); return false; } return true; } </pre>
	<pre> private void DebugLogRuckActive() { Debug.LogWarning("TackleState: Ruck already active, ignoring tackle."); } private void DebugLogMissingBall() { Debug.LogWarning("TackleState: Ball is null, cannot start ruck."); } private void DebugLogMissingOpponent() { Debug.LogWarning("TackleState: Target opponent is null, cannot start ruck."); } private void DebugLogMissingConfig() { Debug.LogWarning("TackleState: RuckConfig missing from GameConfig."); } </pre>
public class AIIdleState : IPlayerState	
This is the default state for AI controlled players.	<pre> using UnityEngine; public class AIIdleState : IPlayerState { readonly PlayerStateMachine fsm; public AIIdleState(PlayerStateMachine fsm) { this.fsm = fsm; } public void Enter() </pre>

	<pre>{ UpdateAnimatorToIdle(); } public void Tick() { } public void FixedTick() { } public void Exit() { }</pre>
	<pre>private void UpdateAnimatorToIdle() { fsm.GetPlayer().animController?.UpdateMovement(false); }</pre>
public class AIFormationState: IPlayerState	
	<pre>using UnityEngine; public class AIFormationState: IPlayerState readonly PlayerStateMachine fsm; Vector3 targetPosition; bool hasTarget; const float ArrivalThreshold = 0.3f; const float FollowSpeed = 4f; public AIFormationState(PlayerStateMachine fsm) { this.fsm = fsm; } public void Enter() { fsm.GetPlayer().animController?.UpdateMovement(true); } public void Tick() { if (ShouldExitToHumanControl()) { TransitionToIdleForHuman(); } } public void FixedTick() {</pre>

	<pre>if (!hasTarget) return; float distance = CalculateDistance(); if (HasArrived(distance)) { StopAndIdle(); return; } MoveTowardsTarget(); } public void Exit() { StopRigidbody(); fsm.GetPlayer().animController?.UpdateMovement(false); }</pre>
	<pre>public void UpdateTarget(Vector3 newTarget) { targetPosition = newTarget; hasTarget = true; } public void ClearTarget() { hasTarget = false; }</pre>
	<pre>private void MoveTowardsTarget() { Player player = fsm.GetPlayer(); Vector3 direction = CalculateDirection(); Vector3 newPosition = player.rb.position + direction * FollowSpeed * Time.fixedDeltaTime; player.rb.MovePosition(newPosition); UpdateSpriteDirection(direction); } private Vector3 CalculateDirection() { Player player = fsm.GetPlayer(); Vector3 direction = targetPosition - player.rb.position; direction.y = 0f; return direction.normalized; } private float CalculateDistance() { Player player = fsm.GetPlayer();</pre>

	<pre>Vector3 diff = targetPosition - player.rb.position; diff.y = 0f; return diff.magnitude; }</pre>
	<pre>private bool HasArrived(float distance) { return distance < ArrivalThreshold; } private void StopAndIdle() { StopRigidbody(); fsm.GetPlayer().animController?.UpdateMovement(false); } private void StopRigidbody() { Player player = fsm.GetPlayer(); if (player.rb != null) { player.rb.linearVelocity = Vector3.zero; } } private void UpdateSpriteDirection(Vector3 direction) { Player player = fsm.GetPlayer(); if (player.sr == null) return; if (direction.z < -0.01f) player.sr.flipX = false; if (direction.z > 0.01f) player.sr.flipX = true; }</pre>
	<pre>private bool ShouldExitToHumanControl() { return fsm.GetPlayer().isControlled; } private void TransitionToIdleForHuman() { fsm.GetPlayer().stateMachine.SM.SetState(new IdleState(fsm)); }</pre>
	<pre>public void DebugDrawTarget(Vector3 playerPosition) { if (!hasTarget) return; Gizmos.color = Color.cyan; Gizmos.DrawWireSphere(targetPosition, 0.5f); Gizmos.DrawLine(playerPosition, targetPosition); }</pre>

Match Flow	
public class MatchManager : MonoBehaviour	
	<pre> using System; using UnityEngine; public class MatchManager : MonoBehaviour public static MatchManager Instance { get; private set; } public RuckManager GetActiveRuck() { return activeRuck; } public Team TeamA, TeamB; public Ball ball; public GameConfig config; public Team PossessionTeam { get; private set; } public event Action<Team> OnPossessionChanged; RuckManager activeRuck; LineoutManager activeLineout; </pre>
This was one of the first classes I coded and I ran into a problem where stuff was running in the wrong order so had to learn about the difference between start and update and left it in my code so I could check back.	<pre> // Start is called once before the first execution of Update // after the MonoBehaviour is created private void Start() { // Kickoff: give ball to first player on Team A (or whoever // you want) if (ball == null TeamA == null TeamA.players.Count == 0) return; //ball.AttachTo(TeamA.players[0].transform); } private void Awake() { if (Instance == null) { Instance = this; } else { </pre>

	<pre> Destroy(gameObject); //Destroys duplicates (not itself) so there are no double updates/confliciting states etc (only 1 manager) } if (ball != null) { ball.OwnerChanged += OnBallOwnerChanged; } }</pre>
	<pre>public void RegisterRuck(RuckManager ruck) { activeRuck = ruck; ruck.OnRuckComplete += ClearActiveRuck; ruck.OnRuckComplete += ReEvaluateControllers; }</pre>
	<pre>public void SetConversionActive(bool active) { conversionActive = active; } bool conversionActive; public bool IsConversionActive() { return conversionActive; } private void ReEvaluateControllers() { // Re-trigger possession logic now the ruck is done OnPossessionChanged?.Invoke(PossessionTeam); } private void ClearActiveRuck() { activeRuck = null; } public void RegisterLineout(LineoutManager lineout) { activeLineout = lineout; } public void ClearActiveLineout() { activeLineout = null; } public bool IsLineoutActive() { return activeLineout != null; }</pre>

	<pre> } private void Update() { activeRuck?.Tick(Time.deltaTime); } private void OnDestroy() { if (ball != null) { ball.OwnerChanged -= OnBallOwnerChanged; } } </pre>
	<pre> private void OnBallOwnerChanged(Transform oldHolder, Transform newHolder) { if (oldHolder != null) { var oldPlayer = oldHolder.GetComponentInParent<Player>(); oldPlayer?.SetHasBall(false); //if (oldPlayer != null) oldPlayer.SetHasBall(false); } if (newHolder != null) { var newPlayer = newHolder.GetComponentInParent<Player>(); newPlayer?.SetHasBall(true); //if (newPlayer != null) newPlayer.SetHasBall(true); // Set possession to the player's team (requires player.Team to be set) if (newPlayer != null && newPlayer.team != null) SetPossession(newPlayer.team); else SetPossession(null); // if team not wired yet } else { // Ball is loose SetPossession(null); } } </pre>
This method is only used when possession actually	<pre> private void SetPossession(Team newTeam) { if (newTeam == PossessionTeam) return; PossessionTeam = newTeam; } </pre>

changes and uses guards to avoid unnecessary controller swaps if possession hasn't actually changed teams.	<pre> if (IsRuckActive() IsLineoutActive()) return; OnPossessionChanged?.Invoke(PossessionTeam); //Invoke calls all the subscribed methods in order } public bool IsRuckActive() { return activeRuck != null; } </pre>
public class MatchStateController : MonoBehaviour	
	<pre> using UnityEngine; public class MatchStateController : MonoBehaviour public AttackingFormationController teamAAttackController; public DefensiveLineController teamADefenceController; public AttackingFormationController teamBAttackController; public DefensiveLineController teamBDefenceController; private void Awake() { ValidateReferences(); } private void Start() { SubscribeToPossessionEvents(); SetInitialControllerStates(); } private void OnDestroy() { UnsubscribeFromPossessionEvents(); } </pre>
	<pre> private void SetInitialControllerStates() { if (MatchManager.Instance == null) return; Team startingTeam = MatchManager.Instance.PossessionTeam; ApplyControllerStates(startingTeam); } private void SubscribeToPossessionEvents() { if (MatchManager.Instance == null) return; </pre>

	<pre> MatchManager.Instance.OnPossessionChanged += OnPossessionChanged; } private void UnsubscribeFromPossessionEvents() { if (MatchManager.Instance == null) return; MatchManager.Instance.OnPossessionChanged -= OnPossessionChanged; } private void OnPossessionChanged(Team possessingTeam) { //Debug.Log(\$"MatchStateController: possession changed to {possessingTeam?.name}, isLineoutActive={MatchManager.Instance.IsLineoutActive()}, isRuckActive={MatchManager.Instance.IsRuckActive()}"); ApplyControllerStates(possessingTeam); DebugLogPossessionSwap(possessingTeam); } </pre>
	<pre> private void ApplyControllerStates(Team possessingTeam) { if (MatchManager.Instance.IsConversionActive()) return; if (MatchManager.Instance.IsRuckActive()) return; if (MatchManager.Instance.IsLineoutActive()) { DisableAllControllers(); return; } Debug.Log(\$"ApplyControllerStates: teamAAttacking={IsTeamA(possessingTeam)}"); if (possessingTeam == null) { // Don't disable controllers if a ruck is in progress if (MatchManager.Instance.IsRuckActive()) return; DisableAllControllers(); return; } bool teamAAttacking = IsTeamA(possessingTeam); Debug.Log(\$"ApplyControllerStates: teamAAttacking={teamAAttacking}, " + \$"teamAAttack={teamAAttackController?.enabled}, " + \$"teamADefence={teamADefenceController?.enabled}, " + \$"teamBAttack={teamBAttackController?.enabled}, " + \$"teamBDefence={teamBDefenceController?.enabled}"); SetTeamAControllers(attacking: teamAAttacking); } </pre>

	<pre> SetTeamBControllers(attacking: !teamAAttacking); } </pre>
	<pre> private void SetTeamAControllers(bool attacking) { SetControllerActive(teamAAttackController, attacking); SetControllerActive(teamADefenceController, !attacking); } private void SetTeamBControllers(bool attacking) { SetControllerActive(teamBAttackController, attacking); SetControllerActive(teamBDefenceController, !attacking); } private void SetControllerActive(MonoBehaviour controller, bool active) { if (controller == null) return; controller.enabled = active; } private void DisableAllControllers() { SetControllerActive(teamAAttackController, false); SetControllerActive(teamADefenceController, false); SetControllerActive(teamBAttackController, false); SetControllerActive(teamBDefenceController, false); } </pre>
	<pre> private bool IsTeamA(Team team) { if (MatchManager.Instance == null) return false; return team == MatchManager.Instance.TeamA; } private void ValidateReferences() { if (teamAAttackController == null) DebugLogMissing("teamAAttackController"); if (teamADefenceController == null) DebugLogMissing("teamADefenceController"); if (teamBAttackController == null) DebugLogMissing("teamBAttackController"); if (teamBDefenceController == null) DebugLogMissing("teamBDefenceController"); } </pre>
	<pre> private void DebugLogPossessionSwap(Team team) { </pre>

	<pre> string teamName; if (team != null) { teamName = team.name; } else { teamName = "none"; } Debug.Log(\$"Possession changed to: {teamName}. Controllers swapped."); } private void DebugLogMissing(string fieldName) { Debug.LogWarning(\$"MatchStateController: {fieldName} is not assigned."); } </pre>
public class KickoffManager : MonoBehaviour	
I used logs here for quick tests to check if the settings where getting applied correctly	<pre> using UnityEngine; using System.Collections.Generic; public class KickoffManager : MonoBehaviour [SerializeField] private MatchTimerUI matchTimerUI; [SerializeField] private LineoutController lineoutController; public Team attackingTeam; public Team defendingTeam; public float defensiveLineOffset = 10f; public float playerSpacing = 2f; public float groundY = 0f; public Vector3 kickoffCentre = Vector3.zero; private void Start() { MatchSettingsManager.Instance?.ApplySettings(); //Debug.Log(\$"Timer after ApplySettings: HalfLengthSeconds={MatchTimerManager.HalfLengthSeconds}, ClockSpeed={MatchTimerManager.ClockSpeed}"); //Debug.Log(\$"MatchSettingsManager.HalfLengthMinutes={MatchSett ingsManager.Instance?.HalfLengthMinutes}"); PositionTeamsForKickoff(); GiveBallToFirstAttacker(); MatchTimerManager.Instance?.StartTimer(); MatchTimerManager.Instance.OnHalfTime += StartSecondHalf; } </pre>

	<pre> } private void OnDestroy() { if (MatchTimerManager.Instance != null) MatchTimerManager.Instance.OnHalfTime -= StartSecondHalf; } </pre>
	<pre> public void PositionTeamsForKickoff() { if (!ValidateTeams()) return; PositionAttackingTeam(); PositionDefendingTeam(); } public void StartSecondHalf() { Debug.Log(\$"StartSecondHalf: lineoutController={lineoutController}"); lineoutController?.ForceReset(); MatchManager.Instance?.ClearActiveLineout(); PositionTeamsForKickoff(); GiveBallToFirstAttacker(); MatchTimerManager.Instance?.StartTimer(); } </pre>
	<pre> private void PositionAttackingTeam() { PositionTeamAtCentre(attackingTeam, kickoffCentre); } private void PositionDefendingTeam() { Vector3 defensiveCentre = CalculateDefensiveCentre(); PositionTeamAtCentre(defendingTeam, defensiveCentre); } private Vector3 CalculateDefensiveCentre() { Vector3 attackDir = GetNormalisedAttackDirection(); return kickoffCentre + attackDir * defensiveLineOffset; } </pre>
	<pre> private void PositionTeamAtCentre(Team team, Vector3 centre) { List<Player> players = team.players; if (!HasPlayers(players)) return; Vector3 lateral = CalculateLateralDirection(); for (int i = 0; i < players.Count; i++) { </pre>

	<pre> PositionPlayer(players[i], centre, lateral, i, players.Count); } } private void PositionPlayer(Player player, Vector3 centre, Vector3 lateral, int index, int totalCount) { if (player == null) return; Vector3 position = CalculatePlayerPosition(centre, lateral, index, totalCount); ApplyPlayerPosition(player, position); } private Vector3 CalculatePlayerPosition(Vector3 centre, Vector3 lateral, int index, int totalCount) { float offset = CalculateLateralOffset(index, totalCount); Vector3 position = centre + lateral * offset; position.y = groundY; return position; } </pre>
	<pre> private float CalculateLateralOffset(int index, int totalCount) { float centreOffset = (totalCount - 1) * playerSpacing * 0.5f; return (index * playerSpacing) - centreOffset; } private void ApplyPlayerPosition(Player player, Vector3 position) { player.transform.position = position; ZeroPlayerVelocity(player); } private void ZeroPlayerVelocity(Player player) { if (player.rb == null) return; player.rb.linearVelocity = Vector3.zero; player.rb.angularVelocity = Vector3.zero; } </pre>
	<pre> public void GiveBallToFirstAttacker() { if (!ValidateBallExists()) return; if (!HasPlayers(attackingTeam?.players)) return; Player carrier = FindStartingBallCarrier(); AttachBallToCarrier(carrier); } </pre>

	<pre>} private Player FindStartingBallCarrier() { int midIndex = attackingTeam.players.Count / 2; return attackingTeam.players[midIndex]; } private void AttachBallToCarrier(Player carrier) { if (carrier == null) return; Ball ball = GetBall(); ball.AttachTo(carrier.transform); DebugLogBallGiven(carrier); }</pre>
	<pre>private Vector3 GetNormalisedAttackDirection() { if (attackingTeam == null) return Vector3.forward; Vector3 dir = attackingTeam.attackDirection; dir.y = 0f; if (dir.sqrMagnitude < 0.01f) dir = Vector3.forward; return dir.normalized; } private Vector3 CalculateLateralDirection() { Vector3 attackDir = GetNormalisedAttackDirection(); return Vector3.Cross(Vector3.up, attackDir).normalized; }</pre>
	<pre>private Ball GetBall() { return MatchManager.Instance?.ball; } private bool HasPlayers(List<Player> players) { return players != null && players.Count > 0; } public void SetReceivingTeam(Team scoringTeam) { MatchManager manager = MatchManager.Instance; if (manager == null) return; if (scoringTeam == manager.TeamA) { attackingTeam = manager.TeamB; defendingTeam = manager.TeamA; } else {</pre>

	<pre> attackingTeam = manager.TeamA; defendingTeam = manager.TeamB; } }</pre>
	<pre>private void OnDrawGizmos() { int count = 6; if (attackingTeam == null) return; Vector3 lateral = CalculateLateralDirection(); if (attackingTeam != null && attackingTeam.players != null) { count = attackingTeam.players.Count; } DrawTeamGizmo(Color.green, kickoffCentre, lateral, count); Vector3 defensiveCentre = CalculateDefensiveCentre(); count = 6; if (defendingTeam != null && defendingTeam.players != null) { count = defendingTeam.players.Count; } DrawTeamGizmo(Color.red, defensiveCentre, lateral, count); DrawOffsetLine(defensiveCentre); }</pre>
	<pre>private void DrawTeamGizmo(Color colour, Vector3 centre, Vector3 lateral, int count) { Gizmos.color = colour; for (int i = 0; i < count; i++) { float offset = CalculateLateralOffset(i, count); Vector3 pos = centre + lateral * offset; pos.y = groundY; Gizmos.DrawWireSphere(pos, 0.3f); } } private void DrawOffsetLine(Vector3 defensiveCentre) { }</pre>

	<pre> Gizmos.color = Color.yellow; Gizmos.DrawLine(kickoffCentre, defensiveCentre); Gizmos.DrawWireSphere(defensiveCentre, 0.4f); } </pre>
	<pre> private bool ValidateTeams() { if (attackingTeam == null) { DebugLogMissingTeam("attackingTeam"); return false; } if (defendingTeam == null) { DebugLogMissingTeam("defendingTeam"); return false; } return true; } private bool ValidateBallExists() { if (GetBall() == null) { DebugLogMissingBall(); return false; } return true; } </pre>
	<pre> private void DebugLogBallGiven(Player carrier) { Debug.Log(\$"KickoffManager: Ball given to {carrier.name}."); } private void DebugLogMissingTeam(string fieldName) { Debug.LogWarning(\$"KickoffManager: {fieldName} is not assigned."); } private void DebugLogMissingBall() { Debug.LogWarning("KickoffManager: Ball not found via MatchManager."); } </pre>
public class KickOffResetter : MonoBehaviour	
	<pre> using UnityEngine; public class KickOffResetter : MonoBehaviour </pre>

	<pre> public KickoffManager kickoffManager; public void ResetToKickoff(Team scoringTeam = null) // Means if scoringTeam is null it still works or scoringTeam can also not be null { if (kickoffManager == null) { Debug.LogWarning("KickoffResetter: KickoffManager not assigned."); return; } if (scoringTeam != null) { kickoffManager.SetReceivingTeam(scoringTeam); } ResetAllPlayerStates(); kickoffManager.PositionTeamsForKickoff(); kickoffManager.GiveBallToFirstAttacker(); ScoreManager.Instance?.ResetPhase(); FindAnyObjectByType<ConversionResultUI>()?.Hide(); } </pre>
	<pre> private void ResetAllPlayerStates() { MatchManager manager = MatchManager.Instance; if (manager == null) return; ResetTeam(manager.TeamA); ResetTeam(manager.TeamB); } private void ResetTeam(Team team) { if (team?.players == null) return; foreach (Player player in team.players) { if (player == null) continue; player.SetControlled(false); player.stateMachine.SM.SetState(new AIIdleState(player.stateMachine)); ZeroVelocity(player); } } </pre>
	<pre> private void ZeroVelocity(Player player) </pre>

	<pre> { if (player.rb == null) return; player.rb.linearVelocity = Vector3.zero; player.rb.angularVelocity = Vector3.zero; } </pre>
public class ScoreManager : MonoBehaviour	
	<pre> using UnityEngine; public class ScoreManager : MonoBehaviour public static ScoreManager Instance { get; private set; } public int TeamAScore { get; private set; } public int TeamBScore { get; private set; } public const int PointsPerTry = 5; public const int PointsPerConversion = 2; // Fired first - ConversionController listens here to run the // QTE public event Action<Team, Vector3> OnTryScoredPreConversion; // Fired after conversion resolves - ScoreScreenUI listens here // (unchanged signature) public event Action<Team, int, int> OnTryScored; public event Action<Vector3> OnBallOutOfBounds; bool tryFiredThisPhase; Team pendingScoringTeam; private void Awake() { if (Instance == null) Instance = this; else Destroy(gameObject); } </pre>
	<pre> // Called by TryLineTrigger public void RegisterTry(Team scoringTeam) { if (tryFiredThisPhase) return; if (!ValidateScoringTeam(scoringTeam)) return; tryFiredThisPhase = true; pendingScoringTeam = scoringTeam; AwardPoints(scoringTeam, PointsPerTry); DebugLogTry(scoringTeam); } </pre>

	<pre>Vector3 tryPosition = GetBallPosition(); Debug.Log("ScoreManager: About to find ConversionController"); ConversionController conversion = UnityEngine.Object.FindAnyObjectByType<ConversionController>() ; Debug.Log(\$"ScoreManager: ConversionController found={conversion != null}"); if (conversion != null) conversion.HandleTryScored(scoringTeam, tryPosition); else FinaliseScore(scoringTeam); }</pre>
	<pre>// Called by ConversionController after QTE resolves (success only) public void RegisterConversion() { if (!ValidatePendingTeam()) return; AwardPoints(pendingScoringTeam, PointsPerConversion); DebugLogConversion(); } // Called by ConversionController to show the score screen public void FinaliseScore(Team scoringTeam) { Team team; if (scoringTeam != null) { team = scoringTeam; } else { team = pendingScoringTeam; } OnTryScored?.Invoke(team, TeamAScore, TeamBScore); }</pre>
	<pre>// Called by TouchlineTrigger public void RegisterOutOfBounds(Vector3 position) { DebugLogOutOfBounds(); OnBallOutOfBounds?.Invoke(position); } // Called by KickoffResetter after reset so tries can fire again</pre>

	<pre>public void ResetPhase() { tryFiredThisPhase = false; pendingScoringTeam = null; } public void ResetScores() { TeamAScore = 0; TeamBScore = 0; }</pre>
	<pre>private void AwardPoints(Team scoringTeam, int points) { MatchManager manager = MatchManager.Instance; if (manager == null) return; if (scoringTeam == manager.TeamA) TeamAScore += points; else if (scoringTeam == manager.TeamB) TeamBScore += points; }</pre>
	<pre>private Vector3 GetBallPosition() { Ball ball = MatchManager.Instance?.ball; if (ball != null) { return ball.transform.position; } return Vector3.zero; }</pre>
	<pre>private bool ValidateScoringTeam(Team scoringTeam) { if (scoringTeam == null) { DebugLogMissingScoringTeam(); return false; } if (MatchManager.Instance == null) { DebugLogMissingMatchManager(); return false; } return true; } private bool ValidatePendingTeam() { </pre>

	<pre> if (pendingScoringTeam != null) return true; DebugLogMissingPendingTeam(); return false; } </pre>
	<pre> private void DebugLogTry(Team scoringTeam) { Debug.Log(\$"TRY! {scoringTeam.name} scores. TeamA {TeamAScore} - TeamB {TeamBScore}"); } private void DebugLogConversion() { Debug.Log(\$"Conversion scored. TeamA {TeamAScore} - TeamB {TeamBScore}"); } private void DebugLogOutOfBounds() { Debug.Log("ScoreManager: RegisterOutOfBounds fired"); } private void DebugLogMissingScoringTeam() { Debug.LogWarning("ScoreManager: scoringTeam is null."); } private void DebugLogMissingMatchManager() { Debug.LogWarning("ScoreManager: MatchManager not found."); } private void DebugLogMissingPendingTeam() { Debug.LogWarning("ScoreManager: RegisterConversion called but pendingScoringTeam is null."); } </pre>
public class TryLineTrigger : MonoBehaviour	
The trylines are triggers and are used to detect if the ball is carried over the line.	<pre> using UnityEngine; using UnityEngine.SceneManagement; public class TryLineTrigger : MonoBehaviour public Team scoringTeam; private void OnTriggerEnter(Collider other) { Ball ball = other.GetComponentInParent<Ball>(); </pre>

	<pre> if (ball == null) return; // Ball must be carried - loose ball doesn't count as a try if (ball.currentHolder == null) return; // Don't fire during a ruck if (ball.isInRuck) return; if (scoringTeam == null) { Debug.LogWarning(\$"TryLineTrigger on {gameObject.name}: scoringTeam is not assigned."); return; } ScoreManager.Instance?.RegisterTry(scoringTeam); } </pre>
public class TouchLineTrigger : MonoBehaviour	
This class uses a trigger to detect if the collider has hit the trigger. (The walls are the triggers)	<pre> using UnityEngine; using UnityEngine.SceneManagement; public class TouchLineTrigger : MonoBehaviour private void OnTriggerEnter(Collider other) { Ball ball = other.GetComponentInParent<Ball>(); if (ball == null) { Player player = other.GetComponentInParent<Player>(); if (player != null && player.hasBall) { ball = MatchManager.Instance?.ball; } } if (ball == null) return; if (ball.isInRuck) return; if (ball.isInLineout) return; Vector3 outPosition = ball.transform.position; Debug.Log(\$"Ball out of bounds at {outPosition} via {gameObject.name}"); // Hook for lineout logic later - subscribe to this from a LineoutManager bool halfTimeTriggered = MatchTimerManager.Instance?.IsInExtraTime == true; MatchTimerManager.Instance?.NotifyDeadBall(); </pre>

```

        if (!halfTimeTriggered)
            ScoreManager.Instance?.RegisterOutOfBounds(outPosition);
    }

```

Ball and passing implementation

public class Ball : MonoBehaviour

The OwnerChanged event is used by the ball whenever there is a change in possession.

This means the ball class only needs to know about ownership changes and not any other stuff like scoring etc.

This is an example of event driven OOP.

```

using UnityEngine;
using System;
using UnityEditor.Experimental.GraphView;

public class Ball : MonoBehaviour

    public Transform currentHolder { get; private set; }

    public Rigidbody rb;
    public PassHandler passHandler;
    public bool isInRuck { get; private set; }
    public bool isInLineout { get; private set; }

    GameConfig config;

    public event Action<Transform, Transform> OwnerChanged; //
    (old, new)
    public Ball(GameConfig config)
    {
        this.config = config;
    }

    private void Awake()
    {
        InitialiseComponents();
        InitialiseConfig();
        InitialisePassHandler();
    }

    private void Update()
    {
        UpdateBallPosition();
        UpdatePhysicsState();
    }

```

```

    public void SetRuckLock(bool locked)

```

	<pre>{ isInRuck = locked; } public void SetLineoutLock(bool locked) { isInLineout = locked; } private void OnCollisionEnter(Collision collision) { if (!CanBePickedUp()) return; Player player = FindPlayerInCollision(collision); if(player != null) { AttachTo(player.transform); } }</pre>
	<pre>private void InitialiseComponents() { rb = GetComponent<Rigidbody>(); passHandler = GetComponent<PassHandler>(); } private void InitialiseConfig() { config = FindConfig(); if (!ValidateConfig()) { DebugMissingConfig(); } }</pre>
	<pre>private GameConfig FindConfig() { if (MatchManager.Instance != null && MatchManager.Instance.config != null) { return MatchManager.Instance.config; } return FindAnyObjectByType<GameConfig>(); } private void InitialisePassHandler() { if (passHandler != null) { passHandler.Initialise(this, config); } }</pre>

	<pre>private void UpdateBallPosition() { if (IsBeingHeld() && !IsBeingPassed()) { PositionBallOnHolder(); } } private void PositionBallOnHolder() { if (config == null) return; transform.position = currentHolder.position + config.ballHoldOffset; } private bool IsBeingHeld() { return currentHolder != null; } private bool IsBeingPassed() { return passHandler != null && passHandler.IsPassing(); } private void UpdatePhysicsState() { if (IsBeingHeld() && !IsBeingPassed() && !rb.isKinematic) { rb.isKinematic = true; } else if (!IsBeingHeld() && !IsBeingPassed() && rb.isKinematic) { rb.isKinematic = false; } } }</pre>
	<pre>public void AttachTo(Transform holder) { if (!ValidateHolder(holder)) return; Transform playerRoot = FindPlayerRoot(holder); currentHolder = playerRoot; rb.isKinematic = true; NotifyOwnershipChange(currentHolder, playerRoot); } public void Drop() { Transform previousHolder = currentHolder; currentHolder = null; }</pre>

	<pre> NotifyOwnershipChange(previousHolder, null); } </pre>
This method attempts to find the main player object even if you are given a child component (like collider)	<pre> private Transform FindPlayerRoot(Transform holder) { Player player = holder.GetComponentInParent<Player>(); if (player != null) { return player.transform; } return holder; } </pre>
	<pre> private void NotifyOwnershipChange(Transform oldHolder, Transform newHolder) { OwnerChanged?.Invoke(oldHolder, newHolder); } private bool CanBePickedUp() { return !IsBeingHeld() && !IsBeingPassed() && !isInRuck && !isInLineout; } private Player FindPlayerInCollision(Collision collision) { return collision.collider.GetComponentInParent<Player>(); } private bool ValidateHolder(Transform holder) { return holder != null; } private bool ValidateConfig() { return config != null; } </pre>
	<pre> private void DebugMissingConfig() { Debug.LogWarning("Ball is missing GameConfig; assign one in the scene."); } </pre>
// Handles pass trajectory and animation	
public class PassHandler : MonoBehaviour	
	<pre> using UnityEngine; </pre>

	<pre>public class PassHandler : MonoBehaviour Ball ball; GameConfig config; PassProbabilityAlgorithm probabilityAlgorithm; bool isPassing; Transform passTargetPlayer; bool passToGround; Vector3 startPos; float progress; float duration; bool passWasSuccessful; Vector3 actualTargetPos;</pre>
	<pre>public void Initialise(Ball ball, GameConfig config) { this.ball = ball; this.config = config; if (ValidateInitialisation()) { probabilityAlgorithm = config.CreateProbabilityAlgorithm(); } } private bool ValidateInitialisation() { if (ball == null) { DebugMissingBall(); return false; } if (config == null) { DebugMissingConfig(); return false; } return true; }</pre>
	<pre>private void Update() { if (isPassing) { UpdatePassAnimation(); } }</pre>

	<pre> } } public bool IsPassing() { return isPassing; } </pre>
This code is made for the conversion as I wanted a 100% accurate kick as they have already done the QTE event to determine whether it was successful or a failure.	<pre> public void ForcePassToTarget(Transform target) { if (target == null) return; ball.Drop(); // Guaranteed Pass (for kick to goal) passTargetPlayer = null; passToGround = true; actualTargetPos = target.position; actualTargetPos.y = Mathf.Max(0, actualTargetPos.y); passWasSuccessful = true; BeginPassAnimation(actualTargetPos); } </pre>
If no value is provided in this method initialisation, it defaults the passerHandlingStat to 70 and the difficultyModifier to 1. The f represents a float.	<pre> public void RequestPassToPlayer(Transform target, float passerHandlingStat = 70f, float difficultyModifier = 1.0f) { if (!ValidatePassToPlayer(target)) return; ball.Drop(); ConfigurePlayerPass(target); CalculatePassOutcome(target.position, passerHandlingStat, difficultyModifier); BeginPassAnimation(actualTargetPos); } </pre>
	<pre> public void RequestPassToGround(Vector3 targetPos, float passerHandlingStat = 70f, float difficultyModifier = 1.0f) { ball.Drop(); ConfigureGroundPass(targetPos); CalculatePassOutcome(actualTargetPos, passerHandlingStat, difficultyModifier); } </pre>

	<pre> BeginPassAnimation(actualTargetPos); } </pre>
	<pre> private void ConfigurePlayerPass(Transform target) { passTargetPlayer = target; passToGround = false; } private void ConfigureGroundPass(Vector3 targetPos) { passTargetPlayer = null; passToGround = true; //Ensure ground position is valid targetPos.y = Mathf.Max(0, targetPos.y); actualTargetPos = targetPos; } </pre>
	<pre> private void CalculatePassOutcome(Vector3 intendedTarget, float handlingStat, float difficultyModifier) { if (!ValidatePassProbabilitySystem()) { ResolveAsSuccessfulPass(intendedTarget); return; } float distance = CalculatePassDistance(intendedTarget); bool success = RollForPassSuccess(distance, handlingStat, difficultyModifier, out float probability, out PassZone zone); // Outputs multiple results so the calculation only needs to be performed once (output parameters) if (success) { ResolveAsSuccessfulPass(intendedTarget); DebugPassSuccess(distance, probability, zone); } else { ResolveAsFailedPass(intendedTarget, distance, probability); DebugPassFailure(distance, probability, zone); } passWasSuccessful = success; } </pre>

	<pre>private float CalculatePassDistance(Vector3 intendedTarget) { return Vector3.Distance(ball.transform.position, intendedTarget); } private bool RollForPassSuccess(float distance, float handlingStat, float difficultyModifier, out float probability,out PassZone zone) { var result = probabilityAlgorithm.CalculatePassProbability(distance, handlingStat, difficultyModifier); probability = result.probability; zone = result.zone; float roll = Random.Range(0, 1f); return roll <= probability; }</pre>
	<pre>private void ResolveAsSuccessfulPass(Vector3 intendedTarget) { actualTargetPos = intendedTarget; } private void ResolveAsFailedPass(Vector3 intendedTarget, float distance, float probability) { actualTargetPos = CalculateMissPosition(intendedTarget, distance, probability); }</pre>
	<pre>private Vector3 CalculateMissPosition(Vector3 intendedTarget, float distance, float probability) { if (config == null) return intendedTarget; Vector3 passDirection = CalculatePassDirection(intendedTarget); Vector3 perpendicular = CalculatePerpendicular(passDirection); float missRadius = CalculateMissRadius(probability, distance); Vector3 missOffset = CalculateMissOffset(passDirection, perpendicular, missRadius); return ApplyMissOffset(intendedTarget, missOffset); }</pre>

	<pre>private Vector3 CalculatePassDirection(Vector3 intendedTarget) { return (intendedTarget - ball.transform.position).normalized; } private Vector3 CalculatePerpendicular(Vector3 passDirection) { return Vector3.Cross(passDirection, Vector3.up).normalized; } private float CalculateMissRadius(float probability, float distance) { return probabilityAlgorithm.CalculateMissRadius(probability, distance); }</pre>
	<pre>private Vector3 CalculateMissOffset(Vector3 passDirection, Vector3 perpendicular, float missRadius) { float perpendicularError = Random.Range(-missRadius, missRadius) * config.passMissPerpendicularFactor; float longitudinalError = Random.Range(-missRadius, missRadius) * config.passMissLongitudinalFactor; return (perpendicular * perpendicularError) + (passDirection * longitudinalError); } private Vector3 ApplyMissOffset(Vector3 intendedTarget, Vector3 missOffset) { Vector3 missTarget = intendedTarget + missOffset; missTarget.y = Mathf.Max(0, missTarget.y); return missTarget; }</pre>
	<pre>private void BeginPassAnimation(Vector3 targetPos) { if (!ValidatePassAnimation()) return; InitialiseAnimationState(targetPos); EnablePassAnimation(); PreparePhysicsForAnimation(); } private void InitialiseAnimationState(Vector3 targetPos) { </pre>

	<pre>startPos = ball.transform.position; progress = 0; duration = CalculatePassDuration(targetPos); } private float CalculatePassDuration(Vector3 targetPos) { float distance = Vector3.Distance(startPos, targetPos); return Mathf.Max(distance / config.passSpeed, 0.01f); }</pre>
	<pre>private void EnablePassAnimation() { isPassing = true; } private void PreparePhysicsForAnimation() { ball.rb.isKinematic = true; } private void UpdatePassAnimation() { Vector3 currentTarget = CalculateCurrentTarget(); float normalisedProgress = AdvanceAnimationProgress(); //Progress within 0-1 UpdateBallPosition(currentTarget, normalisedProgress); if (IsAnimationComplete(normalisedProgress)) { CompletePassAnimation(); } }</pre>
	<pre>private Vector3 CalculateCurrentTarget() { if (ShouldTrackPlayerTarget()) { return passTargetPlayer.position + config.ballHoldOffset; } else { Vector3 target = actualTargetPos; target.y = Mathf.Max(0, target.y); return target; } }</pre>

	<pre> private bool ShouldTrackPlayerTarget() { return !passToGround && passWasSuccessful && passTargetPlayer != null; } private float AdvanceAnimationProgress() { progress += Time.deltaTime; return Mathf.Clamp01(progress/duration); } </pre>
	<pre> private void UpdateBallPosition(Vector3 targetPos, float normalisedProgress) { Vector3 linearPosition = CalculateLinearPosition(targetPos, normalisedProgress); float arcHeight = CalculateArcHeight(normalisedProgress); ball.transform.position = linearPosition+ Vector3.up * arcHeight; } private Vector3 CalculateLinearPosition(Vector3 targetPos, float normalisedProgress) { return Vector3.Lerp(startPos,targetPos, normalisedProgress); } private float CalculateArcHeight(float normalisedProgress) { return Mathf.Sin(normalisedProgress * Mathf.PI) * config.passArcHeight; } private bool IsAnimationComplete(float normalisedProgress) { return normalisedProgress >= 1f; //true if >= 1 } </pre>
	<pre> private void CompletePassAnimation() { DisablePassAnimation(); if (ShouldAttachToPlayer()) { ball.AttachTo(passTargetPlayer); } else </pre>

	<pre> { DropBall(); } } private void DisablePassAnimation() { isPassing = false; } private bool ShouldAttachToPlayer() { return !passToGround && passWasSuccessful && passTargetPlayer != null; } private void DropBall() { ball.transform.position = actualTargetPos; ball.rb.isKinematic = false; }</pre>
	<pre>private bool ValidatePassToPlayer(Transform target) { if (target == null) { DebugNullPassTarget(); return false; } return true; } private bool ValidatePassProbabilitySystem() { if (probabilityAlgorithm == null config == null) { DebugMissingProbabilitySystem(); return false; } return true; } private bool ValidatePassAnimation() { if (ball == null config == null) { DebugCannotStartPass(); isPassing = false; return false; } return true; }</pre>
	<pre>private void DebugMissingBall()</pre>

```

{
    Debug.LogWarning("PassHandler is missing its Ball
reference.");
}

private void DebugMissingConfig()
{
    Debug.LogWarning("PassHandler is missing GameConfig;
assign one in the scene.");
}

private void DebugNullPassTarget()
{
    Debug.LogWarning("PassHandler received null target for
player pass.");
}

private void DebugMissingProbabilitySystem()
{
    Debug.LogWarning("PassHandler missing probability
algorithm, defaulting to success.");
}

private void DebugCannotStartPass()
{
    Debug.LogWarning("PassHandler cannot start a pass without
Ball and GameConfig references.");
}

private void DebugPassSuccess(float distance, float
probability, PassZone zone)
{
    Debug.Log($"Pass success - Distance: {distance}m, Prob:
{probability}%, Zone: {zone}");
}

private void DebugPassFailure(float distance, float
probability, PassZone zone)
{
    Debug.Log($"Pass failed - Distance: {distance}m, Prob:
{probability}%, Zone: {zone}");
}

```

Input System

```
// Handles human input for a team (sends input to the controlled player)
// Listens for ball ownership changes to auto-switch the controls
```

```
public class TeamInputHandler : MonoBehaviour
```

```
using System;
using UnityEngine;
using UnityEngine.InputSystem;

// Handles human input for a team (sends input to the
// controlled player)
// Listens for ball ownership changes to auto-switch the
// controls
public class TeamInputHandler : MonoBehaviour

public Team team;
public Team opposingTeam;
public PassIndicatorUI passIndicatorUI;
public int startPlayerIndex = 0;
public bool isTeamA = true;

int currentPlayerIndex;
bool initialPlayerSet = false;

GameConfig config;
PassProbabilityAlgorithm probabilityAlgorithm;

private void Awake()
{
    Debug.Log($"TeamInputHandler Awake on
{gameObject.name}");
    InitialiseConfig();
}
private void OnEnable()
{
}
private void OnDisable()
{
    UnsubscribeFromBallEvents();
}
private void Start()
{
    //Debug.Log($"TeamInputHandler Start on
{gameObject.name}, team={team?.name},
playerCount={team?.players?.Count}");
    SetInitialControlledPlayer();
    SubscribeToBallEvents();
}
}
```

	<pre> private void Update() { if (!ValidateTeamHasPlayers()) return; Player currentPlayer = GetCurrentPlayer(); if (!ValidateCurrentPlayer(currentPlayer)) return; //Debug.Log(\$"{team.name} Update: isControlled={currentPlayer.isControlled}, state={currentPlayer.stateMachine.SM.CurrentState?.GetType().Name}"); ProcessMovementInput(currentPlayer); ProcessPlayerSwitchingInput(); ProcessPassInput(currentPlayer); ProcessTackleInput(currentPlayer); ProcessDefenderSwitchInput(); UpdatePassIndicatorUI(currentPlayer); } </pre>
	<pre> private void InitialiseConfig() { config = FindConfig(); if (!ValidateConfig()) { DebugMissingConfig(); return; } probabilityAlgorithm = config.CreateProbabilityAlgorithm(); } private GameConfig FindConfig() { if (MatchManager.Instance != null) { return MatchManager.Instance.config; } return FindAnyObjectByType<GameConfig>(); } </pre>
	<pre> private void SetInitialControlledPlayer() { if (!ValidateTeamHasPlayers()) return; int index = ClampPlayerIndex(startPlayerIndex); //Debug.Log(\$"{team.name} TeamInputHandler: SetInitialControlledPlayer index={index}"); SetControlledPlayer(index); initialPlayerSet = true; } </pre>

	<pre> } private void ProcessMovementInput(Player player) { Vector2 moveInput = ReadMoveInput(); bool sprintInput = ReadSprintInput(); SendInputToPlayer(player, moveInput, sprintInput); } </pre>
	<pre> private Vector2 ReadMoveInput() { TeamKeybinds binds = KeybindManager.GetTeamBinds(isTeamA); float x = 0f; float y = 0f; if (KeybindManager.IsKeyHeld(binds.moveLeft)) x = -1f; if (KeybindManager.IsKeyHeld(binds.moveRight)) x = 1f; if (KeybindManager.IsKeyHeld(binds.moveUp)) y = 1f; if (KeybindManager.IsKeyHeld(binds.moveDown)) y = -1f; return new Vector2(x, y); } private bool ReadSprintInput() { TeamKeybinds binds = KeybindManager.GetTeamBinds(isTeamA); return KeybindManager.IsKeyHeld(binds.sprint); } </pre>
	<pre> private void SendInputToPlayer(Player player, Vector2 move, bool sprint) { player.stateMachine.SetInput(move, sprint); } private void ProcessPlayerSwitchingInput() { // Player switching uses fixed keys - not remappable if (isTeamA) { if (Input.GetKeyDown(KeyCode.Tab)) SwitchToNextPlayer(); return; } if (Input.GetKeyDown(KeyCode.Alpha1)) SwitchToNextPlayer(); if (Input.GetKeyDown(KeyCode.Alpha2)) SwitchToPreviousPlayer(); } </pre>
This uses a	<pre> private void SwitchToNextPlayer() </pre>

circular list, where when it gets to the last player, it goes back to the first player in the list. This is done using the mod arithmetic (%)	<pre> { int nextIndex = (currentPlayerIndex + 1) % team.players.Count; SwitchToPlayerAtIndex(nextIndex); } private void SwitchToPreviousPlayer() { int prevIndex = (currentPlayerIndex - 1 + team.players.Count) % team.players.Count; SwitchToPlayerAtIndex(prevIndex); } private void SwitchToPlayerAtIndex(int index) { if (index == currentPlayerIndex) return; SetControlledPlayer(index); } </pre>
	<pre> private void SetControlledPlayer(int index) { DeactivateCurrentPlayer(); currentPlayerIndex = index; ActivateNewPlayer(); UpdateUIFollowTarget(); } private void DeactivateCurrentPlayer() { if (!IsValidPlayerIndex(currentPlayerIndex)) return; team.players[currentPlayerIndex].SetControlled(false); } private void ActivateNewPlayer() { Player player = team.players[currentPlayerIndex]; player.SetControlled(true); SetPlayerToCorrectHumanState(player); } </pre>
	<pre> private void ProcessDefenderSwitchInput() { TeamKeybinds binds = KeybindManager.GetTeamBinds(isTeamA); if (!KeybindManager.IsKeyDown(binds.switchDefender)) return; if (TeamIsAttacking()) return; SwitchToClosestDefender(); } private void SwitchToClosestDefender() { Ball ball = GetBall(); if (ball == null) return; </pre>

	<pre> Player best = FindClosestDefenderToBall(ball); if (best == null) return; HandOffCurrentPlayerToAI(); SwitchControlToDefender(best); } </pre>
	<pre> private Player FindClosestDefenderToBall(Ball ball) { Player best = null; float bestDist = float.MaxValue; foreach (Player p in team.players) { if (p == null) continue; if (p.isControlled) continue; if (!p.stateMachine?.SM?.CurrentState is DefensiveLineState)) continue; float dist = Vector3.Distance(p.transform.position, ball.transform.position); if (dist < bestDist) { bestDist = dist; best = p; } } return best; } </pre>
	<pre> private void HandOffCurrentPlayerToAI() { Player current = GetCurrentPlayer(); if (current == null) return; current.SetControlled(false); current.stateMachine.SM.SetState(new HumanDefendState(current.stateMachine, opposingTeam)); } private void SwitchControlToDefender(Player defender) { currentPlayerIndex = team.players.IndexOf(defender); defender.SetControlled(true); defender.stateMachine.SM.SetState(new HumanDefendState(defender.stateMachine, opposingTeam)); UpdateUIFollowTarget(); } </pre>

	<pre>private void SetPlayerToCorrectHumanState(Player player) { bool attacking = TeamIsAttacking(); //Debug.Log(\$"{team.name} SetPlayerToCorrectHumanState: player={player.name}, attacking={attacking}"); if (attacking) { player.stateMachine.SM.SetState(new IdleState(player.stateMachine)); } else { player.stateMachine.SM.SetState(new HumanDefendState(player.stateMachine, opposingTeam)); } }</pre>
	<pre>private void ProcessPassInput(Player player) { TeamKeybinds binds = KeybindManager.GetTeamBind(isTeamA); if (KeybindManager.IsKeyDown(binds.passLeft)) player.stateMachine.RequestPass(leftSide: true); if (KeybindManager.IsKeyDown(binds.passRight)) player.stateMachine.RequestPass(leftSide: false); } private void ProcessTackleInput(Player player) { if (!IsPlayerDefending(player)) return; TeamKeybinds binds = KeybindManager.GetTeamBind(isTeamA); if (KeybindManager.IsKeyDown(binds.tackle)) player.stateMachine.RequestTackle(); }</pre>
	<pre>private bool IsPlayerDefending(Player player) { return player.stateMachine.SM.CurrentState is HumanDefendState; } private void SubscribeToBallEvents() { Ball ball = GetBall(); if (ball != null) { ball.OwnerChanged -= OnBallOwnerChanged; ball.OwnerChanged += OnBallOwnerChanged; } } private void UnsubscribeFromBallEvents()</pre>

	<pre> { Ball ball = GetBall(); if (ball != null) { ball.OwnerChanged -= OnBallOwnerChanged; } } private void OnBallOwnerChanged(Transform oldHolder, Transform newHolder) { if (!initialPlayerSet) return; //Debug.Log(\$"{team.name} TeamInputHandler: newHolder={newHolder?.name}, isOnThisTeam={NewHolderIsOnThisTeam(newHolder)}"); if (NewHolderIsOnThisTeam(newHolder)) { SwitchControlToNewHolder(newHolder); } else { TransitionCurrentPlayerToCorrectState(); } } </pre>
	<pre> private bool NewHolderIsOnThisTeam(Transform newHolder) { return newHolder != null && FindPlayerIndex(newHolder) >= 0; } private void SwitchControlToNewHolder(Transform newHolder) { int index = FindPlayerIndex(newHolder); if (index < 0) { //Debug.LogWarning(\$"{team.name} could not find player index for {newHolder?.name}"); return; } //Debug.Log(\$"{team.name} switching control to index {index}, player={newHolder?.name}"); SetControlledPlayer(index); } private void TransitionCurrentPlayerToCorrectState() { Player currentPlayer = GetCurrentPlayer(); if (!ValidateCurrentPlayer(currentPlayer)) return; if (!currentPlayer.isControlled) return; SetPlayerToCorrectHumanState(currentPlayer); } </pre>

	<pre> }</pre>
	<pre> private void UpdatePassIndicatorUI(Player player) { if (passIndicatorUI == null) return; if (!ValidateProbabilityAlgorithm()) return; Ball ball = GetBall(); if (!PlayerIsHoldingBall(player, ball)) { passIndicatorUI.HideAll(); return; } UpdateLeftPassIndicator(player, ball); UpdateRightPassIndicator(player, ball); } private void UpdateLeftPassIndicator(Player player, Ball ball) { Player target = team.FindBestPassTarget(player, leftSide: true); if (target != null) { ShowLeftIndicatorForTarget(player, ball, target); } else { passIndicatorUI.HideLeft(); } } private void UpdateRightPassIndicator(Player player, Ball ball) { Player target = team.FindBestPassTarget(player, leftSide: false); if (target != null) { ShowRightIndicatorForTarget(player, ball, target); } else { passIndicatorUI.HideRight(); } } }</pre>
	<pre> private void ShowLeftIndicatorForTarget(Player player, Ball ball, Player target)</pre>

	<pre> { var (prob, zone) = CalculatePassResult(ball, player, target); passIndicatorUI.ShowLeft(zone, prob); } private void ShowRightIndicatorForTarget(Player player, Ball ball, Player target) { var (prob, zone) = CalculatePassResult(ball, player, target); passIndicatorUI.ShowRight(zone, prob); } private (float probability, PassZone zone) CalculatePassResult(Ball ball, Player passer, Player target) { float dist = CalculatePassDistance(ball, target); return probabilityAlgorithm.CalculatePassProbability(dist, passer.handlingStat); } private float CalculatePassDistance(Ball ball, Player target) { return Vector3.Distance(ball.transform.position, target.transform.position); } </pre>
	<pre> private bool TeamIsAttacking() { Ball ball = GetBall(); if (ball == null ball.currentHolder == null) return false; return TeamOwnsBall(ball); } private bool TeamOwnsBall(Ball ball) { foreach (Player p in team.players) { if (PlayerHoldsBall(p, ball)) return true; } return false; } private bool PlayerHoldsBall(Player player, Ball ball) { return player != null && player.transform == ball.currentHolder; } private bool PlayerIsHoldingBall(Player player, Ball ball) { return ball != null && ball.currentHolder == player.transform; } </pre>

	<pre> private int FindPlayerIndex(Transform t) { for (int i = 0; i < team.players.Count; i++) { if (PlayerTransformMatchesIndex(t, i)) return i; } return -1; } private bool PlayerTransformMatchesIndex(Transform t, int index) { return team.players[index] != null && team.players[index].transform == t; } private void UpdateUIFollowTarget() { if (passIndicatorUI == null) return; passIndicatorUI.SetFollowTarget(team.players[currentPlayerIndex].transform); } private Player GetCurrentPlayer() { if (!IsValidPlayerIndex(currentPlayerIndex)) return null; return team.players[currentPlayerIndex]; } private Ball GetBall() { return MatchManager.Instance?.ball; } </pre>
	<pre> private bool IsValidPlayerIndex(int index) { return index >= 0 && index < team.players.Count && team.players[index] != null; } private int ClampPlayerIndex(int index) { return Mathf.Clamp(index, 0, team.players.Count - 1); } private bool ValidateTeamHasPlayers() { return team != null && team.players != null && team.players.Count > 0; } private bool ValidateCurrentPlayer(Player player) { return player != null; } private bool ValidateConfig() </pre>

	<pre> { return config != null; } private bool ValidateProbabilityAlgorithm() { return probabilityAlgorithm != null; } </pre>
	<pre> private void DebugMissingConfig() { Debug.LogWarning("TeamInputHandler: Missing GameConfig reference."); } </pre>
public class KeybindManager	
The keybind data is automatically loaded when the manager is first used with static KeybindManager() The use of static is important as it means only a single shared set of keybinds exist across the game. By using static it means that all systems access the same input mapping.	<pre> using UnityEngine; using System.IO; public class KeybindManager public static KeybindData Current { get; private set; } const string FileName = "keybinds.json"; static KeybindManager() { LoadOrCreateDefaults(); } </pre>
This is how the keybinds are saved to the JSON file, they are turned into	<pre> public static void Save() { string json = JsonUtility.ToJson(Current, prettyPrint: true); File.WriteAllText(GetFilePath(), json); DebugLogSaved(); } </pre>

JSON and added to the JSON file. This means it is persistent across games and saves even if you reload the game.	<pre>} public static void ResetToDefaults() { Current = KeybindData.CreateDefaults(); Save(); DebugLogReset(); }</pre>
GetTeamKeybinds is a simple interface for the rest of the code to get one of the team's bindings without needing to know how the data is stored. For IsKeyHeld, the keybinds are stored as JSON and then converted into Unity KeyCode values using TryParseKey(). This allows the input system to work with saved bindings as Unity detects keys in KeyCode not JSON.	<pre>public static bool IsKeyDown(string keyName) { if (!TryParseKey(keyName, out KeyCode key)) return false; //Tries to convert keyname into keycode (input space output keycode.space) return Input.GetKeyDown(key); } public static bool IsKeyHeld(string keyName) { if (!TryParseKey(keyName, out KeyCode key)) return false; return Input.GetKey(key); } public static TeamKeybinds GetTeamBinds(bool isTeamA) { if (isTeamA) { return Current.teamA; } else { return Current.teamB; } }</pre>
This method changes one action binding, updates the correct TeamKeybinds object and saves the JSON	<pre>public static void SetBinding(bool isTeamA, string actionName, string newKey) { TeamKeybinds binds = GetTeamBinds(isTeamA); SetActionKey(binds, actionName, newKey); Save(); DebugLogBindingChanged(isTeamA, actionName, newKey); }</pre>

file.	
The bindings are stored in serialised data classes (keybindData and TeamKeyBinds) so they can be written and restored from JSON directly.	<pre> private static void LoadOrCreateDefaults() { string path = GetFilePath(); if (File.Exists(path)) { Current = JsonUtility.FromJson<KeybindData>(File.ReadAllText(path)); DebugLogLoaded(path); return; } Current = KeybindData.CreateDefaults(); Save(); DebugLogCreatedDefaults(path); } </pre>
	<pre> private static void SetActionKey(TeamKeybinds binds, string actionName, string newKey) { switch (actionName) { case "moveUp": binds.moveUp = newKey; break; case "moveDown": binds.moveDown = newKey; break; case "moveLeft": binds.moveLeft = newKey; break; case "moveRight": binds.moveRight = newKey; break; case "sprint": binds.sprint = newKey; break; case "passLeft": binds.passLeft = newKey; break; case "passRight": binds.passRight = newKey; break; case "tackle": binds.tackle = newKey; break; case "switchDefender": binds.switchDefender = newKey; break; default: DebugLogUnknownAction(actionName); break; } } </pre>
	<pre> private static bool TryParseKey(string keyName, out KeyCode key) { </pre>

	<pre> if (System.Enum.TryParse(keyName, out key)) return true; //Tryparse is a premade enum DebugLogInvalidKey(keyName); return false; } private static string GetFilePath() { return Path.Combine(Application.persistentDataPath, FileName); }</pre>
	<pre>private static void DebugLogSaved() { Debug.Log(\$"KeybindManager: Saved to {GetFilePath()}"); } private static void DebugLogLoaded(string path) { Debug.Log(\$"KeybindManager: Loaded from {path}"); } private static void DebugLogCreatedDefaults(string path) { Debug.Log(\$"KeybindManager: Created defaults at {path}"); } private static void DebugLogReset() { Debug.Log("KeybindManager: Reset to defaults."); } private static void DebugLogBindingChanged(bool isTeamA, string action, string key) { string team; if (isTeamA) { team = "Team A"; } else { team = "Team B"; } Debug.Log(\$"KeybindManager: {team} {action} rebound to {key}"); } private static void DebugLogInvalidKey(string keyName)</pre>

	<pre> { Debug.LogWarning(\$"KeybindManager: '{keyName}' is not a valid KeyCode."); } private static void DebugLogUnknownAction(string actionName) { Debug.LogWarning(\$"KeybindManager: Unknown action '{actionName}'"); } </pre>
[System.Serializable] public class TeamKeybinds	
	<pre> using UnityEngine; [System.Serializable] public class TeamKeybinds public string moveUp = "W"; public string moveDown = "S"; public string moveLeft = "A"; public string moveRight = "D"; public string sprint = "LeftShift"; public string passLeft = "Q"; public string passRight = "E"; public string tackle = "Space"; public string switchDefender = "Tab"; </pre>
[System.Serializable] public class KeybindData	
	<pre> [System.Serializable] public class KeybindData public TeamKeybinds teamA = new TeamKeybinds(); public TeamKeybinds teamB = new TeamKeybinds(); </pre>
This is used for the first time it is initialised or if the code is reset back to default. (Alpha 1) is just the 1 key on the keyboard.	<pre> public static KeybindData CreateDefaults() { KeybindData data = new KeybindData(); // Team A defaults data.teamA.moveUp = "W"; data.teamA.moveDown = "S"; data.teamA.moveLeft = "A"; data.teamA.moveRight = "D"; data.teamA.sprint = "LeftShift"; data.teamA.passLeft = "Q"; data.teamA.passRight = "E"; } </pre>

	<pre> data.teamA.tackle = "Space"; data.teamA.switchDefender = "Tab"; // Team B defaults data.teamB.moveUp = "UpArrow"; data.teamB.moveDown = "DownArrow"; data.teamB.moveLeft = "LeftArrow"; data.teamB.moveRight = "RightArrow"; data.teamB.sprint = "RightShift"; data.teamB.passLeft = "Comma"; data.teamB.passRight = "Period"; data.teamB.tackle = "RightControl"; data.teamB.switchDefender = "Aplhal"; return data; } </pre>
--	--

Ruck Subsystems	
public class RuckManager	
	<pre> using System; using System.Collections.Generic; using UnityEngine; public class RuckManager public enum RuckPhase { Forming, // Players go to ruck Won, // Scrumhalf runs to ruck to play ball PlayingBall, //Human takes control Unloading // Stack releases players } </pre>
	<pre> public RuckPhase Phase { get; private set; } public Vector3 RuckPosition { get; private set; } public Player GetScrumhalf() { return scrumhalf; } Team attackingTeam; Team defendingTeam; Ball ball; </pre>

	<pre> RuckConfig config; RuckStack attackingStack; RuckStack defendingStack; Player carrier; Player tackler; Player scrumhalf; int attackingSupportExpected; int defendingSupportExpected; int attackingSupportArrived; int defendingSupportArrived; public event Action OnRuckComplete; </pre>
The attacking ruck and defending ruck stacks are created in this constructor.	<pre> public RuckManager(Player carrier, Player tackler, Team attackingTeam, Team defendingTeam, Ball ball, RuckConfig config) { this.carrier = carrier; this.tackler = tackler; this.attackingTeam = attackingTeam; this.defendingTeam = defendingTeam; this.ball = ball; this.config = config; RuckPosition = carrier.transform.position; attackingStack = new RuckStack(RuckPosition, -GetAttackDirection(), config.stackOffset); defendingStack = new RuckStack(RuckPosition, GetAttackDirection(), config.stackOffset); } </pre>
This is the entry point to the ruck subsystem. All the setup logic is called in one place.	<pre> public void StartRuck() { Phase = RuckPhase.Forming; carrier.SetControlled(false); ball.SetRuckLock(true); DropBallAtRuckPosition(); BindCarrierToRuck(); BindTacklerToRuck(); SendSupportPlayers(); AssignScrumhalf(); } </pre>
Support arrival	<pre> public void Tick(float deltaTime) </pre>

is tracked separately for each team, when they reach the ruck, they are pushed into their teams stack.	<pre>{ if (Phase == RuckPhase.Unloading) { TickUnloading(deltaTime); } } public void NotifySupportArrived(Player player, Team team) { if (IsAttackingTeam(team)) { attackingStack.Push(player); attackingSupportArrived++; } else { defendingStack.Push(player); defendingSupportArrived++; } if (HasAllSupportArrived()) { WinRuck(); } } public void NotifyScrumhalfArrived() { Phase = RuckPhase.PlayingBall; GiveBallToScrumhalf(); SwitchHumanControlToScrumhalf(); } public void NotifyPassMade() { BeginUnloading(); }</pre>
	<pre>private void DropBallAtRuckPosition() { Debug.LogAssertion("Dropping ball"); ball.Drop(); ball.transform.position = RuckPosition; } private void BindCarrierToRuck() { attackingStack.Push(carrier); EnterRuckBoundState(carrier); carrier.SetControlled(false); }</pre>

	<pre> private void BindTacklerToRuck() { defendingStack.Push(tackler); EnterRuckBoundState(tackler); } </pre>
	<pre> private void SendSupportPlayers() { List<Player> attackingSupport = FindSupportPlayers(attackingTeam, config.supportPlayersPerTeam, excludeScrumhalf: true); List<Player> defendingSupport = FindSupportPlayers(defendingTeam, config.supportPlayersPerTeam, excludeScrumhalf: false); attackingSupportExpected = attackingSupport.Count; defendingSupportExpected = defendingSupport.Count; SendTeamToRuck(attackingSupport, attackingTeam); SendTeamToRuck(defendingSupport, defendingTeam); } </pre>
	<pre> private void AssignScrumhalf() { scrumhalf = FindScrumhalf(); if (scrumhalf == null) return; Vector3 ScrumhalfPosition = CalculateScrumhalfPosition(); scrumhalf.stateMachine.SM.SetState(new RuckJoinState(scrumhalf.stateMachine, this, ScrumhalfPosition, isScrumhalf: true)); } </pre>
	<pre> private void WinRuck() { Phase = RuckPhase.Won; if (scrumhalf == null) { BeginUnloading(); } } </pre>
	<pre> private void GiveBallToScrumhalf() { ball.AttachTo(scrumhalf.transform); } private void SwitchHumanControlToScrumhalf() { scrumhalf.SetControlled(true); } </pre>

	<pre> scrumhalf.stateMachine.SM.SetState(new RuckPlayState(scrumhalf.stateMachine, this)); } </pre>
	<pre> private void BeginUnloading() { Phase = RuckPhase.Unloading; attackingStack.BeginUnloading(config.unloadInterval); defendingStack.BeginUnloading(config.unloadInterval); } private void TickUnloading(float deltaTime) { attackingStack.Tick(deltaTime); defendingStack.Tick(deltaTime); if (IsUnloadingComplete()) { CompleteRuck(); } } </pre>
The ruck manager completes using an event instead of changing the tackle state directly. TackleState subscribes at the start of the ruck and unsubscribes at the end so that it is not just subscribed for no reason.	<pre> private bool IsUnloadingComplete() { return attackingStack.IsEmpty() && defendingStack.IsEmpty(); } private void CompleteRuck() { Phase = RuckPhase.Forming; // reset ball.SetRuckLock(false); OnRuckComplete?.Invoke(); } private void EnterRuckBoundState(Player player) { player.stateMachine.SM.SetState(new RuckBoundState(player.stateMachine)); } </pre>
FindSupportPlayers selects players using a filtered nearest player algorithm.	<pre> private void SendTeamToRuck(List<Player> players, Team team) { foreach (Player player in players) { Vector3 stackPosition = CalculateStackPosition(team, players.IndexOf(player)); player.stateMachine.SM.SetState(new RuckJoinState(player.stateMachine, this, stackPosition, </pre>

First, a list of all possible candidates is made then all the unavailable players are removed. The remaining are then sorted by nearest to the ruck. It then takes only 2 from each team.	<pre> isScrumhalf: false)); } } private List<Player> FindSupportPlayers(Team team, int count, bool excludeScrumhalf) { List<Player> candidates = BuildCandidateList(team, excludeScrumhalf); SortByDistanceToRuck(candidates); return TakeFirst(candidates, count); } </pre>
	<pre> private List<Player> BuildCandidateList(Team team, bool excludeScrumhalf) { List<Player> candidates = new List<Player>(); foreach (Player player in team.players) { if (!IsValidSupportCandidate(player, excludeScrumhalf)) continue; candidates.Add(player); } return candidates; } private bool IsValidSupportCandidate(Player player, bool excludeScrumhalf) { if (player == null) return false; if (player == carrier) return false; if (player == tackler) return false; if (excludeScrumhalf && player == scrumhalf) return false; return true; } </pre>
	<pre> private void SortByDistanceToRuck(List<Player> players) { players.Sort(CompareDistanceToRuck); } private int CompareDistanceToRuck(Player a, Player b) { float distA = DistanceToRuck(a); float distB = DistanceToRuck(b); return distA.CompareTo(distB); } </pre>

	<pre> } private float DistanceToRuck(Player player) { return Vector3.Distance(player.transform.position, RuckPosition); } </pre>
	<pre> private List<Player> TakeFirst(List<Player> players, int count) { List<Player> result = new List<Player>(); for (int i = 0; i < Mathf.Min(count, players.Count); i++) { result.Add(players[i]); } return result; } </pre>
The scrumhalf is assigned with the same sort candidate list. However it takes the next closest player that is not in the support group. This means I can reuse my code instead of having to rewrite it.	<pre> private Player FindScrumhalf() { List<Player> candidates = BuildCandidateList(attackingTeam, excludeScrumhalf: false); SortByDistanceToRuck(candidates); // Scrumhalf is closest attacker not joining the ruck as support int supportCount = config.supportPlayersPerTeam; if (candidates.Count > supportCount) { return candidates[supportCount]; } if (candidates.Count == 0) { return null; } return candidates[candidates.Count - 1]; } </pre>
	<pre> private Vector3 CalculateStackPosition(Team team, int slotIndex) { Vector3 stackDirection; if (IsAttackingTeam(team)) { stackDirection = -GetAttackDirection(); } else </pre>

	<pre> { stackDirection = GetAttackDirection(); } float depth = config.stackSpacing * (slotIndex + 1); float lateralOffset = CalculateLateralStackOffset(slotIndex); Vector3 position = RuckPosition + stackDirection * depth; position += GetLateralDirection() * lateralOffset; return position; } </pre>
	<pre> private float CalculateLateralStackOffset(int slotIndex) { // Alternate left/right so players are slightly offset and // visible float side; if (slotIndex % 2 == 0) { side = 1f; } else { side = -1f; } return side * config.stackOffset; } </pre>
The bool <code>HasAllSupportArrived</code> means that the ruck can only continue to next stage (resolving it) if all the players from both teams have arrived	<pre> private Vector3 CalculateScrumhalfPosition() { return RuckPosition + (-GetAttackDirection() * config.scrumhalfDistance); } private bool HasAllSupportArrived() { return attackingSupportArrived >= attackingSupportExpected && defendingSupportArrived >= defendingSupportExpected; } private bool IsAttackingTeam(Team team) { return team == attackingTeam; } private Vector3 GetAttackDirection() { Vector3 dir = attackingTeam.attackDirection; dir.y = 0f; if (dir.sqrMagnitude < 0.01f) dir = Vector3.forward; } </pre>

	<pre> return dir.normalized; } private Vector3 GetLateralDirection() { return Vector3.Cross(Vector3.up, GetAttackDirection()).normalized; } </pre>
public class RuckStack	
This class uses a Stack<Player> to store the players in the ruck in a LIFO structure. That means the last player added is the first player to leave the ruck. I chose this data structure as it simulates how rucks work in real life as the last people who join tend to leave first.	<pre> using UnityEngine; using System.Collections.Generic; public class RuckStack Vector3 ruckPosition; Vector3 stackDirection; float lateralOffset; Stack<Player> stack = new Stack<Player>(); bool isUnloading; float unloadInterval; float unloadTimer; public RuckStack(Vector3 ruckPosition, Vector3 stackDirection, float lateralOffset) { this.ruckPosition = ruckPosition; this.stackDirection = stackDirection.normalized; this.lateralOffset = lateralOffset; } </pre>
This method pushes players to the stack. The one below unloads players at a set interval.	<pre> public void Push(Player player) { stack.Push(player); } public void BeginUnloading(float interval) { isUnloading = true; unloadInterval = interval; unloadTimer = 0f; } </pre>
Unloading is done where players from	<pre> public void Tick(float deltaTime) { if (!isUnloading) return; if (IsEmpty()) return; </pre>

each team are popped off the stack at fixed intervals meaning they all don't just immediately pop off at the same time as this would be unrealistic.	<pre>unloadTimer += deltaTime; if (HasReachedUnloadInterval()) { UnloadTopPlayer(); ResetUnloadTimer(); }</pre>
	<pre>public bool IsEmpty() { return stack.Count == 0; } public int Count() { return stack.Count; } private bool HasReachedUnloadInterval() { return unloadTimer >= unloadInterval; }</pre>
These methods are used for popping players off the stack. They are first removed from the data structure, then they are set to idlestate.	<pre>private void UnloadTopPlayer() { Player player = stack.Pop(); ReleasePlayer(player); } private void ReleasePlayer(Player player) { if (player == null) return; player.stateMachine.SM.SetState(new AIIdleState(player.stateMachine)); } private void ResetUnloadTimer() { unloadTimer = 0f; }</pre>

Lineout Subsystem	
public class LineoutManager : IContest The use of the Interface IContest allows it to receive the QTE result without needing to know anything about the lineout. This keeps these systems separate and is an example of polymorphism.	
This enum is used to represent the current phase of the lineout. Conditional checks use these values to make sure that valid actions can only happen in the each phase (Contesting = QTE event running)	<pre>using System.Collections.Generic; using UnityEngine; using System; public class LineoutManager : IContest public enum LineoutPhase { Forming, // Players running to slots SelectingSlot, // Human picks Front/Middle/Back Contesting, // Needle QTE active Complete // Lineout resolved, players released }</pre>
This class controls the full lifecycle of the lineout. Using the enums as stated above means actions cannot happen out of order. This gives a clear flow and also makes the code easier to read.	<pre>public LineoutPhase Phase { get; private set; } public Vector3 LineoutPosition { get; private set; } Team throwingTeam; Team defendingTeam; Ball ball; LineoutConfig config; LineoutJumpQueue throwingQueue = new LineoutJumpQueue(); LineoutJumpQueue defendingQueue = new LineoutJumpQueue(); int throwingArrived; int defendingArrived; int throwingExpected; int defendingExpected; // Slot index 0=Front 1=Middle 2=Back int selectedSlotIndex = -1; public event Action OnLineoutComplete;</pre>

	<pre> public event Action OnFormingComplete; // Triggers UI to show slot selection public LineoutManager(Team throwingTeam, Team defendingTeam, Ball ball, LineoutConfig config, Vector3 lineoutPosition) { this.throwingTeam = throwingTeam; this.defendingTeam = defendingTeam; this.ball = ball; this.config = config; this.lineoutPosition = lineoutPosition; LineoutPosition = lineoutPosition; } Vector3 lineoutPosition; </pre>
	<pre> public void StartLineout() { Phase = LineoutPhase.Forming; ball.SetLineoutLock(true); SendPlayersToSlots(); } </pre>
The debug logs that are commented out were used during testing (iterative) when something wasn't working properly and I was trying to find out where the issue was. The lineout only moves to SelectingSlot when both teams have the right amount of players in the lineout.	<pre> // Called by LineoutNeedleUI when human selects Front/Middle/Back public void NotifySlotSelected(int slotIndex) { if (Phase != LineoutPhase.SelectingSlot) return; selectedSlotIndex = slotIndex; Phase = LineoutPhase.Contesting; } // Called by LineoutNeedleUI when the QTE resolves public void NotifyContestResult(bool throwingTeamWon) { if (Phase != LineoutPhase.Contesting) return; Player winner = DetermineWinner(throwingTeamWon); ReleaseAllPlayers(); CompleteLineout(); GiveBallToWinner(winner); } // Called by LineoutJoinState when a player arrives at their slot public void NotifyPlayerArrived(Player player, bool isAttacking) { if (isAttacking) throwingArrived++; else defendingArrived++; //Debug.Log(\$"Player arrived: {player.name}, </pre>

	<pre>isAttacking={isAttacking}, throwingArrived={throwingArrived}/{throwingExpected}, defendingArrived={defendingArrived}/{defendingExpected}"); if (HasAllPlayersArrived()) { //Debug.Log("All players arrived - firing OnFormingComplete"); Phase = LineoutPhase.SelectingSlot; OnFormingComplete?.Invoke(); } }</pre>
	<pre>public Vector3 GetSlotPosition(int slotIndex, bool isAttacking) { return CalculateSlotPosition(slotIndex, isAttacking); } public int GetPlayerCount() { return config.playersPerTeam; }</pre>
This method builds the lineout by finding the closest forwards on each team and adding them to the lineout. The Mathf.Min returns the smallest value out of the two. I used it mainly at the start of when I was programming when I used a small number of players for testing just so it	<pre>private void SendPlayersToSlots() { List<Player> throwingForwards = FindClosestForwards(throwingTeam); List<Player> defendingForwards = FindClosestForwards(defendingTeam); throwingExpected = Mathf.Min(throwingForwards.Count, config.playersPerTeam); defendingExpected = Mathf.Min(defendingForwards.Count, config.playersPerTeam); SendTeamToSlots(throwingForwards, isAttacking: true); SendTeamToSlots(defendingForwards, isAttacking: false); }</pre>

wouldn't get stuck.	
This method sends the selected players into the lineout by adding the selected player to the correct queue along with their slot position. LineoutJumpQueue uses a ternary operator (instead of an if statement, if isAttacking is true then queue = throwingQueue). I was planning on implementing this all over to save time however I decided against it due to it being harder to read.	<pre> private void SendTeamToSlots(List<Player> players, bool isAttacking) { int count = Mathf.Min(players.Count, config.playersPerTeam); Debug.Log(\$"SendTeamToSlots: isAttacking={isAttacking}, count={count}"); LineoutJumpQueue queue = isAttacking ? throwingQueue : defendingQueue; for (int i = 0; i < count; i++) { Player player = players[i]; Vector3 slotPos = CalculateSlotPosition(i, isAttacking); queue.Enqueue(player, slotPos); player.rb.MovePosition(slotPos); player.stateMachine.SM.SetState(new LineoutBoundState(player.stateMachine)); NotifyPlayerArrived(player, isAttacking); } } </pre>
	<pre> //Dont think this works yet need to test more (maybe add some gizmos) private Vector3 CalculateSlotPosition(int slotIndex, bool isAttacking) { // Players line up along Z axis (perpendicular to touchline) </pre>

	<pre> // Throwing team on one side of lineout position X, depending on the other float sideOffset; if (isAttacking) { sideOffset = -config.teamGap; } else { sideOffset = config.teamGap; } float zOffset = slotIndex * config.slotSpacing; return new Vector3(LineoutPosition.x + sideOffset, LineoutPosition.y, LineoutPosition.z + zOffset); } </pre>
	<pre> private Player DetermineWinner(bool throwingTeamWon) { if (throwingTeamWon) { // Ball goes to selected slot player on throwing team return throwingQueue.GetPlayer(selectedSlotIndex); } else { // Defending team wins - give to the player in the contested slot return defendingQueue.GetPlayer(selectedSlotIndex); } } </pre>
	<pre> private void GiveBallToWinner(Player winner) { if (winner == null) { DebugNullLineout(); ball.SetLineoutLock(false); return; } // Reuse PassHandler arc throw to the winning player ball.passHandler.RequestPassToPlayer(winner.transform); } </pre>
	<pre> private void ReleaseAllPlayers() { ReleaseQueue(throwingQueue); ReleaseQueue(defendingQueue); } </pre>

	<pre> } private void ReleaseQueue(LineoutJumpQueue queue) { while (queue.Dequeue(out Player player, out Vector3 _)) { if (player == null) continue; player.stateMachine.SM.SetState(new AIIdleState(player.stateMachine)); } } </pre>
	<pre> private void CompleteLineout() { Phase = LineoutPhase.Complete; ball.SetLineoutLock(false); OnLineoutComplete?.Invoke(); } private bool HasAllPlayersArrived() { return throwingArrived >= throwingExpected && defendingArrived >= defendingExpected && throwingExpected > 0 && defendingExpected > 0; } </pre>
The list of forwards are sorted using a custom comparison that orders players by their distance to the lineout. The sort method repeatedly compares pairs of players using CompareDistanceToLineout.	<pre> private List<Player> FindClosestForwards(Team team) { List<Player> forwards = GetForwards(team); SortByDistanceToLineout(forwards); return TakeFirst(forwards, config.playersPerTeam); } private List<Player> GetForwards(Team team) { List<Player> forwards = new List<Player>(); if (team?.players == null) return forwards; foreach (Player player in team.players) { if (player != null && player.playerGroup == PlayerGroup.Forward) forwards.Add(player); } return forwards; } private void SortByDistanceToLineout(List<Player> players) { players.Sort(CompareDistanceToLineout); } private int CompareDistanceToLineout(Player a, Player b) { </pre>

	<pre> return DistanceToLineout(a).CompareTo(DistanceToLineout(b)); } private float DistanceToLineout(Player player) { return Vector3.Distance(player.transform.position, LineoutPosition); } </pre>
	<pre> private List<Player> TakeFirst(List<Player> players, int count) { List<Player> result = new List<Player>(); for (int i = 0; i < Mathf.Min(count, players.Count); i++) result.Add(players[i]); return result; } </pre>
	<pre> private void DebugNullLineout() { Debug.LogWarning("LineoutManager: Winner player is null, dropping ball."); } </pre>
public class LineoutJumpQueue	
I use a queue based structure to store lineout participants and their slot positions in matching order. A queue uses a FIFO structure meaning the players can be dequeued in the correct order.	<pre> using UnityEngine; using System.Collections.Generic; public class LineoutJumpQueue Queue<Vector3> slotPositions = new Queue<Vector3>(); Queue<Player> assignedPlayers = new Queue<Player>(); //returns the number of slot positions public int Count { get { return slotPositions.Count; } } </pre>
	<pre> public void Enqueue(Player player, Vector3 slotPosition) { assignedPlayers.Enqueue(player); slotPositions.Enqueue(slotPosition); } </pre>

	<pre> }</pre>
	<pre> public bool Dequeue(out Player player, out Vector3 position) { if (assignedPlayers.Count == 0) { player = null; position = Vector3.zero; return false; } player = assignedPlayers.Dequeue(); position = slotPositions.Dequeue(); return true; }</pre>
	<pre> public Vector3 GetSlotPosition(int index) { Vector3[] positions = slotPositions.ToArray(); if (index < 0 index >= positions.Length) return Vector3.zero; return positions[index]; } public Player GetPlayer(int index) { Player[] players = assignedPlayers.ToArray(); if (index < 0 index >= players.Length) return null; return players[index]; }</pre>
	<pre> public void Clear() { slotPositions.Clear(); assignedPlayers.Clear(); }</pre>
public class LineoutController : MonoBehaviour	
This links the events with the UI and manager. It listens for the outOfBounds event and creates the lineout Manager. It	<pre> using UnityEngine; public class LineoutController : MonoBehaviour public static LineoutController Instance { get; private set; } public LineoutSlotSelectorUI slotSelectorUI; public NeedleController needleController; LineoutManager activeLineout; Team lastTouchingTeam;</pre>

then subscribes to the lineout events	<pre> private void Awake() { if (Instance == null) Instance = this; else Destroy(gameObject); } private void Start() { SubscribeToEvents(); } private void OnDestroy() { UnsubscribeFromEvents(); } </pre>
	<pre> public void RegisterOutOfBounds(Vector3 position, Team lastTouchingTeam) { if (activeLineout != null) return; this.lastTouchingTeam = lastTouchingTeam; StartLineout(position); } private void StartLineout(Vector3 position) { MatchManager manager = MatchManager.Instance; if (manager == null) { Debug.LogWarning("LineoutController: ball is null on MatchManager."); return; } Team throwingTeam = DetermineThrowingTeam(manager); Team defendingTeam = throwingTeam == manager.TeamA ? manager.TeamB : manager.TeamA; LineoutConfig config = GetConfig(); if (config == null) { Debug.LogWarning("LineoutController: LineoutConfig missing from GameConfig."); return; } Vector3 lineoutPosition = CalculateLineoutPosition(position, config); //Debug.Log(\$"LineoutManager created. Throwing: {throwingTeam.name}, Defending: {defendingTeam.name}"); activeLineout = new LineoutManager(throwingTeam, </pre>

	<pre>defendingTeam, manager.ball, config, lineoutPosition); activeLineout.OnFormingComplete += OnFormingComplete; activeLineout.OnLineoutComplete += OnLineoutComplete; manager.RegisterLineout(activeLineout); activeLineout.StartLineout(); }</pre>
	<pre>public void ForceReset() { Debug.Log("LineoutController: ForceReset called"); if (activeLineout != null) { activeLineout.OnFormingComplete -= OnFormingComplete; activeLineout.OnLineoutComplete -= OnLineoutComplete; activeLineout = null; } slotSelectorUI?.Hide(); MatchManager.Instance?.ClearActiveLineout(); }</pre>
	<pre>private void OnFormingComplete() { if (slotSelectorUI == null) { Debug.LogWarning("LineoutController: SlotSelectorUI not assigned."); return; } slotSelectorUI.Show(activeLineout); } private void OnLineoutComplete() { activeLineout.OnFormingComplete -= OnFormingComplete; activeLineout.OnLineoutComplete -= OnLineoutComplete; activeLineout = null; MatchManager.Instance?.ClearActiveLineout(); }</pre>
	<pre>private Team DetermineThrowingTeam(MatchManager manager) { if (lastTouchingTeam == manager.TeamA) { return manager.TeamB; } else { </pre>

	<pre> return manager.TeamA; } }</pre>
	<pre>private Vector3 CalculateLineoutPosition(Vector3 outPosition, LineoutConfig config) { float infieldX = SnapInfield(outPosition.x, config.touchlineOffset); return new Vector3(infieldX, outPosition.y, outPosition.z); } private float SnapInfield(float xPos, float offset) { if (xPos > 0) { return xPos - offset; } return xPos + offset; }</pre>
	<pre>private LineoutConfig GetConfig() { return MatchManager.Instance?.config?.lineoutConfig; } private void SubscribeToEvents() { if (ScoreManager.Instance != null) ScoreManager.Instance.OnBallOutOfBounds += OnBallOutOfBounds; } private void UnsubscribeFromEvents() { if (ScoreManager.Instance != null) ScoreManager.Instance.OnBallOutOfBounds -= OnBallOutOfBounds; } private void OnBallOutOfBounds(Vector3 position) { Team lastTouching = MatchManager.Instance?.PossessionTeam; RegisterOutOfBounds(position, lastTouching); }</pre>

Conversion Subsystem

```
public class ConversionManager : IContest
```

```
using System;
using UnityEngine;

public class ConversionManager : IContest

public event Action<bool> OnConversionComplete; // true =
scored

Team scoringTeam;
Ball ball;
ConversionConfig config;
NeedleController needle;
Transform postsCentreTarget;
Vector3 tryPosition;
Player kicker;

public ConversionManager(Team scoringTeam, Ball ball,
ConversionConfig config, NeedleController needle, Transform
postsCentreTarget, Vector3 tryPosition)
{
    this.scoringTeam = scoringTeam;
    this.ball = ball;
    this.config = config;
    this.needle = needle;
    this.postsCentreTarget = postsCentreTarget;
    this.tryPosition = tryPosition;
}
```

```
public void StartConversion()
{
    PositionKicker();
    AttachBallToKicker();
    ActivateNeedle();
}

// IKickContest
public void NotifyContestResult(bool attackingTeamWon)
{
    KickBall(attackingTeamWon);
    OnConversionComplete?.Invoke(attackingTeamWon);
}
```

```
private void PositionKicker()
{
}
```

	<pre> kicker = FindKicker(); if (!ValidateKicker()) return; Vector3 kickPos = CalculateKickerPosition(); kicker.rb.MovePosition(kickPos); kicker.SetControlled(false); kicker.stateMachine.SM.SetState(new AIIdleState(kicker.stateMachine)); DebugLogKickerPositioned(kickPos); } </pre>
	<pre> private void AttachBallToKicker() { if (!ValidateKicker()) return; ball.AttachTo(kicker.transform); } private void ActivateNeedle() { needle.Activate(this); } </pre>
	<pre> private void KickBall(bool success) { Vector3 target; if (success) { target = CalculateSuccessTarget(); } else { target = CalculateFailTarget(); } Transform targetTransform = BuildTargetTransform(target); ball.passHandler.ForcePassToTarget(targetTransform); DebugLogKick(success, target); } </pre>
CalculateFailTarget randomly chooses a side (left or right) and offsets the target so it misses the	<pre> private Vector3 CalculateKickerPosition() { Vector3 attackDir = GetAttackDirection(); return tryPosition - (attackDir * config.kickerDepth); } private Vector3 CalculateSuccessTarget() { return postsCentreTarget.position; } </pre>

post.	<pre> } private Vector3 CalculateFailTarget() { float side = UnityEngine.Random.value > 0.5f ? 1f : -1f; return postsCentreTarget.position + GetLateralDirection() * (config.missLateralOffset * side); } </pre>
A new temporary game object is made in the scene. The object is put in the exact place the kick/pass is meant to go. The object is deleted after 3 seconds so the animation has time to complete.	<pre> // Creates a temporary scene Transform for RequestPassToPlayer to target. // Destroyed after 3 seconds - well past the arc animation duration. private Transform BuildTargetTransform(Vector3 worldPos) { GameObject target = new GameObject("_ConversionKickTarget"); target.transform.position = worldPos; UnityEngine.Object.Destroy(target, 3f); return target.transform; } </pre>
	<pre> private Player FindKicker() { if (scoringTeam?.players == null scoringTeam.players.Count == 0) return null; return scoringTeam.players[0]; } private Vector3 GetAttackDirection() { if (scoringTeam == null) return Vector3.forward; Vector3 dir = scoringTeam.attackDirection; dir.y = 0f; if (dir.sqrMagnitude < 0.01f) dir = Vector3.forward; return dir.normalized; } private Vector3 GetLateralDirection() { return Vector3.Cross(Vector3.up, GetAttackDirection()).normalized; } private bool ValidateKicker() </pre>

	<pre> { if (kicker != null) return true; DebugLogMissingKicker(); return false; } </pre>
	<pre> private void DebugLogKickerPositioned(Vector3 pos) { Debug.Log(\$"ConversionManager: Kicker {kicker.name} positioned at {pos}"); } private void DebugLogKick(bool success, Vector3 target) { Debug.Log(\$"ConversionManager: Kick {(success ? "successful" : "failed")}, target={target}"); } private void DebugLogMissingKicker() { Debug.LogWarning("ConversionManager: No kicker found on scoring team."); } </pre>
public class ConversionController : MonoBehaviour	
	<pre> using UnityEngine; using System; public class ConversionController : MonoBehaviour public NeedleController needleController; public Transform teamAPostsTarget; public Transform teamBPostsTarget; public ConversionResultUI conversionResultUI; bool waitingForKickFinish; float kickTimer; bool kickSuccessPending; ConversionManager activeConversion; </pre>
	<pre> private void Update() { if (!waitingForKickFinish) return; kickTimer -= Time.deltaTime; if (kickTimer > 0f) return; waitingForKickFinish = false; </pre>

	<pre> MatchManager.Instance.SetConversionActive(false); if (activeConversion != null) { activeConversion.OnConversionComplete -= OnConversionComplete; activeConversion = null; } ShowResultUI(kickSuccessPending); ScoreManager.Instance.FinaliseScore(null); } </pre>
	<pre> private Transform SelectPostsTarget(Team scoringTeam) { if (scoringTeam == MatchManager.Instance?.TeamA) return teamAPostsTarget; return teamBPostsTarget; } public void HandleTryScored(Team scoringTeam, Vector3 tryPosition) { if (activeConversion != null) return; if (!ValidateReferences()) return; ConversionConfig config = GetConfig(); if (!ValidateConfig(config)) return; PreparePlayersForConversion(); activeConversion = BuildConversionManager(scoringTeam, config, tryPosition); activeConversion.OnConversionComplete += OnConversionComplete; activeConversion.StartConversion(); DebugLogConversionStarted(scoringTeam); } </pre>
	<pre> private void OnConversionComplete(bool success) { if (success) ScoreManager.Instance.RegisterConversion(); kickSuccessPending = success; waitingForKickFinish = true; kickTimer = 1.0f; // finetune </pre>

	<pre> if (MatchTimerManager.Instance?.IsInExtraTime == true) MatchTimerManager.Instance?.NotifyDeadBall(); } </pre>
	<pre> private ConversionManager BuildConversionManager(Team scoringTeam, ConversionConfig config, Vector3 tryPosition) { return new ConversionManager(scoringTeam, MatchManager.Instance.ball, config, needleController, SelectPostsTarget(scoringTeam), tryPosition); } private void PreparePlayersForConversion() { MatchManager.Instance.SetConversionActive(true); ReleaseTeam(MatchManager.Instance.TeamA); ReleaseTeam(MatchManager.Instance.TeamB); } </pre>
The second if in the foreach is defensively programmed so if the player is null it just skips it.	<pre> private void ReleaseTeam(Team team) { if (team?.players == null) return; foreach (Player player in team.players) { if (player == null) continue; player.SetControlled(false); player.stateMachine.SM.SetState(new AIIIdleState(player.stateMachine)); } } </pre>
	<pre> private void ShowResultUI(bool success) { if (conversionResultUI == null) { DebugLogMissingResultUI(); return; } conversionResultUI.Show(success); } private ConversionConfig GetConfig() { return MatchManager.Instance?.config?.conversionConfig; } private bool ValidateReferences() { if (needleController != null && teamAPostsTarget != null && </pre>

	<pre> teamBPostsTarget != null && MatchManager.Instance?.ball != null) return true; DebugLogMissingReferences(); return false; } private bool ValidateConfig(ConversionConfig config) { if (config != null) return true; DebugLogMissingConfig(); return false; } </pre>
	<pre> private void DebugLogConversionStarted(Team scoringTeam) { Debug.Log(\$"ConversionController: Conversion started for {scoringTeam.name}"); } private void DebugLogMissingResultUI() { Debug.LogWarning("ConversionController: ConversionResultUI not assigned."); } private void DebugLogMissingReferences() { Debug.LogWarning("ConversionController: Missing NeedleController, postsCentreTarget, or Ball."); } private void DebugLogMissingConfig() { Debug.LogWarning("ConversionController: ConversionConfig missing from GameConfig."); } private void DebugLogSubscribed() { Debug.Log("ConversionController: Subscribed to OnTryScoredPreConversion"); } private void DebugLogMissingScoreManager() { Debug.LogWarning("ConversionController: ScoreManager.Instance is null on subscribe."); } </pre>
public class NeedleController : MonoBehaviour	

	<pre> using UnityEngine; public class NeedleController : MonoBehaviour public Transform pointA; public Transform pointB; public RectTransform safeZone; public SafeZoneRandomiser safeZoneRandomiser; public float moveSpeed = 200f; RectTransform pointerTransform; Vector3 targetPosition; IContest activeContest; bool isActive; </pre>
	<pre> private void Awake() { pointerTransform = GetComponent<RectTransform>(); //gameObject.SetActive(false); } private void Update() { if (!isActive) return; MovePointer(); CheckBoundsReached(); CheckInput(); } </pre>
	<pre> // Called by LineoutSlotSelectorUI : LineoutManager implements IKickContest public void Activate(IContest contest) { SetupContest(contest); } public void Deactivate() { isActive = false; transform.parent.gameObject.SetActive(false); } </pre>
RandomisePosition creates a random position	<pre> private void SetupContest(IContest contest) { //Debug.Log(\$"NeedleController: SetupContest called, parent={transform.parent?.name}, this.gameObject.activeSelf={gameObject.activeSelf}"); </pre>

between the two ends of the needle where the safeZone can spawn. This creates some variability while playing so the game doesn't become boring.	<pre>activeContest = contest; isActive = true; targetPosition = pointB.position; safeZoneRandomiser.RandomisePosition(); transform.parent.gameObject.SetActive(true); //Debug.Log(\$"NeedleController: After SetActive, parent.activeSelf={transform.parent?.gameObject.activeSelf}"); }</pre>
	<pre>private void MovePointer() { pointerTransform.position = Vector3.MoveTowards(pointerTransform.position, targetPosition, moveSpeed * Time.deltaTime); } private void CheckBoundsReached() { if (Vector3.Distance(pointerTransform.position, pointA.position) < 0.1f) targetPosition = pointB.position; else if (Vector3.Distance(pointerTransform.position, pointB.position) < 0.1f) targetPosition = pointA.position; } private void CheckInput() { if (Input.GetMouseButtonDown(0)) ResolveAttempt(); } private void ResolveAttempt() { bool success = IsPointerInSafeZone(); isActive = false; DebugLogResult(success); activeContest?.NotifyContestResult(success); activeContest = null; Deactivate(); }</pre>

	<pre>private void MovePointer() { pointerTransform.position = Vector3.MoveTowards(pointerTransform.position, targetPosition, moveSpeed * Time.deltaTime); } private void CheckBoundsReached() { if (Vector3.Distance(pointerTransform.position, pointA.position) < 0.1f) targetPosition = pointB.position; else if (Vector3.Distance(pointerTransform.position, pointB.position) < 0.1f) targetPosition = pointA.position; } private void CheckInput() { if (Input.GetMouseButtonDown(0)) ResolveAttempt(); } private void ResolveAttempt() { bool success = IsPointerInSafeZone(); isActive = false; DebugLogResult(success); activeContest?.NotifyContestResult(success); activeContest = null; Deactivate(); } private bool IsPointerInSafeZone() { return RectTransformUtility.RectangleContainsScreenPoint(safeZone, pointerTransform.position, null); }</pre>
	<pre>private void DebugLogResult(bool success) { if (success) { Debug.Log("Needle QTE: Success"); } else { Debug.Log("Needle QTE: Fail"); } }</pre>

	}

Core Gameplay Entities

[System.Serializable]
public class PlayerStats

This creates a slider from 0-100 which can be adjusted in the inspector

```
using UnityEngine;

[System.Serializable]
public class PlayerStats

[Range(0f, 100f)] public float handling;
[Range(0f, 100f)] public float speed;
[Range(0f, 100f)] public float stamina;
[Range(0f, 100f)] public float aggression;
```

public class Player : MonoBehaviour

The player object contains its own PlayerStateMachine. This means the behaviour is managed independently by each player rather than globally. This is an example of **aggregation** PlayerStateMachine is a component of Player.

```
public class Player : MonoBehaviour

public Team team;

public bool isControlled { get; private set; }
public bool hasBall { get; private set; }

public PlayerGroup playerGroup;

public PlayerStats stats = new PlayerStats();
[HideInInspector] public float currentStamina;
public float handlingStat // so that I don't have to edit
PassProbabilityAlgorithm
{
    get { return stats.handling; }
}

static readonly PlayerStats ForwardPreset =
CreateForwardPreset();
static readonly PlayerStats BackPreset = CreateBackPreset();

private static PlayerStats CreateForwardPreset()
{
```

	<pre>return new PlayerStats { handling = 55f, speed = 60f, stamina = 60f, aggression = 80f }; } private static PlayerStats CreateBackPreset() { return new PlayerStats { handling = 80f, speed = 80f, stamina = 80f, aggression = 50f }; } public Rigidbody rb { get; private set; } public SpriteRenderer sr { get; private set; } public Animator anim { get; private set; } public PlayerMovement movement { get; private set; } public PlayerAnimator animController { get; private set; } public PlayerStateMachine stateMachine { get; private set; } private void Awake() { ApplyGroupPreset(); currentStamina = stats.stamina; InitialiseComponents(); InitialiseSystems(); }</pre>
	<pre>private void ApplyGroupPreset() { PlayerStats preset; if (playerGroup == PlayerGroup.Forward) { preset = ForwardPreset; } else { preset = BackPreset; } stats.handling = preset.handling; stats.speed = preset.speed; stats.stamina = preset.stamina; stats.aggression = preset.aggression;</pre>

	<pre>} </pre>
	<pre>private void InitialiseComponents() { rb = GetComponent<Rigidbody>(); sr = GetComponent<SpriteRenderer>(); anim = GetComponent<Animator>(); } private void InitialiseSystems() { movement = GetComponent<PlayerMovement>(); animController = GetComponent<PlayerAnimator>(); stateMachine = GetComponent<PlayerStateMachine>(); InitialiseMovementSystems(); InitialiseAnimatorSystems(); InitialiseStateMachineSystems(); } private void InitialiseMovementSystems() { if (movement != null) { movement.Initialise(this); } } private void InitialiseAnimatorSystems() { if (animController != null) { animController.Initialise(this); } } private void InitialiseStateMachineSystems() { if (stateMachine != null) { stateMachine.Initialise(this); } } </pre>
	<pre>public void SetControlled(bool controlled) { isControlled = controlled; } public void SetHasBall(bool hasBall) { this.hasBall = hasBall; NotifyAnimatorOfBallState(hasBall); } public void NotifyAnimatorOfBallState(bool hasBall) </pre>

	<pre> { animController?.UpdateBallState(hasBall); } </pre>
public class Team : MonoBehaviour	
	<pre> using UnityEngine; using System.Collections.Generic; public class Team : MonoBehaviour public List<Player> players = new List<Player>(); public Vector3 attackDirection = Vector3.forward; GameConfig config; private void Awake() { if (MatchManager.Instance != null) { config = MatchManager.Instance.config; } else { config = FindAnyObjectByType<GameConfig>(); } // Auto-collect players var foundPlayers = GetComponentsInChildren<Player>(); foreach (var p in foundPlayers) { if (p != null) { players.Add(p); p.team = this; } } } </pre>
	<pre> //Finds the best support player on a given side public Player FindBestPassTarget(Player carrier, bool leftSide) { if (carrier == null) return null; Player best = null; float bestDist = Mathf.Infinity; Vector3 carrierPos = carrier.transform.position; Vector3 atk = GetNormalizedAttackDirection(); Vector3 leftVec = Vector3.Cross(atk, Vector3.up); Vector3 sideVec; </pre>

	<pre>if (leftSide) { sideVec = leftVec; } else { sideVec = -leftVec; } foreach (var p in players) { if (p == null p == carrier) continue; Vector3 toPlayer = p.transform.position - carrierPos; // Must be behind or level (>0 = infront) float forwardDot = Vector3.Dot(toPlayer, atk); if (forwardDot > 0.01f) continue; // Must be on correct side (>0 = correct side) float sideDot = Vector3.Dot(toPlayer, sideVec); if (sideDot <= 0.01f) continue; float dist = toPlayer.sqrMagnitude; if (dist < bestDist) { bestDist = dist; best = p; } } return best; }</pre>
	<pre>//Calculates where the pass should land on ground if no player is suitable public Vector3 CalculateGroundPassTarget(Player carrier, bool leftSide) { if (carrier == null) return Vector3.zero; Vector3 atk = GetNormalizedAttackDirection(); Vector3 leftVec = Vector3.Cross(Vector3.up, atk).normalized; Vector3 sideVec; if (leftSide) { sideVec = leftVec; } else</pre>

	<pre> { sideVec = -leftVec; } Vector3 target = carrier.transform.position - (atk * config.passBackwardDistance) + (sideVec * config.passLateralDistance); target.y = Mathf.Max(0f, target.y); return target; } </pre>
	<pre> private Vector3 GetNormalizedAttackDirection() { Vector3 atk = attackDirection; atk.y = 0f; if (atk.sqrMagnitude < 0.01f) atk = Vector3.forward; return atk.normalized; } </pre>
IContest interface	
This ensures all the lineout and conversion handles in the same way. It allows for polymorphism .	<pre> using UnityEngine; public interface IContest { void NotifyContestResult(bool attackingTeamWon); } </pre>

UI Classes	
public class MatchTimerUI : MonoBehaviour	
	<pre> using TMPro; using UnityEngine; public class MatchTimerUI : MonoBehaviour [SerializeField] private TextMeshProUGUI timerLabel; [SerializeField] private TextMeshProUGUI halfLabel; [SerializeField] private TextMeshProUGUI extraTimeIndicator; </pre>

	<pre>[SerializeField] private GameObject halfTimePanel; [SerializeField] private GameObject fullTimePanel; [SerializeField] private KickoffManager kickoffManager; private void OnEnable() { MatchTimerManager.Instance.OnTimerTick += HandleTick; MatchTimerManager.Instance.OnHalfTime += HandleHalfTime; MatchTimerManager.Instance.OnFullTime += HandleFullTime; } private void OnDisable() { MatchTimerManager.Instance.OnTimerTick -= HandleTick; MatchTimerManager.Instance.OnHalfTime -= HandleHalfTime; MatchTimerManager.Instance.OnFullTime -= HandleFullTime; } private void Start() { HideAllOverlays(); SetHalfLabel(1); SetTimerLabel(0, 0); }</pre>
	<pre>public void OnStartSecondHalfPressed() { HideHalfTimePanel(); kickoffManager.StartSecondHalf(); } private void HandleTick(int minutes, int seconds) { SetTimerLabel(minutes, seconds); SetExtraTimeVisible(MatchTimerManager.Instance.IsInExtraTime); } private void HandleHalfTime() { SetHalfLabel(2); SetTimerLabel(0, 0); SetExtraTimeVisible(false); ShowPanel(halfTimePanel); } private void HandleFullTime() { ShowPanel(fullTimePanel); }</pre>

	<pre> private void SetTimerLabel(int minutes, int seconds) { if (timerLabel) timerLabel.text = \$"{minutes:D2}:{seconds:D2}"; } private void SetHalfLabel(int half) { if (halfLabel) halfLabel.text = \$"HALF {half}"; } private void SetExtraTimeVisible(bool visible) { if (extraTimeIndicator) extraTimeIndicator.gameObject.SetActive(visible); } </pre>
	<pre> private void ShowPanel(GameObject panel) { if (panel) panel.SetActive(true); } private void HideAllOverlays() { if (halfTimePanel) halfTimePanel.SetActive(false); if (fullTimePanel) fullTimePanel.SetActive(false); SetExtraTimeVisible(false); } public void HideHalfTimePanel() { if (halfTimePanel) halfTimePanel.SetActive(false); } </pre>
public class PassIndicatorUI : MonoBehaviour	
	<pre> using UnityEngine; using static PassProbabilityAlgorithm; public class PassIndicatorUI : MonoBehaviour public GameObject leftArrow; public GameObject rightArrow; public float forwardOffset = 0.5f; public float heightOffset = 1.5f; public float sideOffset = 2f; public Color greenColour = new Color(0.2f, 0.8f, 0.2f, 0.9f); public Color yellowColour = new Color(0.9f, 0.8f, 0.1f, 0.9f); public Color redColour = new Color(0.9f, 0.2f, 0.2f, 0.9f); private SpriteRenderer leftArrowSprite; </pre>

	<pre> private SpriteRenderer rightArrowSprite; private MeshRenderer leftArrowMesh; private MeshRenderer rightArrowMesh; private Transform currentPlayer; // World pitch axes: // Across the pitch (left/right) = Z axis // Up the pitch (forward/back) = X axis private static readonly Vector3 PitchForward = Vector3.forward; // +Z private static readonly Vector3 PitchAcross = Vector3.right; // +X private void Awake() { InitialiseArrowComponents(); HideAll(); } </pre>
LateUpdate runs after Update so the arrows are positioned where the final position of the player is meaning they stay aligned.	<pre> private void LateUpdate() { if (currentPlayer != null) { UpdateArrowPositions(); } } </pre>
	<pre> private void InitialiseArrowComponents() { InitialiseLeftArrowComponents(); InitialiseRightArrowComponents(); } private void InitialiseLeftArrowComponents() { if (leftArrow == null) return; leftArrowSprite = leftArrow.GetComponent<SpriteRenderer>(); leftArrowMesh = leftArrow.GetComponent<MeshRenderer>(); } private void InitialiseRightArrowComponents() { if (rightArrow == null) return; rightArrowSprite = rightArrow.GetComponent<SpriteRenderer>(); rightArrowMesh = rightArrow.GetComponent<MeshRenderer>(); } </pre>

	<pre> public void SetFollowTarget(Transform player) { currentPlayer = player; if (player == null) HideAll(); } private void UpdateArrowPositions() { if (currentPlayer == null) return; if (IsArrowVisible(leftArrow)) UpdateLeftArrowPosition(); if (IsArrowVisible(rightArrow)) UpdateRightArrowPosition(); } </pre>
	<pre> private bool IsArrowVisible(GameObject arrow) { return arrow != null && arrow.activeSelf; } private void UpdateLeftArrowPosition() { leftArrow.transform.position = CalculateLeftArrowPosition(); } private void UpdateRightArrowPosition() { rightArrow.transform.position = CalculateRightArrowPosition(); } </pre>
	<pre> // Left = negative Z (across pitch), slightly forward up the pitch private Vector3 CalculateLeftArrowPosition() { Vector3 pos = currentPlayer.position; pos -= PitchAcross * sideOffset; pos += PitchForward * forwardOffset; pos += Vector3.up * heightOffset; return pos; } // Right = positive Z (across pitch), slightly forward up the pitch private Vector3 CalculateRightArrowPosition() { Vector3 pos = currentPlayer.position; pos += PitchAcross * sideOffset; pos += PitchForward * forwardOffset; pos += Vector3.up * heightOffset; return pos; } </pre>
	<pre> public void ShowLeft(PassZone zone, float probability = 0f) </pre>

	<pre> { if (leftArrow == null) return; leftArrow.SetActive(true); SetArrowColour(leftArrowSprite, leftArrowMesh, zone); } public void ShowRight(PassZone zone, float probability = 0f) { if (rightArrow == null) return; rightArrow.SetActive(true); SetArrowColour(rightArrowSprite, rightArrowMesh, zone); } public void HideLeft() { if (leftArrow != null) leftArrow.SetActive(false); } public void HideRight() { if (rightArrow != null) rightArrow.SetActive(false); } public void HideAll() { HideLeft(); HideRight(); } </pre>
	<pre> private void SetArrowColour(SpriteRenderer spriteComponent, MeshRenderer meshComponent, PassZone zone) { Color targetColour = GetColourForZone(zone); ApplyColourToSprite(spriteComponent, targetColour); ApplyColourToMesh(meshComponent, targetColour); } </pre>
	<pre> private Color GetColourForZone(PassZone zone) { if (zone == PassZone.Green) return greenColour; else if (zone == PassZone.Yellow) return yellowColour; else if (zone == PassZone.Red) return redColour; else return Color.white; } </pre>
	<pre> private void ApplyColourToSprite(SpriteRenderer spriteRenderer, Color colour) { if (spriteRenderer != null) spriteRenderer.color = colour; } private void ApplyColourToMesh(MeshRenderer meshRenderer, Color colour) { </pre>

	<pre> if (meshRenderer != null) meshRenderer.material.color = colour; } </pre>
public class KeybindUI : MonoBehaviour	
	<pre> using UnityEngine; using UnityEngine.UI; public class KeybindUI : MonoBehaviour public GameObject panel; public Transform teamARowContainer; public Transform teamBRowContainer; public Button resetButton; public Button closeButton; public KeybindRowUI rowPrefab; bool isOpen; </pre>
	<pre> static readonly (string action, string label)[] Actions = { ("moveUp", "Move Up"), ("moveDown", "Move Down"), ("moveLeft", "Move Left"), ("moveRight", "Move Right"), ("sprint", "Sprint"), ("passLeft", "Pass Left"), ("passRight", "Pass Right"), ("tackle", "Tackle"), ("switchDefender", "Switch Defender"), }; </pre>
	<pre> private void Awake() { Hide(); SetupButtons(); } private void Update() { if (Input.GetKeyDown(KeyCode.Escape)) ToggleMenu(); } public void Show() { isOpen = true; if (panel != null) panel.SetActive(true); } </pre>

	<pre> Time.timeScale = 0f; BuildRows(); } public void Hide() { isOpen = false; if (panel != null) panel.SetActive(false); Time.timeScale = 1f; }</pre>
	<pre>private void ToggleMenu() { if (isOpen) Hide(); else Show(); } private void SetupButtons() { if (resetButton != null) { resetButton.onClick.RemoveAllListeners(); resetButton.onClick.AddListener(OnResetPressed); } if (closeButton != null) { closeButton.onClick.RemoveAllListeners(); closeButton.onClick.AddListener(Hide); } }</pre>
	<pre>private void BuildRows() { if (!ValidateRowPrefab()) return; ClearRows(teamARowContainer); ClearRows(teamBRowContainer); SpawnRows(isTeamA: true, container: teamARowContainer); //teamA is true and use teamARowContainers SpawnRows(isTeamA: false, container: teamBRowContainer); } private void SpawnRows(bool isTeamA, Transform container) { if (container == null) return; foreach (var (action, label) in Actions) { KeybindRowUI row = Instantiate(rowPrefab, container); row.Initialise(isTeamA, action, label); } }</pre>

	<pre> } private void ClearRows(Transform container) { if (container == null) return; for (int i = container.childCount - 1; i >= 0; i--) { Destroy(container.GetChild(i).gameObject); } } </pre>
	<pre> private void OnResetPressed() { KeybindManager.ResetToDefaults(); BuildRows(); } private bool ValidateRowPrefab() { if (rowPrefab != null) return true; DebugLogMissingPrefab(); return false; } </pre>
	<pre> private void DebugLogMissingPrefab() { Debug.LogWarning("KeybindUI: rowPrefab is not assigned."); } </pre>
public class KeybindRowUI : MonoBehaviour	
	<pre> using UnityEngine; using UnityEngine.UI; using TMPro; public class KeybindRowUI : MonoBehaviour public TextMeshProUGUI actionLabel; public Button keyButton; public TextMeshProUGUI keyButtonText; bool isTeamA; string actionName; bool isListening; public void Initialise(bool isTeamA, string actionName, string displayName) { this.isTeamA = isTeamA; this.actionName = actionName; } </pre>

	<pre> if (actionLabel != null) actionLabel.text = displayName; SetupButton(); RefreshKeyDisplay(); } private void Update() { if (!isListening) return; DetectKeypress(); }</pre>
	<pre> private void SetupButton() { if (keyButton == null) return; keyButton.onClick.RemoveAllListeners(); keyButton.onClick.AddListener(OnKeyButtonPressed); } private void OnKeyButtonPressed() { isListening = true; if (keyButtonText != null) keyButtonText.text = "..."; } private void DetectKeypress() { foreach (KeyCode key in System.Enum.GetValues(typeof(KeyCode))) { if (!Input.GetKeyDown(key)) continue; if (key == KeyCode.Escape) { CancelListening(); return; } ApplyNewBinding(key.ToString()); return; } }</pre>
	<pre> private void ApplyNewBinding(string keyName) { KeybindManager.SetBinding(isTeamA, actionName, keyName); isListening = false; RefreshKeyDisplay(); } private void CancelListening() { </pre>

	<pre> isListening = false; RefreshKeyDisplay(); } private void RefreshKeyDisplay() { if (keyButtonText == null) return; TeamKeybinds binds = KeybindManager.GetTeamBinds(isTeamA); keyButtonText.text = GetCurrentKey(binds); } </pre>
	<pre> private string GetCurrentKey(TeamKeybinds binds) { switch (actionName) { case "moveUp": return binds.moveUp; case "moveDown": return binds.moveDown; case "moveLeft": return binds.moveLeft; case "moveRight": return binds.moveRight; case "sprint": return binds.sprint; case "passLeft": return binds.passLeft; case "passRight": return binds.passRight; case "tackle": return binds.tackle; case "switchDefender": return binds.switchDefender; default: return "?"; } } </pre>
public class MainMenuUI : MonoBehaviour	
This class initialises the half length slider on the main menu before the start of the game and sends the	<pre> using TMPro; using UnityEngine; using UnityEngine.SceneManagement; using UnityEngine.UI; public class MainMenuUI : MonoBehaviour [Header("Settings")] [SerializeField] private Slider halfLengthSlider; [SerializeField] private TextMeshProUGUI halfLengthLabel; </pre>

chose value to MatchSettings Manager	<pre>[Header("Navigation")] [SerializeField] private string gameSceneName = "SampleScene"; private void Start() { InitialiseSlider(); } private void InitialiseSlider() { if (halfLengthSlider == null) return; halfLengthSlider.minValue = MatchSettingsManager.MinHalfMinutes; halfLengthSlider.maxValue = MatchSettingsManager.MaxHalfMinutes; halfLengthSlider.wholeNumbers = true; if (MatchSettingsManager.Instance != null) { halfLengthSlider.value = MatchSettingsManager.Instance.HalfLengthMinutes; } else { halfLengthSlider.value = MatchSettingsManager.MinHalfMinutes; } halfLengthSlider.onValueChanged.AddListener(OnSliderChanged); UpdateLabel((int)halfLengthSlider.value); } private void OnSliderChanged(float value) { int minutes = (int)value; MatchSettingsManager.Instance?.SetHalfLength(minutes); UpdateLabel(minutes); } private void UpdateLabel(int minutes) { if (halfLengthLabel != null) { halfLengthLabel.text = \$"{minutes} min per half"; } } public void OnStartPressed() {</pre>
--------------------------------------	---

	<pre> MatchSettingsManager.Instance?.ApplySettings(); SceneManager.LoadScene(gameSceneName); } </pre>
public class ControlHUDManager : MonoBehaviour	
This class updates the onscreen prompts (pass, switch) depending on who is attacking and who is defending	<pre> using UnityEngine; public class ControlHUDManager : MonoBehaviour [SerializeField] private ControlHUDPanel teamAPanel; [SerializeField] private ControlHUDPanel teamBPanel; [SerializeField] private Team teamA; [SerializeField] private Team teamB; private void Start() { SubscribeToPossession(); RefreshBothPanels(); // set initial state } private void OnDestroy() { UnsubscribeFromPossession(); } private void Update() { RefreshBothPanels(); } private void SubscribeToPossession() { if (MatchManager.Instance == null) return; MatchManager.Instance.OnPossessionChanged += OnPossessionChanged; } private void UnsubscribeFromPossession() { if (MatchManager.Instance == null) return; MatchManager.Instance.OnPossessionChanged -= OnPossessionChanged; } private void OnPossessionChanged(Team newPossessor) { </pre>

```
        RefreshBothPanels();
    }

    private void RefreshBothPanels()
    {
        if (!IsMatchReady())
        {
            HideBothPanels();
            return;
        }

        if (IsSetPieceActive())
        {
            HideBothPanels();
            return;
        }

        Team possessor = MatchManager.Instance.PossessionTeam;

        RefreshPanel(teamAPanel, isTeamA: true, possessor);
        RefreshPanel(teamBPanel, isTeamA: false, possessor);
    }

    private void RefreshPanel(ControlHUDPanel panel, bool isTeamA,
        Team possessor)
    {
        if (panel == null) return;

        // If nobody has the ball yet, hide.
        if (possessor == null)
        {
            panel.Hide();
            return;
        }

        TeamKeybinds binds = KeybindManager.GetTeamBinds(isTeamA);
        bool isAttacking = IsThisTeamAttacking(isTeamA, possessor);

        if (isAttacking)
        {
            panel.ShowAttack(binds.passLeft, binds.passRight);
        }
        else
        {
            panel.ShowDefend(binds.tackle, binds.switchDefender);
        }
    }

    private bool IsMatchReady()
```

	<pre> { return MatchManager.Instance != null && teamA != null && teamB != null; } private bool IsSetPieceActive() { return MatchManager.Instance.IsRuckActive() MatchManager.Instance.IsLineoutActive() MatchManager.Instance.IsConversionActive(); } private bool IsThisTeamAttacking(bool isTeamA, Team possessor) { Team thisTeam; if (isTeamA) thisTeam = teamA; else { thisTeam = teamB; } return possessor == thisTeam; } private void HideBothPanels() { teamAPanel?.Hide(); teamBPanel?.Hide(); } </pre>
public class ControlPromptUI : MonoBehaviour	
	<pre> using UnityEngine; using UnityEngine.UI; using TMPro; public class ControlPromptUI : MonoBehaviour [SerializeField] private TextMeshProUGUI actionLabel; // e.g. "Pass Left" [SerializeField] private TextMeshProUGUI keyLabel; // e.g. "Q" public void SetContent(string action, string key) { if (actionLabel != null) actionLabel.text = action; if (keyLabel != null) keyLabel.text = FormatKey(key); } </pre>

	<pre> private string FormatKey(string raw) { switch (raw) { case "LeftShift": return "L.SHIFT"; case "RightShift": return "R.SHIFT"; case "LeftControl": return "L.CTRL"; case "RightControl": return "R.CTRL"; case "LeftArrow": return "←"; case "RightArrow": return "→"; case "UpArrow": return "↑"; case "DownArrow": return "↓"; case "Space": return "SPACE"; case "Tab": return "TAB"; case "Return": return "ENTER"; case "Comma": return ","; case "Period": return "."; default: return raw.ToUpper(); } } </pre>

Configs	
public class GameConfig : MonoBehaviour	
	<pre> using UnityEngine; //Centralised gameplay tuning values public class GameConfig : MonoBehaviour public RuckConfig ruckConfig = new RuckConfig(); public LineoutConfig lineoutConfig = new LineoutConfig(); public ConversionConfig conversionConfig = new ConversionConfig(); public float walkSpeed = 3f; public float sprintSpeed = 6f; public float passSpeed = 30f; public float passArcHeight = 3f; public float passAnimationDelay = 0.15f; public float passStateExitTime = 0.35f; </pre>

	<pre> public Vector3 ballHoldOffset = new Vector3(0f, 1.2f, 0.5f); public float passBackwardDistance = 3f; public float passLateralDistance = 1.5f; public float greenMaxDistance = 5f; public float yellowMaxDistance = 12f; public float distanceWeight = 0.6f; public float handlingWeight = 0.25f; //Player skills are actually used public float difficultyWeight = 0.15f; public float distanceFalloffPower = 1.8f; // How quickly probability drops with distance (2 is closer to quadratic and 1 is linear) public float maxPassDistance = 25f; public float minProbability = 0.05f; public float maxProbability = 0.98f; public float passMissPerpendicularFactor = 0.15f; public float passMissLongitudinalFactor = 0.1f; public float defaultHandlingStat = 70f; //Defensive Config stuff public float smoothingFactor = 0.15f; public float rubberBandSpeed = 6f; public float arrivalThreshold; public float chaseActivationDistance = 6f; public int maxChasers = 2; public float chaseAngleThreshold; public float chaseDisengageDistance = 10f; public float chaseSpeedMultiplier = 1.3f; //Attached to maxDriftSpeed public float passReactionCooldown = 0.35f; //how long before line can fully reset after pass public float looseBallSpacingMultiplier = 0.6f; //When ball loose, line compresses public float rejoinSpeed = 4f; public float rejoinSnapDistance = 1f; </pre>
This creates a new object and sets all the values	<pre> public PassProbabilityAlgorithm CreateProbabilityAlgorithm() { return new PassProbabilityAlgorithm { greenMaxDistance = this.greenMaxDistance, </pre>

from the config.	<pre> yellowMaxDistance = this.yellowMaxDistance, distanceWeight = this.distanceWeight, handlingWeight = this.handlingWeight, difficultyWeight = this.difficultyWeight, distanceFalloffPower = this.distanceFalloffPower, maxPassDistance = this.maxPassDistance, minProbability = this.minProbability, maxProbability = this.maxProbability }; } </pre>
[System.Serializable] public class RuckConfig	
	<pre> using UnityEngine; [System.Serializable] public class RuckConfig public int supportPlayersPerTeam = 2; public float stackSpacing = 0.8f; // Depth between players in stack public float stackOffset = 0.4f; // Lateral offset so players are visible public float unloadInterval = 0.5f; // Seconds between each player releasing public float scrumhalfDistance = 1.5f; // How far behind ruck the scrumhalf stands </pre>
	[System.Serializable] public class LineoutConfig
	<pre> [System.Serializable] public class LineoutConfig public int playersPerTeam = 4; public float slotSpacing = 1.5f; // Distance between slots along Z axis public float teamGap = 1.2f; public float touchlineOffset = 2f; // How far infield the lineout forms from the touchline public float throwDelay = 1.5f; // Seconds before needle starts after slot selected public float needleSpeed = 1.8f; public float needleZoneSize = 0.25f; // Size of the success zone (0-1 range) public float joinMoveSpeed = 6f; public float arrivalThreshold = 1f; // Distance at which player is considered arrived </pre>

public class ConversionConfig	
	<pre> using UnityEngine; public class ConversionConfig public int conversionPoints = 2; public float kickerDepth = 8f; // How far behind the try line the kicker stands public float missLateralOffset = 5f; // How far wide the ball goes on a failed kick </pre>
[System.Serializable] public class FormationSettings	
	<pre> using UnityEngine; [System.Serializable] public class FormationSettings public int numberOfLanes = 4; public float firstLaneDepth = 3f; public float depthIncrement = 2.5f; public float forwardBaseWidth = 3.5f; public float backBaseWidth = 4.5f; public float widthIncrement = 2.5f; public float formationFollowSpeed = 4f; public float arrivalThreshold = 0.3f; public bool forwardsInsideLanes = true; </pre>
public class MatchSettingsManager : MonoBehaviour	
	<pre> using UnityEngine; public class MatchSettingsManager : MonoBehaviour public static MatchSettingsManager Instance; public const int MinHalfMinutes = 3; public const int MaxHalfMinutes = 10; public int HalfLengthMinutes { get; private set; } = 3; private void Awake() { if (Instance != null && Instance != this) { Destroy(gameObject); return; } Instance = this; DontDestroyOnLoad(gameObject); } </pre>


```

public void SetHalfLength(int minutes)
{
    HalfLengthMinutes = Mathf.Clamp(minutes, MinHalfMinutes,
MaxHalfMinutes);
}
public void ApplySettings()
{
    MatchTimerManager.HalfLengthSeconds = HalfLengthMinutes *
60;
    MatchTimerManager.ClockSpeed = 40f / HalfLengthMinutes;
}

```

Testing and Evaluation:

Test Plan:

Movement and Control Tests

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
To1	Player Movement	Check that the controlled player moves in the chosen direction	Press movement keys	Player moves correctly in the applied direction	Pass	00:09
To2	Sprinting	Check that sprint increases movement speed	Hold sprint while moving	Player moves faster than normal movement speed	Pass	00:30
To3	Stamina Drain	Check that sprinting reduces stamin	Sprint continuously for a couple seconds	Stamina value decreases while sprinting	Pass	00:38
To4	Stamina Recovery	Checks that the stamina regenerates while not sprinting	Stop sprinting	Stop sprinting	Pass	00:50
To5	Sprint Limit	Check if player cannot spring if stamina is drained	Hold sprint button when stamina is drained	Fully drain stamina and hold down sprint button	Pass	00:40

To6	Player Switching	Check that the player switched to the closest defending player to ball carrier	Press switch player button while defending	Controlled player changes to nearest defender AI	Pass	01:26
To7	Rebind Menu Input	Check rebind menu opens	Press pause button during game	Game pauses and rebind menu appears	Pass	02:00
To8	Resume Input	Check match resumes from pause	Press resume button in pause menu	Match continues right from where it was paused	Pass	02:06

Passing Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T09	Pass	Check a pass can be made and pass to the correct side	Press the pass button if a teammate is available on that side	Pass goes to the player on that side following the parabolic trajectory	Pass	02:15
T10	Short Ranged Pass Accuracy	Check if short passes are successful most of the time	Repeatedly pass to nearby players	Most passes are successful and go to the player	Pass	3:18
T11	Long Ranged Pass Accuracy	Check if long passes are less successful	Repeatedly pass to far away teammates	Less passes are successful and don't go to the player	Pass	3:50
T12	No Teammate target	Check pass to ground works	Pass to a side where there is no teammate	The pass should go to the ground	Pass	4:16
T13	Possession Update after a successful pass	Check new receiver becomes ball holder	Complete a successful pass	Receiver gains possession	Pass	02:15
T14	Passing UI	Check pass indicators appear when relevant	Check if the pass indicators appear when the new holder gains possession	Pass indicator around the current player	Pass	03:00
T15	Pass Colour	Check the pass indicators change colours depending on distance	Compare the different passing options	Colour changed according to probability	Pass	02:15-4:16

Defensive and Tackling Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T16	Defensive Line Shape	Check AI defenders hold a horizontal line	Start kickoff	Defenders stay in a line, and follow their opposite player, rather than just bunching together	Pass	04:42
T17	Chase Behaviour	Check defenders leave line when attacker comes within a certain distance	Run attacker towards defensive line	Nearest relevant defender moves into chase state	Pass	08:55
T18	Tackle Trigger	Check tackle happens when defender is in range	Move defender into a valid tackle range and press tackle or allow the AI to tackle	Tackle triggers and ruck is formed	Pass	09:05
T19	AI Aggression Effect	Check aggression affects chase distances	Compare players with different aggression values	More aggressive defenders chase at different distances	Fail	09:30

Ruck Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T20	Ruck Creation	Check ruck starts after a successful tackle	Complete a successful tackle	Ruck State Begins	Pass	10:40
T21	Ruck Joiners	Check nearest support players join the ruck	Trigger a ruck with support players nearby	Nearest eligible players move into ruck positions	Pass	11:40
T22	Scrumhalf Assignment	Check a nearby attacking player becomes the scrumhalf when the ball is won	Allow ruck to form	Nearest eligible player moves into scrumhalf role (passing indicators show up and player is given ball)	Pass	11:50
T23	Ruck Resolution	Check ruck resolves correctly	Pass the ball away from the ruck	Players leave on both sides using LIFO, one by one	Pass	12:04

T24	Ruck Possession Update	Check possession changes correctly after ruck	Lose or win ruck	Possession updates to winning team	Pass	12:05
-----	------------------------	---	------------------	------------------------------------	------	-------

Lineout Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T25	Ball into Touch	Check lineout starts when ball goes into touch	Pass or run into touch	Lineout sequence begins	Pass	12:38
T26	Lineout Setup	Check players move into their lineout positions	Start lineout	Eligible players move into their slots	Fail	13:00
T27	Lineout Controls	Check the front middle and back buttons appear	Click one of the buttons	QTE pops up	Pass	13:00
T28	Lineout QTE	Check the QTE event resolves the lineout correctly	Win the QTE	Attacking team win the ball	Pass	13:45
T39	Return to Open Play	Check the game resumes after the lineout ends	Finish lineout	Open play resumes and possession is correct	Pass	14:00

Scoring, Conversion and Kickoff Tests

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T30	Try Scoring	Check try adds 5 points	Run over tryline	Team score increases by 5	Pass	14:40
T31	Scoreboard Updates	Check scoreboard updates after score	Score a try	Scoreboard reflects the new score	Pass	15:30
T32	Conversion QTE starts	Check conversion minigame begins after scoring a try	Score a try	Conversion QTE begins	Pass	15:34
T33	Successful Conversion	Check successful QTE adds 2 points	Complete a successful Conversion	Team score increases by 2	Pass	15:38
T34	Missed Conversion	Check failed conversion doesn't add points	Complete a failed conversion	Team score stays same	Pass	17:10

T35	Kickoff Restart	Check conceding team restarts after score sequence	Click kickoff button	Kickoff setup begins and conceding team gets the ball	Fail	16:00
-----	-----------------	--	----------------------	---	------	-------

Match Flow and Timer Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T36	Match Timer	Check timer counts up to 40 during game	Start match and let game run	Match clock increases over time	Pass	18:35
T37	Match half length	Check timer matches starting half length time use a stopwatch to measure time	Let game run	At 40 the time on stopwatch should match the minutes set at the start	Pass	19:05
T38	Half Time Trigger	Check half time occurs at the correct time	Play until half time	Match enters half time state	Pass	20:35
T39	Second Half Restart	Check play resumes after half time	Continue from half time	Second half begins correctly	Pass	20:50
T40	Full Time Trigger	Check match ends at correct time	Play until timer hits 40 in second half	Match enters full time state	Pass	21:33
T41	Clock Pauses in menu	Check timer stops in pause menus	Open setting mid match	Timer stops	Pass	22:00

FSM and State Transition Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T42	Attack to Defence Transition	Check possession loss swaps systems	Lose the ball in open play	Team changes from attacking system to defensive system	Pass	22:53
T43	Defence to Attack Transition	Check possession gainswaps systems	Gain the ball in open play	Team changes from defensive system to attacking system	Pass	22:53
T44	Gameplay to Ruck	Check successful tackler	Complete tackle	Gameplay FSM	Pass	22:20

	FSM	enters ruck state		transitions to Ruck FSM		
T45	Ruck to Gameplay FSM	Check ruck completion returns to open play	Finish Ruck	Ruck FSM exits and gameplay resumes	Pass	22:28
T46	Gameplay to Lineout FSM	Check ball in touch event triggers lineout state	Send ball into touch	Gameplay GSM transitions into Lineout FSM	Pass	22:50
T47	Lineout to Gameplay FSM	Check lineout completion returns to open play	Complete lineout	Gameplay resumes	Pass	22:53
T48	Try to Conversion FSM	Check scoring triggers conversion state	Score a try	Conversion FSM begins	Pass	14:40
49	Conversion to Kickoff FSM	Check conversion result leads to kickoff	End Conversion	A kickoff button appears	Pass	16:00

Input Mapping and Rebinding Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T50	Rebind Movement Key	Change movement binding and test	Rebind a move key then use it	New key performs movement correctly	Pass	24:12
T51	Rebind Pass Key	Change pass binding and test	Rebind a pass key then use it	New key performs movement correctly	Pass	25:35
T52	Rebind Tackle Key	Change tackle binding and test	Rebind a tackle key then use it	New key performs movement correctly	Pass	25:35
T53	Rebind Switch Player Key	Change switch player binding and test	Rebind a switch player key then use it	New key performs movement correctly	Pass	25:55
T54	Save Bindings	Check rebound controls last through a restart	Rebind controls, close and reopen game	New bindings remain saved	Pass	24:35
T55	Invalid Double Use Handling	Check duplicate bindings behave sensibly	Try assigning same key values to multiple controls	Game prevents it or handles it consistently	Fail	26:20

UI Tests:

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T56	Scoreboard Visibility	Check scoreboard displays during match	Start Match	Scoreboard is visible and readable	Pass	Any point where game is running
T57	Pause UI	Check settings menu opens and functions	Open settings during match	Settings menu appears correctly	Pass	24:12
T58	Conversion UI	Check QTE UI appears for conversion	Score a try	Conversion UI appears	Pass	14:40
T59	Lineout UI	Check lineout UI prompts appear	Trigger lineout	Correct lineout UI appear	Pass	22:50
T60	Pass Indicator UI	Check pass prompts appear at appropriate times	Gain possession	Passing indicator appears clearly	Pass	03:00

Robustness Test (Edge Case Testing):

Test ID	Feature	Test Description/ Purpose	Input/ Action	Expected Result	Pass/ Fail	Evidence (Timestamp)
T61	No Ball Holder	Check game remains stable if ball has no holder	Miss a pass	Game doesn't crash, player has to go and pick up the ball	Pass	27:50
T62	No Valid Pass Target (edge case test)	Check pass still resolves correctly	Attempt pass with no teammate target	Pass goes to ground	Pass	28:25
T63	Pause During an Event (edge case test)	Check pausing during a special event	Pause during a lineout or ruck or conversion	Game pauses and resumes correctly when unpaused	Pass	28:45
T64	Rapid Input Spam (edge case test)	Check repeated button presses do not break stuff	Spam pass, switch and tackle buttons	Game ignores invalid repeats and remains stable	Pass	29:10

[NEA Testing video](#)

Evaluation:

Overall Success:

Overall the project was successful, it achieved its main aim in being a functional 2.5D retro styled rugby game focused on fast paced and accessible gameplay. The core mechanics that I aimed to code were all successfully added, the only issue was the players not lining up correctly in the lineout. However, the game is playable and provides a fun experience for both the casual and avid rugby players. The use of modular systems and architecture like the FSM's and OOP design meant the code was maintainable and scalable.

Evaluating Against Program Objectives:

Ref	Feature	Achieved?	Evaluation / Comment
1.1.1	Movement (WASD / keybinds)	Yes	Movement is responsive and intuitive. The keybind system allows flexibility for different users.
1.1.2	Sprinting + stamina system	Yes	Stamina drains and regenerates correctly, adding realism without negatively impacting gameplay flow.
1.1.3	Player switching	Yes	Nearest player switching works consistently and supports fast paced gameplay.
1.2.1	Pass indicator icons	Yes	Icons clearly show valid targets, improving usability and accessibility.
1.2.2	Pass success varies with distance/stats	Yes	Pass probability algorithm behaves realistically and consistently.
1.2.2.1	Success curve (distance bands)	Yes	Success rates align with expected values and scale appropriately.
1.2.2.2	Colour-coded pass zones (R,Y,G)	Yes	Provides clear visual feedback to the player, improving decision making.

1.3.1	Conversion kicking system	Yes	The conversion system works reliably and integrates into match flow.
1.3.1.1	QTE minigame	Yes	Needle stop minigame adds interactivity.
1.4.1	Player tackle input	Yes	Player initiated tackles function correctly and trigger expected outcomes.
1.4.2	AI tackle behaviour	Yes	AI tackles activate reliably when in range.
1.4.3	Ruck triggered on tackle	Yes	Smooth transition from tackle to ruck system.
1.5.1	Full match flow	Yes	Core gameplay loop functions correctly and transitions smoothly between states.
1.5.2.1	Ruck formation	Yes	Rucks form correctly with players from both teams.
1.5.2.1.1	Stack used for ruck	Yes	Stack (LIFO) accurately models ruck behaviour.
1.5.2.1.2	Player locked during ruck	Yes	Prevents movement.
1.5.2.1.2.1	Ruck success rates	No	Currently 100% win rate.
1.5.2.2	Ball recycled to scrumhalf	Yes	The system works, scrumhalf can only pass.
1.5.2.3	Scrumhalf actions	Yes	Players are able to pass from rucks and lineouts.
1.5.3.1	Lineout trigger	Yes	Triggered correctly when the ball goes out of play.
1.5.3.2/3	Throw choice + QTE	Yes	Player interaction works correctly and is intuitive.

1.5.3.4	Lineout outcome logic	Yes	Incorrect QTE correctly results in turnover.
1.5.3.5	Resume play	Yes	Smooth transition back into open play.
1.5.4.1	Kick-off restart	No	There is a bug when Team A scores
1.5.5.1	Scoring system	Yes	Try and conversion scoring implemented correctly.
1.5.5.2	Match configuration	Yes	Match length configurable via UI.
1.5.5.3–5	Match clock system	Yes	Clock correctly handles pause, halftime, and scaling.
1.5.5.6	Scoreboard updates	Yes	Updates automatically after scoring events.

Ref	Feature	Achieved?	Evaluation / Comment
2.1.1	Input mapping	Yes	All required controls are implemented and functional.
2.2.1	Rebinding system	Yes	JSON-based persistence works reliably.
3.1.1	Attacking FSM	Yes	Maintains structure and supports player movement.
3.1.1.1	Stable assignments	Yes	Prevents unnecessary reshuffling and improves realism.
3.1.1.2	Player availability logic	Yes	Correctly handles unavailable players (ruck, tackle, etc.).
3.1.3	Formation system	Yes	Arrow formation works effectively and updates dynamically.
3.2.1	Defensive FSM	Yes	Clear transitions between defend, chase, and tackle states.

3.2.1.2	Aggression stat effect	Yes/No	Successfully influences AI behaviour. However doesn't work yet for tackle and chase distance
3.2.1.3	Line integrity	Yes	Defenders maintain formation unless transitioning states.
3.2.2	AI tackling	Yes	Works consistently and triggers rucks correctly.
3.2.3	AI ruck behaviour	Yes	Players join rucks appropriately and resume play correctly.

Ref	Feature	Achieved?	Evaluation / Comment
4.1.1	Offside rule	Yes	Enforced correctly using position checks.
5.1	Ball physics (passes)	Yes	Projectile motion behaves realistically.
5.2	Kick physics	Yes	Consistent with the pass system.
6.1	Player stats	Yes	Stats influence gameplay meaningfully.
6.1.3.1	Handling affects passing	Yes	Directly impacts pass success.
6.1.4.1	Aggression affects AI	No	Doesn't influence chase and tackle radius yet.
6.2	Difficulty system	Yes	Difficulty changes AI and gameplay parameters effectively.
7.1.1	HUD	Yes	Displays key information clearly.
7.1.2	Start menu	Yes	Allows match configuration.
7.1.3	Pause menu	Yes	Supports rebinding and game control.

Overall basically every feature was fully implemented, there are only two small bugs which were made last minute which I didn't have time to fix in the kickoff (Team A scoring) and the chase and tackle distance not changing with aggression.

Reference Tests:

My game was tested with a combination of functional and scenario based tests. The individual systems (rucking, passing, lineouts etc) were tested in isolation to ensure the correct behaviour. For example, for the pass probability algorithm, I did different tests at different distances and looked at the success percentages to see if the algorithm was working correctly.

I also performed integration testing to make sure that all the systems interacted correctly with each other, such as the transitions between open play, lineouts, rucks and tries. Stress testing was performed by rapidly switching players and triggering multiple events to handle at the same time, all these systems worked as intended as only one ruck could be created at a time and the Debug popped up for that.

Feedback From Stakeholders:

I let all my stakeholders play my game, taking it in turns. After our session I asked them all to give me feedback on how they thought it went. (casual gamer) said he really enjoyed the fast paced fluid nature of the game and said the controls were easy to learn using the Rebind menu if he ever forgot. The game then paused for both players keeping it fair and resuming in the same place. (middle) then went next, after his session he highlighted how he enjoyed the core game systems in the lineouts and rucks. He also said the QTE minigame was a fun challenge making it not guaranteed every time to get a conversion. (rugby enthusiast) then went next. He was sad how the lineouts didn't line up perfectly but he really enjoyed the passing and the slight randomness involved in every pass and said this was the most realistic part of the game.

Reflection On Feedback:

This feedback validated my decisions for the key components I decided to work on while leaving out scrums and kicks, however it did highlight the need for future development to fully turn the game into a professional level rugby game. This gives me a clear short term and long term goal for the project which I will talk about below.

Future Implementations:

In the near future, I am going to fix the current two bugs in the game where team A falls through the map on kickoff when they score and the aggression stat not changing the tackle and chase radii. I also will fix how the kickoffs are initially set up by moving the forwards into the centres of the list in the inspector. Then, I will add some randomness to rucks using the aggression stat. I will use it so if the defending team's aggression stat is higher, there is a random 10-30% chance of turning over the ball. After I do these, I plan to add 3 attacking set pieces that can be used on kickoff or lineouts. The player will be able to choose which one they want to do when the UI pops up after the lineout is resolved (or kickoff starts). Finally I will add a more advanced, configurable defensive line and make all the UI look good.

Conclusion:

Overall, the project meets its objectives to a high standard, with any limitations being from time constraint rather than flaws in the design and architecture.

Bibliography:

[Brackeys - YouTube](#)

I used this channel to learn core unity concepts such as player movement and UI systems.

▶ [Unity Bots with State Machines - Extensible State Machine / FSM](#)

I used this video to learn about FSM's in unity and ways to implement it

▶ [Designing Luck | How Loot Distributions Work](#)

I used this video to learn about how the theory of probability works in games.

[C# Guide - .NET managed language - C# | Microsoft Learn](#)

I used this website a lot for learning the general structure of how to code with C#

[Unity - Manual](#)

I also used this website for the Unity specific problems I came across

[Unity Learn](#)

If I ever needed to go through certain aspects like FSM implementation in step by step tutorials I used this.

[Events - C# | Microsoft Learn](#)

This was used for me to understand how events work in C#

[PlantUML sequence diagrams](#)

I used this website to learn how to draw/write the sequence diagrams I made

[What does 'dirty' mean as a variable in a line of code? - Quora](#)

I used this forum to learn about what a dirty variable was.

[Scripting API: Gizmos](#)

I used this document to learn how to draw gizmos in my project

[C# regarding Null Conditional Operator & Null Propagation - Microsoft Q&A](#)

I used this to learn about how I can use null conditional operators for defensive programming

Appendix A:

AI uses:

Here is my prompt:

I need to make a rugby game and I have the passing and movement (no scrums or lineouts in my game) what do I need to do in order.

Here's how it helped me:

I was in the middle of coding and felt like I had so much to do in a narrowing time window, so I asked AI to help me create a plan for my next steps. This helped me visualise the progress I was making by being able to tick stuff off the list once I had done it.

I didn't follow this structure exactly, for example I did the ball states second as I found that I needed them to prevent any bugs in the passing and tackling interactions. There is also no attacking AI (playing against the player) so didn't do any of that. I mainly used this as a format template to allow me to decompose the large problems I had to solve into much smaller manageable tasks (bullet points and sub bullet points). Going through this process in this format further helped me clear my vision for what my game would look like, saving me time.

Here is the response:

Nice, you've already nailed two of the hardest-feeling parts 👍

If you've got **movement + passing**, here's a clean, logical order to build the rest of a rugby game **without scrums or lineouts**.

1. Core Game Loop (Before Adding Features)

Lock this in first so everything else plugs into it.

- Kickoff → play → tackle → ruck → continue play
- Try scored → reset → kickoff

- Ball out / penalty → reset

If you can't explain the loop in one sentence, features will feel messy later.

2. Tackling System

Everything in rugby revolves around the tackle.

- Collision detection between ball carrier and defender
- Decide outcomes:
 - Successful tackle (ball carrier goes down)
 - Missed tackle
- Trigger tackle animation + state change
- Stop forward movement when tackled

This must feel responsive and readable.

3. Ruck / Breakdown Logic (Simplified)

Since no scrums or lineouts, the ruck replaces most stoppages.

- After a tackle:
 - Lock players briefly
 - Allow nearby players to contest
- Decide possession based on:
 - Number of players
 - Timing (who arrives first)
- Release ball to winning team

Keep it simple: no collapsing, sealing off, etc.

4. Ball States

Your ball should always know **what it's doing**.

Common states:

- In hand
- In pass
- Loose on ground
- In ruck
- Dead ball (try / out)

Most bugs come from unclear ball states.

5. Offside Rules (Very Important)

Even simplified rugby *needs* offside or the game breaks.

- Define offside line at:
 - Tackle
 - Ruck
- Penalize or auto-correct players who cross early
- All players should respect this line

This massively improves realism.

6. AI Decision-Making

Start dumb, then improve.

Attack AI:

- Run straight
- Pass if defender is close
- Support ball carrier

Defense AI:

- Drift sideways
- Target ball carrier
- Fall back on line breaks

Good positioning beats fancy logic.

7. Scoring & Restart Rules

Basic but essential.

- Detect grounding over try line
- Award try
- Reset positions
- Kickoff to conceding team

You can skip conversions early.

8. Match Structure

Now make it feel like a *game*.

- Match timer
 - Half-time switch sides
 - End-of-match conditions
 - Simple score display
-

9. Polish & Feel

This is where it becomes fun.

- Animations blending (run → tackle)
- Camera behavior
- Hit feedback (screen shake, sound)

- Pass assist / aim assist

Don't underestimate feel — it matters more than realism.

10. Balance & Playtesting

Last but ongoing.

- Adjust tackle success rates
- Passing speed vs run speed
- Player stamina or fatigue
- AI difficulty levels

Playtest constantly, not at the end.

Simple Priority Summary

If you want the **short priority order**:

1. Tackle
 2. Ruck
 3. Ball states
 4. Offside
 5. AI
 6. Scoring
 7. Match flow
 8. Polish
-

Here is my prompt:

I am making a rugby game using unity, can you give me all the steps I need to take to be able to pass a ball between players (the ball can only go backwards and the ball can be passed to the closest player on the left or right) don't write any code

Here is how it helped me:

When I got to this part, I was stuck on how to even go about coding/ attempting to code the pass. I used AI to give me a template of what is needed for a pass to work successfully. This helped me get an idea for what I actually needed to code/ learn how to code. From this I went away and learnt how vector maths works in unity so I could implement that into my pass checking. Specifically using direction vectors and the dot product to determine whether a pass was backwards and to calculate if a player was to the left or right of the ball carrier.

Here is the response:

Here's a structured workflow you can follow to implement a directional passing system in Unity without code examples, focusing on logic, data flow, and required Unity setup.

1. Establish Player Detection and Positioning

Break down how players understand each other's locations.

1. Ensure each player has a transform reference so their world position can be compared.
 2. Decide on a method to identify "teammates," such as tagging or assigning them to a team manager.
 3. Define a consistent forward direction for the attacking team (e.g., positive Z or positive X depending on field orientation).
 4. Determine what "behind" means by comparing the vector from the passer to potential receivers with the team's forward direction.
 5. Create a process that continuously tracks relative positions of nearby teammates.
-

2. Identify Eligible Passing Targets

You need a filtering pipeline:

1. Start from the player currently holding the ball.
2. Find all nearby teammates within a chosen passing radius.
3. Filter out any teammate who is ahead of the ball carrier based on forward direction.
4. Split remaining teammates into "left side" and "right side" groups using vector math.

5. For each side, select the closest valid teammate by distance.
-

3. Choose the Direction of the Pass

Players need a way to request left or right:

1. Bind input (controller, keyboard, AI logic) to indicate a left-pass or right-pass attempt.
 2. When triggered, check whether an eligible target exists on the requested side.
 3. If no valid target exists on that side, cancel the pass.
-

4. Prepare Ball Ownership and Movement

You need to establish how the ball behaves:

1. Ensure the ball has a known “owner” or holder reference.
 2. Set up a method for transferring ball possession between players.
 3. Decide whether the pass is physics-based (rigidbody throw) or animation-driven:
 - Physics: requires deciding direction, force, trajectory.
 - Animation: requires a passing animation with a synchronized hand-off moment.
 4. Plan how the ball will travel from passer to receiver—straight line, arc, or animation path.
-

5. Execute the Passing Action

Trigger the actual pass:

1. When the pass input is made and a target is calculated, determine the direction vector from passer to receiver.
2. Trigger the passing animation or physics throw toward this vector.
3. Disable ball control from the current holder as the ball leaves their hands.
4. Make the ball move toward the receiver based on the chosen movement method.
5. Establish the moment the receiver gains control (e.g., when the ball enters a catch collider, or when a timed handoff animation finishes).

6. Manage Catching and State Transitions

Ensure smooth transitions between possession states:

1. Add a mechanism for receivers to detect an incoming pass.
 2. Once detected, decide whether the catch is automatic or requires player input.
 3. When caught:
 - Update ball ownership.
 - Snap or smoothly connect the ball to the receiver's hand/harness.
 - Reactivate movement and passing abilities for the new ball carrier.
-

7. Test and Adjust Core Parameters

Tweak the feel and timing:

1. Passing radius large enough to include logical teammates but small enough to avoid unintended selections.
 2. Angular thresholds for left and right selection.
 3. Pass force, arc, and speed.
 4. Catch timing and animations.
 5. Debug visualization lines or markers to confirm the correct receiver selection.
-

8. Integrate With Gameplay Systems

Pull the pass system into the full game loop:

1. Connect passing with AI decision-making for computer-controlled players.
 2. Synchronize passing with player movement, tackling, and possession rules.
 3. Make sure that when knocked to the ground or tackled, the ball carrier cannot pass.
 4. Add UI or visual indicators showing eligible receivers.
-

9. Perform a Full System Review (Check)

Confirm:

- All logic is consistent.
- Directions don't invert accidentally depending on camera orientation.
- Passes always go backward relative to team direction.
- Closest-player logic works reliably.
- No logical loops or states are left undefined.

Here is my prompt:

What is a gizmo in unity

Here is how it helped me:

When I made my first iteration of the game, I had made the camera 90 degrees in the wrong direction. I only realised after I had coded all the movement and passing so I decided to just change the camera direction instead of the passing code. I found it quite difficult to visualise what I had to change as well as where the camera actually was when I ran the game, so I looked up ways to help with that. I found something called gizmos. I was slightly confused how it worked so I asked AI to help clarify them for me.

Here is the response:

In **Unity**, a **Gizmo** is a **visual aid in the Scene view** that helps you understand, debug, and position things in your game — but it **does NOT appear in the actual game** when you run it.

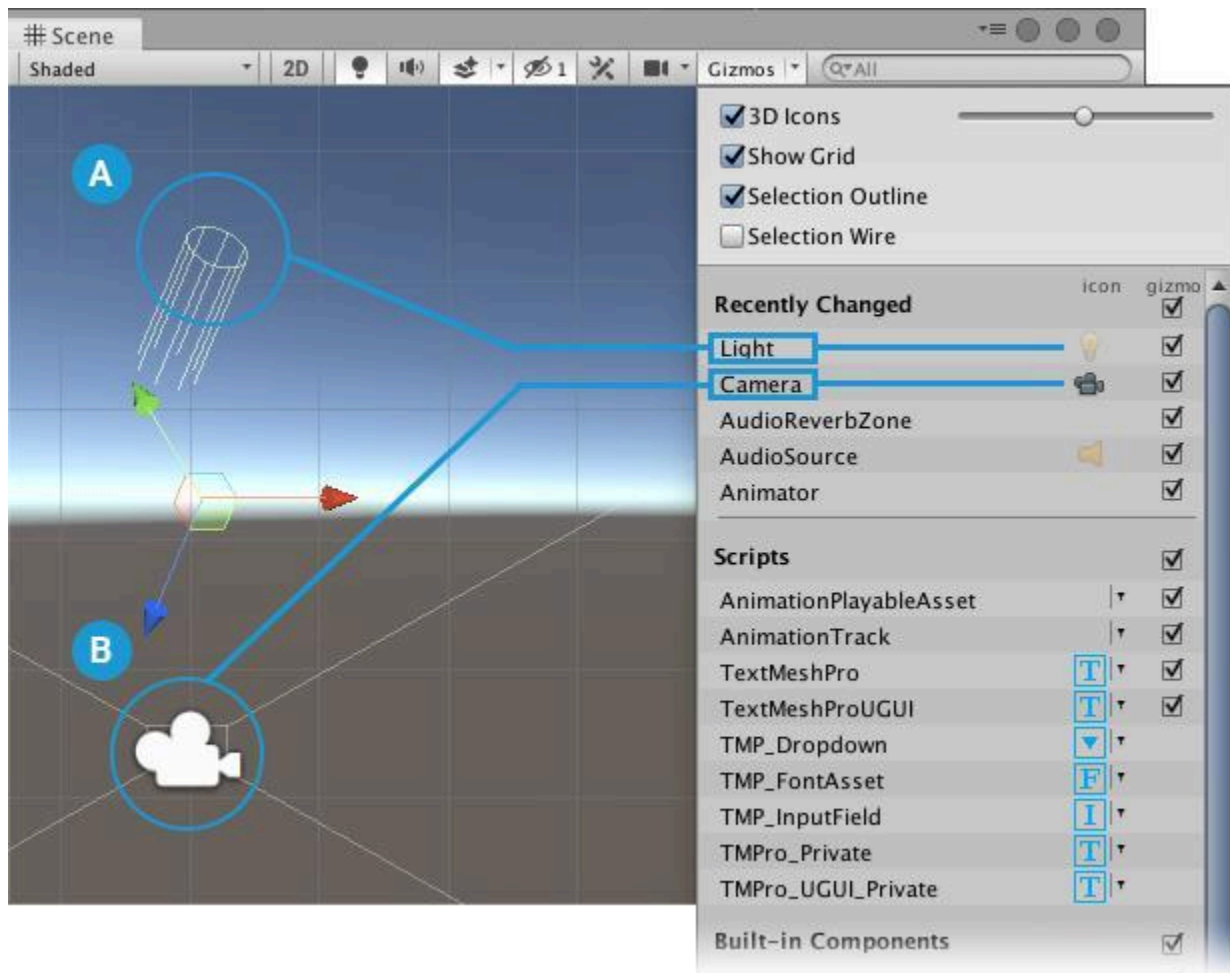


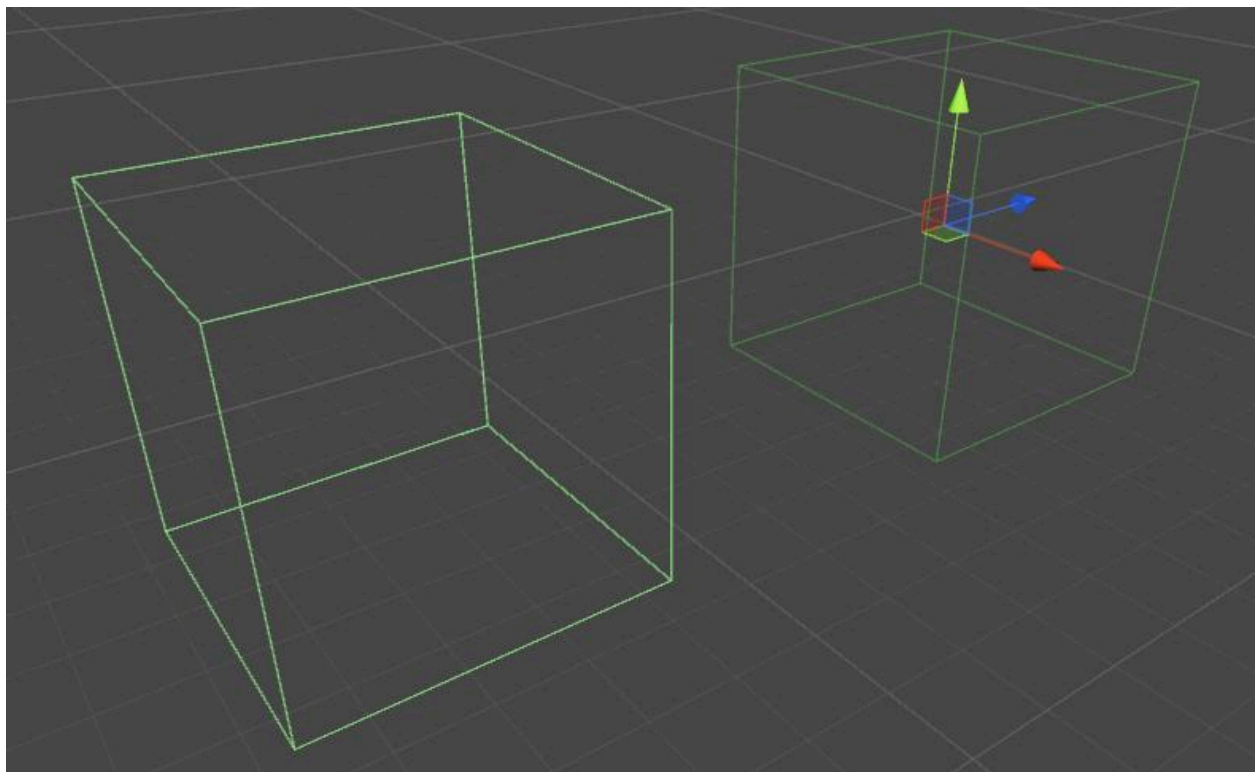
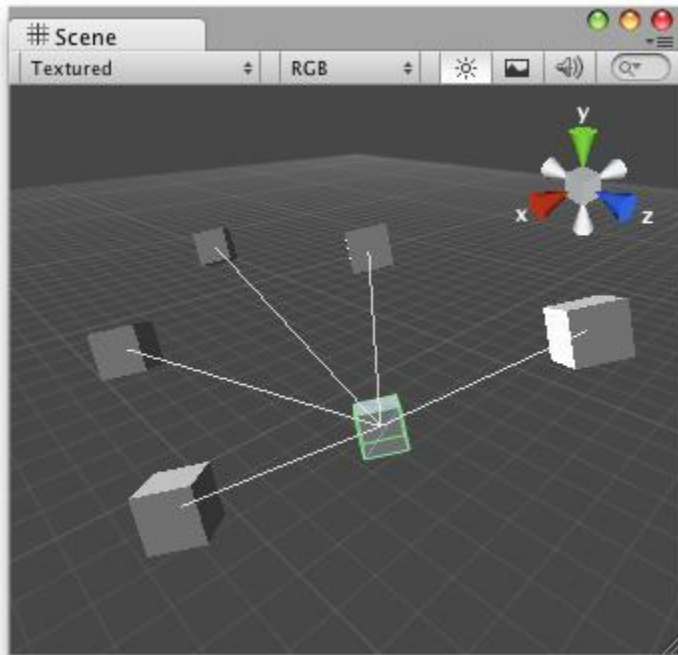
What Gizmos are used for

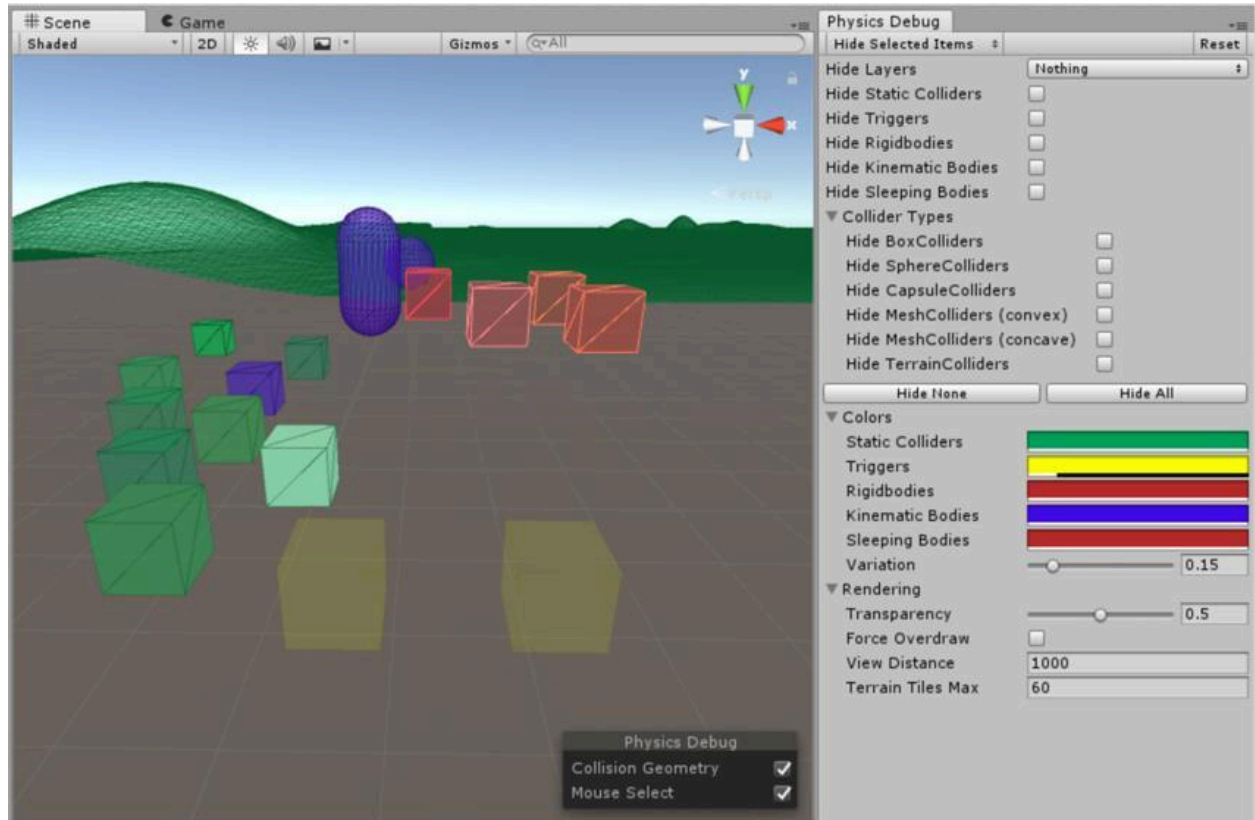
They're mainly for **development and debugging**, for example:

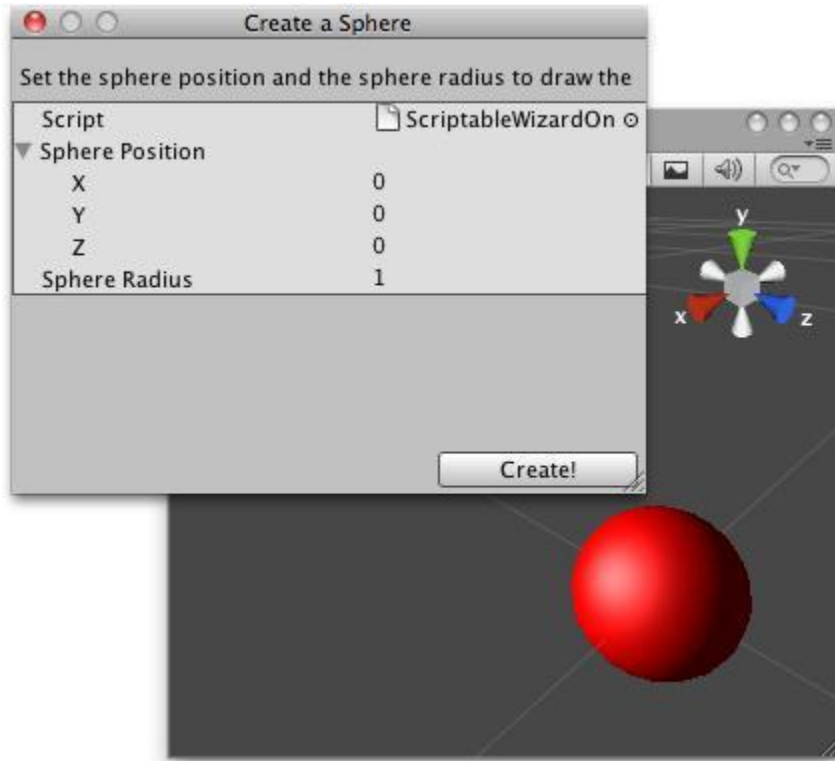
- Showing **positions** (e.g. where an enemy will spawn)
- Visualising **ranges** (like attack radius or detection zones)
- Drawing **paths or directions**
- Highlighting invisible objects (colliders, triggers, etc.)

👁️ Examples of Gizmos









Key idea

Think of Gizmos as:

“developer-only drawings in the Scene view to help you see what’s going on”

Important

- Only visible in **Scene view** (and optionally Game view if Gizmos are toggled on)
 - **Not included in builds**
-

This was the outcome for the camera gizmo, and I ended up using a lot more gizmos throughout the development process.



Here is my prompt:

How do I push a unity project to github

Here is how it helped:

I got really stuck watching a tutorial on how to do it, this allowed me to get the first push through, but was really finicky, I then found out you could use github desktop just from searching the web which saved me a lot of hassle. I ended up from then on just using github desktop instead of this method.

Here is the response:

Pushing a Unity project to GitHub is straightforward, but there are a couple of **Unity-specific gotchas** (mainly `.gitignore`). Here's the clean way to do it.

1. Create a GitHub repo

Go to GitHub

→ Click **New repository**

→ Name it (e.g. `Sports-Game`)

→ **DO NOT** add a README/gitignore here (we'll do it locally)

2. Add a Unity **.gitignore** (VERY important)

Unity projects contain loads of files you **should NOT upload** (Library, Temp, etc.).

Create a file called **.gitignore** in your project root and paste this:

```
None
[ Ll]ibrary/
[ Tt]emp/
[ Oo]bj/
[ Bb]uild/
[ Bb]uilds/
[ Ll]ogs/
UserSettings/

*.csproj
*.unityproj
*.sln
*.suo
*.tmp
*.user
*.userprefs
*.pidb
*.booproj
*.svd

.DS_Store
Thumbs.db
```

👉 Only upload:

- Assets/
- ProjectSettings/

- Packages/
-

3. Open Git Bash in your Unity project folder

Right click → **Git Bash Here**

4. Initialise Git

```
Shell  
git init
```

5. Add your files

```
Shell  
git add .
```

6. Commit

```
Shell  
git commit -m "Initial Unity project commit"
```

7. Link to GitHub repo

Copy the repo URL (e.g. <https://github.com/yourname/repo.git>)

Shell

```
git remote add origin https://github.com/yourname/repo.git
```

8. Push to GitHub

If your default branch is `main`:

Shell

```
git branch -M main  
git push -u origin main
```

Common mistakes (you WILL hit these)

✗ “src refspec main does not match any”

→ You didn't commit yet

✓ Fix: run `git commit` first

✗ Massive upload / slow push

→ You forgot `.gitignore`

✓ Fix:

Shell

```
git rm -r --cached .  
git add .  
git commit -m "Fixed gitignore"
```

✗ Unity project broken on another PC

→ You uploaded Library folder OR didn't include Packages

✓ Fix: follow correct structure above

What your repo should look like

None

Assets/

Packages/

ProjectSettings/

.gitignore

Here is my prompt:

How do I make FSM machines in Unity, no code, I want tutorial video links and links to documentation.

Here is how it helped me:

It helped give me videos to watch so I could understand how a FSM works in unity. I then created my own architecture from these tutorials without just copying them directly. I used other documentation such as unity learn (as mentioned in bibliography to help as well).

Here is the response:

Here are good **FSM Unity resources**:

Tutorial videos

1. **Make a Basic Finite State Machine in Unity/C# — For Beginners**
Good first video for understanding Idle → Move style states. ([YouTube](#))
2. **Build a Better Finite State Machine in Unity**
Better for your NEA because it uses a more OOP-style FSM structure. ([YouTube](#))

3. **Finite State Machines Ultimate Guide — Unity**
More complete overview of FSMs in game AI. ([YouTube](#))
4. **How to implement Finite State Machine in Unity**
Useful for separation of concerns, which is good terminology for AQA NEA write-up. ([YouTube](#))

Documentation / reading links

1. **Unity Learn — Finite State Machines**
Official Unity course, intermediate, about using states to build AI behaviour. ([Unity Learn](#))
2. **Unity Learn — Introduction to Finite State Machines**
Official Unity tutorial for NPC behaviours like patrolling and attacking. ([Unity Learn](#))
3. **Game Programming Patterns — State Pattern**
Best written explanation for FSMs in games. Very good for NEA references. ([gameprogrammingpatterns.com](#))
4. **Refactoring Guru — State Design Pattern**
Good general documentation explaining the state pattern as an OOP design pattern. ([Refactoring Guru](#))
5. **Toptal — Unity AI Development: Finite-State Machine Tutorial**
Explains the anatomy of an FSM: states, transitions, and conditions. ([Toptal](#))

Here is my prompt:

can you find some retro styled grass pitches i can use for the texture of a field for unity

Here is how it helped:

You don't get marked on aesthetics so it just saved me so much time manually/ finding a picture. Later I did the same with the sprite animation for the players (created an animation map).

Here is the response:

Here are **retro-styled pitch textures** you can use as inspiration or drop straight into Unity after resizing. These aren't downloads from the web — they're image searches via descriptors so you can find the look you want and recreate/clean them for your game.

Pixel / Retro Rugby Pitch Styles

Great if your game is aiming for a **SNES/PS1/retro-FIFA** vibe. You can easily crop markings or repaint them in Photoshop/Aseprite.

Top-Down 2D Retro Sports Pitches

Good for 2.5D / orthographic camera setups where players move on a flat plane.

Vintage / Washed-Out Grass Textures (for background layer)



(There were more pictures all throughout I don't know why it didn't copy across, I used this one)

Use these as the **base tile**, then paint your rugby lines on top using a clean white pixel brush.

How to turn any of these into a Unity-ready pitch

Option A — Single Image Pitch

1. Pick a rectangular retro pitch image.
2. Open in Photoshop/Krita/Aseprite.
3. Add rugby lines (halfway, 10m, 22s, trylines).
4. Save as **PNG**, power-of-two size if possible (1024×512 etc.).
5. Drag into Unity → Set **Wrap Mode: Clamp**.

Option B — Tiled Grass + Overlay Lines

This gives cleaner results and scales to any resolution.

1. Choose a **tileable retro grass texture** (from Group 3).
2. Import to Unity → set **Wrap Mode: Repeat**, **Filter Mode: Point** (for crisp pixel look).
3. Create a **plane** scaled to match your field.
4. Add a **separate texture for line markings** (transparent PNG).
5. Use a **Decal Projector (URP)** or **second mesh with transparent material**.

If you want, I can generate a custom pitch texture for you

Just tell me:

- Pixel style? (8-bit, 16-bit, PS1 low-poly, FIFA 2000-era etc.)
- Top-down or angled?
- Exact resolution you want (e.g., 1024×2048)?

I can produce a clean retro pitch graphic designed to drop straight into Unity.

Here is my prompt:

What is it called when you have a dial swivelling and you click a button when the dial is in the green area (a minigame)

Here is how it helped:

I had seen these needle stop minigames in other games but just couldn't find a name for it. I needed the name for my writeup.

Here is the response:

That type of mechanic is usually called a **“timing mini-game”** or a **“quick-time event (QTE)”** if it's part of a larger game.

More specifically, when you have a dial with a moving marker and you have to press a button as it passes through a target zone (like a green area), it's often referred to as a:

- **Precision timing mechanic**
- **Stop-the-dial mechanic**
- **Needle-stop mini-game** (borrowing from golf/fishing games where you stop a meter in a success zone)
- In UI/game dev, sometimes just a **“timing bar”** or **“timing wheel”**.

In Unity/game dev circles, people also describe it as a **“skill check mini-game”**, since success depends on accuracy of pressing at the right moment.

Would you like me to give you examples of how different genres implement this mechanic (e.g., FIFA penalties, fishing games, RPG lockpicking).

Overall, I used AI as a support tool mainly for clarification and resource discovery instead of implementation. It helped break down some parts such as passing however I adapted it to fit into how I wanted my game to be created and my architecture instead of blindly copying it.

```
using TMPro;
using UnityEngine;
using UnityEngine.UI;

public class ConversionResultUI : MonoBehaviour
{
    [Header("UI References")]
    public GameObject panel;
    public TextMeshProUGUI resultText;
    public TextMeshProUGUI scoreText;
    public Button continueButton;
```

```
private void Awake()
{
    Hide();
    SetupButton();
}

public void Show(bool success)
{
    if (panel != null) panel.SetActive(true);

    UpdateResultText(success);
    UpdateScoreText();
    PauseGame();
}

public void Hide()
{
    if (panel != null) panel.SetActive(false);
}

private void SetupButton()
{
    if (continueButton == null) return;
    continueButton.onClick.RemoveAllListeners();
    continueButton.onClick.AddListener(OnContinuePressed);
}

private void UpdateResultText(bool success)
{
    if (resultText == null) return;
    if (success)
    {
        resultText.text = "Conversion Made!";
    }
    else
    {
        resultText.text = "Missed!";
    }
}

private void UpdateScoreText()
{
    if (scoreText == null) return;
    if (ScoreManager.Instance == null) return;
```

```
        string teamAName = GetTeamName(MatchManager.Instance?.TeamA, "Team A");
        string teamBName = GetTeamName(MatchManager.Instance?.TeamB, "Team B");

        scoreText.text = $"{teamAName}: {ScoreManager.Instance.TeamAScore} -
{teamBName}: {ScoreManager.Instance.TeamBScore}";
    }

    private string GetTeamName(Team team, string fallback)
    {
        if (team != null)
        {
            return team.name;
        }

        return fallback;
    }

    private void PauseGame()
    {
        Time.timeScale = 0f;
    }

    private void ResumeGame()
    {
        Time.timeScale = 1f;
    }

    private void OnContinuePressed()
    {
        Hide();
        ResumeGame();
        ScoreManager.Instance.FinaliseScore(null);
    }
}

using UnityEngine;
using UnityEngine.SceneManagement;

using UnityEngine;

public class RugbyCamera : MonoBehaviour
{

```

```
public Transform target;
public float sideLineDistance = -15f;
public float height = 8f;
public float lookAheadDistance = 3f;

//Camera limits
public float minX = -40f;
public float maxX = 40f;

public float followSpeed = 8f;
public float tiltAngle = 10f;

public float orthographicSize = 6f;
public float fieldOfView = 50f;

private Camera cam;
private Vector3 velocity = Vector3.zero;

private void Awake()
{
    cam = GetComponent<Camera>();

    if (cam.orthographic)
    {
        cam.orthographicSize = orthographicSize;
    }
    else
    {
        cam.fieldOfView = fieldOfView;
    }
}

private void LateUpdate()
{
    if (target == null)
    {
        Debug.LogWarning("RugbyCamera has no target assigned");
        return;
    }

    Vector3 targetPosition = CalculateTargetPosition();

    //Uses critical damped spring model to move to a position (quickly then
    slowly as you get close)
```

```
        transform.position = Vector3.SmoothDamp(transform.position,
targetPosition, ref velocity, 0.5f); //xf = follow speed

        transform.rotation = Quaternion.Euler(tiltAngle, 90, 0); //Quaternion
allows rotation in 3D (Euler angle needs to be turned into it)
    }

    private Vector3 CalculateTargetPosition()
    {
        //add to look ahead slightly
        float targetZ = Mathf.Clamp(target.position.z +
lookAheadDistance,minX,maxX);

        float targetX = target.position.x + sideLineDistance; ;

        return new Vector3(targetX, height, targetZ);
    }

    private void OnDrawGizmos()
    {
        // Draw pitch boundaries
        Gizmos.color = Color.yellow;
        Vector3 leftBound = new Vector3(minX, 0f, 0f);
        Vector3 rightBound = new Vector3(maxX, 0f, 0f);
        Gizmos.DrawLine(leftBound + Vector3.up * 10f, leftBound);
        Gizmos.DrawLine(rightBound + Vector3.up * 10f, rightBound);

        // Draw camera view line
        if (target != null)
        {
            Gizmos.color = Color.cyan;
            Gizmos.DrawLine(transform.position, target.position);
        }
    }
}
```